

**Probabilistic and parallel algorithms for centroidal Voronoi tessellations
with application to meshless computing and numerical analysis
on surfaces**

by

Lili Ju

A dissertation submitted to the graduate faculty
in partial fulfillment of the requirements for the degree of
DOCTOR OF PHILOSOPHY

Major: Applied Mathematics

Program of Study Committee:
Max D. Gunzburger, Major Professor
Glenn R. Luecke
Gurpur M. Prabhu
Steven L. Hou
Janet S. Peterson

Iowa State University

Ames, Iowa

2002

Copyright © Lili Ju, 2002. All rights reserved.

Graduate College
Iowa State University

This is to certify that the Doctoral dissertation of
Lili Ju
has met the dissertation requirements of Iowa State University

Committee Member

Committee Member

Committee Member

Committee Member

Major Professor

For the Major Program

ABSTRACT

Centroidal Voronoi tessellations (CVT) are Voronoi tessellations of a region such that the generating points of the tessellations are also the centroids of the corresponding Voronoi regions. Such tessellations are of use in very diverse applications, including data compression, clustering analysis, cell biology, territorial behavior of animals, optimal allocation of resources, and grid generation. A detailed review is given in Chapter 1.

In Chapter 2, some probabilistic methods for determining centroidal Voronoi tessellations and their parallel implementation on distributed memory systems are presented. The results of computational experiments performed on a CRAY T3E-600 system are given for each algorithm. These demonstrate the superior sequential and parallel performance of a new algorithm we introduce.

Then, new algorithms are presented in Chapter 3 for the determination of point sets and associated support regions that can then be used in meshless computing methods. The algorithms are probabilistic in nature so that they are totally meshfree, i.e., they do not require, at any stage, the use of any coarse or fine boundary conforming or superimposed meshes. Computational examples are provided that show, for both uniform and nonuniform point distributions, that the algorithms result in high-quality point sets and high-quality support regions.

The extensions of centroidal Voronoi tessellations to general spaces and sets are also available. For example, tessellations of surfaces in an Euclidean space may be considered. In Chapter 4, a precise definition of such constrained centroidal Voronoi tessellations (CCVT's) is given and a number of their properties are derived, including

their characterization as minimizers of an “energy”. Deterministic and probabilistic algorithms for the construction of CCVT’s are presented and some analytical results for one of the algorithms are given. Some computational examples are provided which serve to illustrate the high quality of CCVT point sets. CCVT point sets are also applied to polynomial interpolation and numerical integration on the sphere.

Finally, some conclusions are given in Chapter 5.

ACKNOWLEDGEMENTS

It is my real pleasure to have Prof. Max D. Gunzburger as my major professor. I deeply appreciate the very effective guidance, inspiring discussions and generous support from him and Prof. Qiang Du of Penn State University, which led to my successful study at Iowa State University.

I would also like to thank all the members in my POS committee: Prof. Glenn R. Luecke, Prof. Gurpur M. Prabhu, Prof. Steven L. Hou and Prof. Janet S. Peterson for their expertise, cooperation, time and effort.

I would like to thank my wife, Yibin Wang for her love, encouragement and support that make me get this work through.

This work was supported by the National Science Foundation under grant number CCR-998830. I would like to thank CRAY Inc. for allowing us to use their CRAY T3E-600 computer located in Eagan, MN. I am also especially grateful to Prof. Glenn R. Luecke because I learned a lot about parallel programming when I worked in his research group. I also would like to thank R. J. Renka for providing his elegant software package STRIPACK for constructing Voronoi diagrams on spheres.

Finally, my thanks also go to those who, directly or indirectly, helped me to finish my thesis.

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGEMENTS	v
CHAPTER 1. OVERVIEW	1
1.1 Introduction	1
1.2 Applications	5
1.2.1 Data compression in image processing	5
1.2.2 Optimal quadrature rules	10
1.2.3 Optimal representation, quantization, and clustering	11
1.2.4 Finite difference schemes having optimal truncation errors	14
1.2.5 Optimal placement of resources	16
1.2.6 Cell division	17
1.2.7 Territorial behavior of animals	19
1.3 Some results about centroidal Voronoi tessellations and their minimiza- tion properties	21
1.3.1 Results involving centroidal Voronoi tessellations as minimizers	21
1.3.2 Existence of a minimizer	22
1.3.3 Equal partition of energy in one dimension	22
CHAPTER 2. PROBABILISTIC ALGORITHMS FOR CENTROIDAL VORONOI TESSELLATIONS AND THEIR PARALLEL IMPLE- MENTATIONS	25

2.1	Determining random numbers from nonuniform densities	25
2.2	Methods for determining centroidal Voronoi tessellations	27
2.2.1	MacQueen's method	27
2.2.2	Lloyd's method	30
2.2.3	A new probabilistic method	31
2.3	Parallel implementations	36
2.3.1	Parallel MacQueen's methods	37
2.3.2	Parallelization of the new algorithm	40
2.3.3	Two-dimensional computational experiments	43
 CHAPTER 3. GENERATION OF POINT SETS AND SUPPORT		
	REGIONS FOR MESHLESS COMPUTING	54
3.1	Introduction	54
3.2	Point placement using centroidal Voronoi tessellations	56
3.3	Sphere supports and overlaps	58
3.4	Numerical Experiments	61
3.4.1	Uniform point distributions	62
3.4.2	Nonuniform point distributions	65
3.4.3	Non-convex domains	65
3.4.4	Point placement under constraints	67
3.4.5	Three-dimensional domains	67
 CHAPTER 4. CONSTRAINED CENTROIDAL VORONOI TESSEL-		
	LATIONS ON GENERAL SURFACES	79
4.1	Introduction	79
4.2	Constrained centroidal Voronoi tessellations	80
4.3	Algorithms for determining CCVT's	85
4.3.1	Some probabilistic and deterministic methods for determining CCVT's	86

4.3.2	Some results about Lloyd's method	90
4.4	Computational examples	93
4.4.1	CCVT's on the surface of the unit sphere	94
4.4.2	CCVT's on a developable surface	95
4.4.3	CCVT's on the surface of a torus	96
4.5	CCVT's for interpolation and quadrature on the sphere	97
CHAPTER 5. CONCLUSIONS AND CURRENT WORK		104

LIST OF TABLES

2.1	Maximum running times and average energies over four realizations of MacQueen's method vs. number of iterations for three density functions.	29
2.2	Maximum running times and average energies over four realizations of Algorithm 3 for a fixed number (3,200,000) of random points sampled.	35
2.3	Average running times and average energies over four realizations of Algorithm 3 for a fixed number (400) of iterations.	36
2.4	Maximum running times and average energies over four realizations of Algorithm 5 for a fixed number of iterations (20,000) and a fixed number (3,200,000) of random points sampled.	40
2.5	Average running times and average energies over four realizations of Algorithm 6 for a fixed number (400) of iterations and a fixed number (3,200,000) of random points sampled;	43
2.6	Maximum running times for four realizations of Algorithms 5 and 6 for a fixed number (16) of processors and a fixed number (9.6×10^6) of random points sampled.	45
3.1	Densities of nonzero entries of the characteristic matrices of coverings of square and circular domains with uniform point distributions.	63

3.2	Densities of nonzero entries of the characteristic matrices of coverings of square and circular domains with nonuniform point distributions.	69
4.1	Energies of 256-point constrained centroidal Voronoi tessellations of a sphere for three density functions.	95

LIST OF FIGURES

1.1	On the left, the Voronoi regions corresponding to 10 randomly selected points in a square; the density function is a constant. The dots are the Voronoi generators and the circles are the centroids of the corresponding Voronoi regions. Note that the generating points and the centroids do not coincide. On the right, a 10-point centroidal Voronoi tessellation of a square; the density function is a constant. The dots are simultaneously the generating points for the Voronoi tessellation and the centroids of the Voronoi regions.	2
1.2	Two centroidal Voronoi tessellations of a square. The points $\mathbf{z}_1$ and $\mathbf{z}_2$ are the centroids of the rectangles on the left or of the triangles on the right.	3
1.3	An 8-bit monochrome picture (top-left) and the compressed 3-bit images by the Monte Carlo simulation (top-right), the centroidal Voronoi algorithm (bottom-left), and the centroidal Voronoi algorithm with dithering (bottom-right).	8

1.4	A top view photograph, using a polarizing filter, of the territories of the male <i>Tilapia mossambica</i> ; each is a pit dug in the sand by its occupant. The boundaries of the territories, the rims of the pits, form a pattern of polygons. The breeding males are the black fish which range in size from about 15 to 20cm. The gray fish are the females, juveniles, and non-breeding males. The fish with a conspicuous spot in its tail, in the upper right corner, is a <i>Cichlasoma maculicauda</i> . Photograph and caption by permission of Academic Press, London.	20
2.1	Energy vs. number of iterations for MacQueen's method. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$. . .	30
2.2	A visual description of Algorithm 3. The open circles denote the current positions of the generators and the dashed lines denote their corresponding Voronoi tessellation. The small squares denote the points sampled during a single iteration of the algorithm. These are grouped according to which Voronoi cell they belong to and then all sample points in each cell are averaged to produce the filled circles. A triangle denotes the new position of a generator which is determined by taking a linear combination of the old position (the open circle) and the local sample average (the filled circle). Note that since no points are sampled in the Voronoi region for z_2 , the position of that generator remains unchanged. The Voronoi regions as depicted only for illustrative purposes; Algorithm 3 does not require the construction of the Voronoi tessellation.	46

2.3	Energy vs. number of iterations for Algorithm 3 with $q = 8000$ and $\{\alpha_1 = 0, \alpha_2 = 1, \beta_1 = 0, \beta_2 = 1\}$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$	47
2.4	Speed up of Algorithm 5 with $q = ps = 160$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$	47
2.5	Speed up of Algorithm 6 with $q = ps = 8,000$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$	47
2.6	Two-dimensional centroidal Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ found by the deterministic Lloyd's method (Algorithm 2); top-left: $\rho(x, y) = 1$; top-left: $\rho(x, y) = e^{-10(x^2+y^2)}$; bottom: $\rho(x, y) = e^{-20(x^2+y^2)} + 0.05 \sin^2(\pi x) \sin^2(\pi y)$	48
2.7	Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = 1$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).	49
2.8	Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = e^{-10(x^2+y^2)}$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).	50

- 2.9 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = e^{-20(x^2+y^2)} + 0.05 \sin^2(\pi x) \sin^2(\pi y)$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d). 51
- 2.10 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$ with the density function $\rho(x, y) = 1$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d). 52
- 2.11 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$ with the density function $\rho(x, y) = e^{-5(1-x^2-y^2)}$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d). 53

3.1	The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top), (2,3) Halton sequence (middle), and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	70
3.2	The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top), (2,3) Halton sequence (middle), and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	71
3.3	The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-3(x^2+y^2)}$	72
3.4	The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-1.5(2+x+y)}$	72
3.5	The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-(1+x)^2}$	73

3.6	The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-4(x^2+y^2)}$	73
3.7	The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-3(1-x^2-y^2)}$	74
3.8	The sets of 128 points in a non-convex domain (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	74
3.9	The sets of 128 points in a non-convex domain (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	75
3.10	The sets of 256 points in a domain with a slit (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	75

3.11	The sets of 256 points in a domain with a slit (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a nonuniform density function.	76
3.12	The sets of 128 points in a square (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	76
3.13	The sets of 128 points in a circle (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.	77
3.14	The sets of 128 points in a circle (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $\rho(x, y) = e^{-2.5(1+y)}$	77
3.15	The sets of 512 points in a three-dimensional non-convex domain for the Monte Carlo (left) and centroidal Voronoi tessellation (right) point selection methods for a uniform density function. .	78
3.16	The sets of 512 points in a three-dimensional non-convex domain for the Monte Carlo (left) and centroidal Voronoi tessellation (right) point selection methods for a non uniform density function.	78

4.1	Voronoi diagrams of 256 generators on the surface of unit sphere. Left: Monte Carlo point set; right: constrained CVT point set; top: $\rho(x, y, z) = 1$; middle $\rho(x, y, z) = e^{-6.0z^2}$; bottom: $\rho(x, y, z) = e^{-3.0(1-z)^2}$	101
4.2	Voronoi diagrams of 256 generators on a developable surface. Left: Monte Carlo point set; right: Constrained CVT point set; top: $\rho(x, y, z) = 1$; bottom: $\rho(x, y, z) = e^{-20.0x^2}$	102
4.3	Voronoi diagrams of 256 generators on a torus. Left: Monte Carlo point set; right: Constrained CVT point set; top: $\rho(x, y, z) = 1$; bottom: $\rho(x, y, z) = e^{-5.0 y }$	102
4.4	Mesh norm of constrained CVT generators vs. degree of the polynomial n ; the number of generators $k_n = (n + 1)^2$	103
4.5	Global polynomial interpolation norms using constrained CVT generators vs. degree of the polynomial n ; the number of gener- ators $k_n = (n + 1)^2$	103
4.6	Maximum and minimum value of the quadrature weights (scaled by $4\pi/k_n$) when the quadrature points are chosen to be con- strained CVT generators vs. degree of the polynomial n ; the number of generators is $k_n = (n + 1)^2$	103

CHAPTER 1. OVERVIEW

In this chapter, we first review some definitions and remarks about centroidal Voronoi tessellations and their generalizations; then, in §1.2, we discuss some applications; finally, some results and properties related to these tessellations are given in §1.3. Most of the content of this chapter can be found in (13).

1.1 Introduction

Given an open set $\Omega \subset \mathbb{R}^N$, the set $\{V_i\}_{i=1}^k$ is called a *tessellation* of Ω if $V_i \subset \overline{\Omega}$ for $i = 1, \dots, k$, $V_i \cap V_j = \emptyset$ for $i \neq j$, and $\cup_{i=1}^k \overline{V_i} = \overline{\Omega}$. Let $|\cdot|$ denote the Euclidean norm on $\mathbb{R}^N$. Given an open set $\Omega \subset \mathbb{R}^N$ and a set of points $\{\mathbf{z}_i\}_{i=1}^k$ belonging to $\overline{\Omega}$, the *Voronoi* region $\widehat{V}_i$ corresponding to the point $\mathbf{z}_i$ is defined by

$$\widehat{V}_i = \{ \mathbf{x} \in \Omega \mid |\mathbf{x} - \mathbf{z}_i| < |\mathbf{x} - \mathbf{z}_j| \text{ for } j = 1, \dots, k, j \neq i \}. \quad (1.1)$$

Clearly, we have

$$\widehat{V}_i \cap \widehat{V}_j = \emptyset \text{ for } i \neq j \quad \text{and} \quad \cup_{i=1}^k \widehat{V}_i = \overline{\Omega}.$$

The set $\{\widehat{V}_i\}_{i=1}^k$ is referred to as a *Voronoi tessellation* or *Voronoi diagram* of Ω , the members of the set $\{\mathbf{z}_i\}_{i=1}^k$ are referred to as *generating points* or *generators*, and each $\widehat{V}_i$ is referred to as the *Voronoi region* or *Voronoi cell* corresponding to $\mathbf{z}_i$. It is well-known that the Voronoi regions are polyhedra. These tessellations and their dual tessellations (in $\mathbb{R}^2$, the Delaunay triangulations) are very useful in a variety of applications. For a comprehensive treatment of Voronoi diagrams, see (52; 13).

Given a density function $\rho(\mathbf{y}) \geq 0$ defined on $\overline{\Omega}$, then for each Voronoi region $\widehat{V}_i$, we could define the *mass centroid* $\mathbf{z}_i^*$ of $\widehat{V}_i$ by

$$\mathbf{z}_i^* = \frac{\int_{\widehat{V}_i} \mathbf{y} \rho(\mathbf{y}) d\mathbf{y}}{\int_{\widehat{V}_i} \rho(\mathbf{y}) d\mathbf{y}} \quad \text{for } i = 1, \dots, k \quad (1.2)$$

The tessellation defined by (1.1) is called a *centroidal Voronoi tessellation* if and only if

$$\mathbf{z}_i = \mathbf{z}_i^*, \quad \text{for } i = 1, \dots, k \quad (1.3)$$

i.e., the points $\mathbf{z}_i$ that serve as generators for the Voronoi regions $\widehat{V}_i$ are themselves the mass centroids of those regions. This situation is quite special in the sense that, given an arbitrary set of points $\{\mathbf{z}_i\}_{i=1}^k$ belonging to some region $\overline{\Omega} \subset \mathbb{R}^N$, these points will not in general be the centroids of their associated Voronoi regions. See Fig. 1.1 for an illustration in two dimensions.

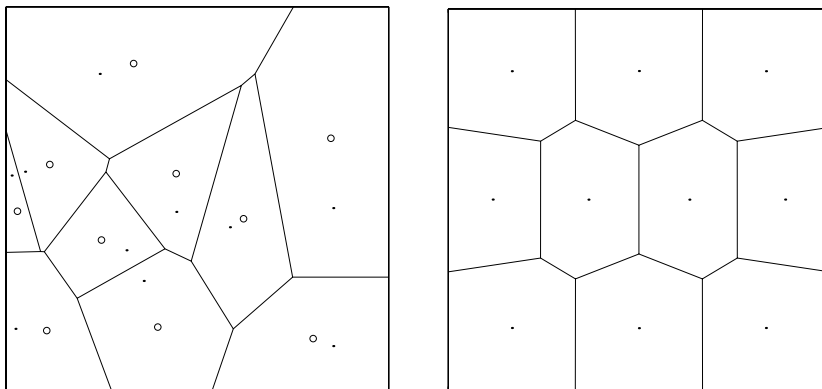


Figure 1.1 On the left, the Voronoi regions corresponding to 10 randomly selected points in a square; the density function is a constant. The dots are the Voronoi generators and the circles are the centroids of the corresponding Voronoi regions. Note that the generating points and the centroids do not coincide. On the right, a 10-point centroidal Voronoi tessellation of a square; the density function is a constant. The dots are simultaneously the generating points for the Voronoi tessellation and the centroids of the Voronoi regions.

As a result, one is left with the following construction problem:

Given:

a region $\Omega \subset \mathbb{R}^N$, a positive integer k , and a density function $\rho(\mathbf{y})$,
defined for $\mathbf{y} \in \Omega$;

find:

k points $\mathbf{z}_i, i = 1, \dots, k$, belonging to $\overline{\Omega}$ and k regions $V_i \subset \overline{\Omega}, i = 1, \dots, k$, that tessellate Ω

such that *simultaneously*

the regions $\{V_i\}_{i=1}^k$ are Voronoi regions for the points $\{\mathbf{z}_i\}_{i=1}^k$ and
the points $\{\mathbf{z}_i\}_{i=1}^k$ are the mass centroids of the regions $\{V_i\}_{i=1}^k$.

It is worth noting that the solution of this problem is in general not unique. For example, consider the case of $N = 2$, $\Omega \subset \mathbb{R}^2$ a square, and $\rho \equiv 1$. Two solutions of the above problem are depicted in Fig. 1.2; other solutions may be obtained through rotations.

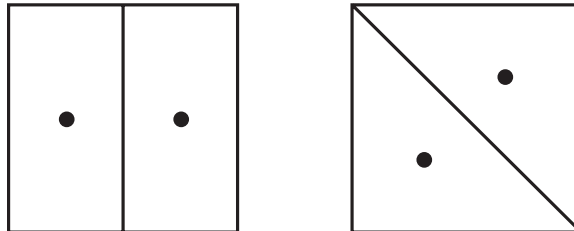


Figure 1.2 Two centroidal Voronoi tessellations of a square. The points $\mathbf{z}_1$ and $\mathbf{z}_2$ are the centroids of the rectangles on the left or of the triangles on the right.

The concept of centroidal Voronoi tessellations also can be applied to the discrete case and to other metrics. We first consider the case where, instead of being given a region $\Omega \subset \mathbb{R}^N$, we are given a *discrete* set of points $W = \{\mathbf{y}_i\}_{i=1}^m$ belonging to $\mathbb{R}^N$. A set $\{V_i\}_{i=1}^k$ is a tessellation of W if $V_i \subset W$ for $i = 1, \dots, k$, $V_i \cap V_j = \emptyset$ for $i \neq j$, and $\cup_{i=1}^k V_i = W$. Given a set of points $\{\mathbf{z}_i\}_{i=1}^k$ belonging to $\mathbb{R}^N$, Voronoi sets are now

defined by

$$\widehat{V}_i = \{ \mathbf{x} \in W \mid |\mathbf{x} - \mathbf{z}_i| \leq |\mathbf{x} - \mathbf{z}_j| \text{ for } j = 1, \dots, k, j \neq i \} \quad (1.4)$$

where equality holds only for $i < j$.

Note that other tie-breaking rules for points equidistant to two or more of the $\mathbf{z}_i$'s can also be used. Also, given a density function $\rho(\mathbf{y}) \geq 0$, defined for $\mathbf{y} \in W$, the mass centroid $\mathbf{z}^*$ of any set $V \subset W$ is now defined by

$$\sum_{\mathbf{y} \in V} \rho(\mathbf{y}) |\mathbf{y} - \mathbf{z}^*|^2 = \inf_{\mathbf{z} \in V^*} \sum_{\mathbf{y} \in V} \rho(\mathbf{y}) |\mathbf{y} - \mathbf{z}|^2, \quad (1.5)$$

where the sums extend over the points belonging to V and V^* can be taken to be V or it can be an even larger set like $\mathbb{R}^N$. In the statistical and vector quantization literature, see, e.g., (26) and (21), respectively, discrete centroidal Voronoi tessellations are often related to optimal *k-means clusters*, and Voronoi regions and centroids are referred to as *clusters* and *cluster centers*, respectively. More discussion of this topic is provided in §1.2.3.

The notions of Voronoi regions and centroids, and therefore, of centroidal Voronoi regions, may be generalized to more abstract spaces and to metrics other than the Euclidean ℓ^2 norm. For example, of interest to us will be

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_{\ell^p}^p. \quad (1.6)$$

For another example, let $d(\mathbf{x}, \mathbf{y})$ denote a distance function induced by a norm which is equivalent to the ℓ^2 norm on $\mathbb{R}^N$. Let $\prec$ be an order of $\mathbb{R}^N$. Then, Voronoi tessellations of Ω , with respect to this metric, are defined by

$$\begin{aligned} \widehat{V}_i = \{ \mathbf{x} \in \Omega \mid & d(\mathbf{x}, \mathbf{z}_i) < d(\mathbf{x}, \mathbf{z}_j) \text{ } j = 1, \dots, k, j \neq i \text{ and} \\ & d(\mathbf{x}, \mathbf{z}_i) = d(\mathbf{x}, \mathbf{z}_j) \text{ } j \prec i \} . \end{aligned} \quad (1.7)$$

Notice that, without the inclusion of the part $d(\mathbf{x}, \mathbf{z}_i) = d(\mathbf{x}, \mathbf{z}_j)$, $\widehat{V}_i$ does not necessarily provide a partition of Ω as there could be non-empty boundaries. Using a partial order

one may decide how portions of boundaries with non-empty interior get assigned to specific Voronoi regions. Such a partition, in general, only guarantees that the Voronoi regions are star-shaped (with respect to the metric) (41). Note that if one considers the interior of the closure, then

$$\widehat{V}_i = \{ \mathbf{x} \in \Omega \quad ; \quad d(\mathbf{x}, \mathbf{z}_i) \leq d(\mathbf{x}, \mathbf{z}_j) \quad j = 1, \dots, k, j \neq i \}. \quad (1.8)$$

In this case, the regions may overlap but they remain convex.

A generalized definition of the centroid $\mathbf{z}^*$ of a region V with respect to a density function $\rho(\cdot)$ might be defined as

$$\int_V \rho(\mathbf{y}) d(\mathbf{z}^*, \mathbf{y}) d\mathbf{y} = \inf_{\mathbf{z} \in V^*} \int_V \rho(\mathbf{y}) d(\mathbf{z}, \mathbf{y}) d\mathbf{y}. \quad (1.9)$$

Here, V^* can be the closure of V or the region defined by (1.8) or even $\mathbb{R}^N$ so long as the existence of $\mathbf{z}^*$ can be guaranteed either by the compactness or the convexity of the region and the convexity of the functional. A similar extension is discussed in Chapter 4. Centroidal Voronoi tessellations of Ω can again be defined by (1.3).

The notion of Voronoi regions may be extended to weighted Voronoi regions (24) and may also be extended to more abstract spaces and the generators can also be lines, areas, or other more abstract objects (52). The metrics need not be induced by a norm as well.

1.2 Applications

In this section, we discuss a number of applications for centroidal Voronoi tessellations.

1.2.1 Data compression in image processing

The first application is to data compression. We will use a context from image processing; however, the basic ideas and principles are valid for many other data compression

applications.

The setting is as follows. We have a color picture composed of many pixels, each of which has an associated color. Each of the colors is a combination of basic, e.g., primary, colors. Let the components of $\mathbf{y}$ represent a possible combination of the basic colors and let $\rho(\mathbf{y})$ denote the number of times the particular combination $\mathbf{y}$ occurs in the picture. We let W denote the set of admissible colors, i.e., the set of allowable combinations of the primary colors. In a given picture, there may be many different colors. For example, there may be on the order of 10^6 pixels in a picture, with each pixel potentially having a different color. Our task is to approximate the picture by replacing the color at each pixel by one from a smaller set of colors $\{\mathbf{z}_i\}_{i=1}^k$, where each $\mathbf{z}_i \in W$. For related discussions, we refer to (35; 29; 37). In particular, a discussion of different clustering algorithms in image compression applications can be found in (35).

A natural question is, how does one effect the assignment of the colors in the original picture to the colors in the set $\{\mathbf{z}_i\}_{i=1}^k$? The shell of such an algorithm is given as follows.

Given:

the set of admissible colors W , a positive integer k , and a density function $\rho(\mathbf{y})$, defined for $\mathbf{y} \in W$;

choose:

k colors $\mathbf{z}_i$, $i = 1, \dots, k$, belonging to W and k subsets $V_i \subset W$, $i = 1, \dots, k$, that tessellate W ;

then:

if $\mathbf{y}$ is a color appearing in the original picture and $\mathbf{y} \in V_i$, replace $\mathbf{y}$ with $\mathbf{z}_i$.

The definition of the above algorithm is completed once one specifies how to choose the sets $\{\mathbf{z}_i\}_{i=1}^k$ and $\{V_i\}_{i=1}^k$. Clearly, one would want to choose these sets so that the

approximate picture using only k colors is a “good” approximation to the given picture that contains many more colors.

In a straightforward way, one may use a *Monte Carlo* method, using $\rho(\mathbf{y})$ (suitably normalized) as a probability density, to choose the set of approximating colors $\{\mathbf{z}_i\}_{i=1}^k$. Once this set is determined, it is natural to choose the assignment sets $\{V_i\}_{i=1}^k$ to be the associated Voronoi sets. Unfortunately, even with $k = 256$, the approximate pictures produced in this manner are not always satisfactory.

A better approximate picture is found if the sets $\{\mathbf{z}_i\}_{i=1}^k$ and $\{V_i\}_{i=1}^k$ are chosen so that the functional

$$\mathcal{E}((\mathbf{z}_i, V_i), i = 1, \dots, k) = \sum_{i=1}^k \sum_{\mathbf{y}_j \in V_i} \rho(\mathbf{y}_j) |\mathbf{y}_j - \mathbf{z}_i|^2 \quad (1.10)$$

is minimized over all possible sets of k points belonging to $W = \{\mathbf{y}_i\}_{i=1}^m$ and all possible tessellations of W into k regions $V_i, i = 1, \dots, k$. Note that no *a priori* relation between the points $\mathbf{z}_i$ and subsets V_i is assumed. The motivation for minimizing $\mathcal{E}$ is clear: one is then minimizing the sum of the squares of the distances, in the color space W and weighted by the density function $\rho(\mathbf{y})$, between the colors $\mathbf{y}_j$ in the picture and the reduced color set $\mathbf{z}_i$. It is important to note that the minimization process is over both all possible subsets $\{\mathbf{z}_i\}_{i=1}^k$ of reduced colors and, independently, over all possible subsets $\{V_i\}_{i=1}^k$ that tessellate W , with the only restriction being that both sets have cardinality k .

It is not hard to show that $\mathcal{E}$ given by (1.10) is minimized when the V_i 's are the Voronoi sets for the $\mathbf{z}_i$'s and, *simultaneously*, the $\mathbf{z}_i$'s are the mass centroids of the V_i 's, in the sense of (1.3) and (1.4), respectively. In other words, the $\mathbf{z}_i$'s and V_i 's form a *centroidal Voronoi tessellation* with respect to the density function $\rho(\mathbf{y})$. Note that the centroidal Voronoi property is a necessary (but in general not sufficient) optimality condition.

The same principle can be applied to the compression of monochrome pictures. In

Fig. 2.5, we give an original 8-bit monochrome picture (top-left), an approximate 3-bit compressed image determined by a Monte Carlo algorithm (top-right), and an approximate 3-bit compressed image determined by a centroidal Voronoi algorithm (bottom-left). Clearly, the centroidal Voronoi algorithm results in a better approximate picture. (Of course, there are other image-data compression algorithms that are more effective than the Monte Carlo algorithm; again, see, e.g., (35) for discussions and comparisons.)



Figure 1.3 An 8-bit monochrome picture (top-left) and the compressed 3-bit images by the Monte Carlo simulation (top-right), the centroidal Voronoi algorithm (bottom-left), and the centroidal Voronoi algorithm with dithering (bottom-right).

Observe that the compressed image obtained by the centroidal Voronoi algorithm suffers from a phenomenon known as *contouring*; see the shoulder in the image on the bottom-left of Fig. 1.3. We emphasize that contouring is a difficulty associated with the *assignment* step of the algorithm, and not with the particular choice for the reduced set of colors used to determine the approximate image. Contouring occurs when the

colors (or, in the case of a monochrome picture, the shades of gray) in the original, uncompressed picture are changing “continuously.” Then, the colors (or shades) of two neighboring pixels in the physical picture can be on opposite sides of an edge of a Voronoi region in color space. As a result, two different colors (or shades) are assigned to the two neighboring pixels so that the distribution of colors in the compressed picture becomes discontinuous, e.g., contours appear in the image.

Contouring can be ameliorated by *dithering*. For example, instead of always assigning a color to the nearest color in the reduced set, i.e., to the generator of the Voronoi region to which the color belongs, one sometimes assigns a color to the second nearest generator. The assignments can be done based on the relative distances from the point to be assigned to the two nearest generators. Then, if these distances are nearly equal, i.e., if the point is near the boundary of its Voronoi polygon in color space, it is almost as probable that the point will be assigned to the second nearest generator. On the other hand, if the point is near its Voronoi generator, it is much more probable that it will not be assigned to the second nearest generator. If d_1 and d_2 denote the distances (in color space) between the point to be assigned and the nearest and second nearest Voronoi generators, respectively, one simple method for determining the assignment is to assign the point to the nearest generator with probability $d_2/(d_1 + d_2)$ and to the second nearest generator with probability $d_1/(d_1 + d_2)$. The bottom-right image in Fig. 1.3 is that of a compressed image determined using the same centroidal Voronoi tessellation for the approximating shades of gray as was used for the bottom-left picture, but for which the assignment is done by this dithering algorithm. Note that the contours have now disappeared.

1.2.2 Optimal quadrature rules

For a given region $\Omega \subset \mathbb{R}^N$ and a given positive integer k , consider quadrature rules of the type

$$\int_{\Omega} f(\mathbf{y}) d\mathbf{y} \approx \sum_{i=1}^k A_i f(\mathbf{z}_i),$$

where $\{\mathbf{z}_i\}_{i=1}^k$ are k points in $\overline{\Omega}$ and $\{A_i\}_{i=1}^k$ are the volumes of a set of regions $\{V_i\}_{i=1}^k$ that tessellate Ω . We would like to choose the $\mathbf{z}_i$'s and V_i 's so that the quadrature rule is, in some sense, as accurate as possible.

Let us examine the quadrature error for the class of Lipschitz continuous functions $f(\mathbf{y})$ with, say, Lipschitz constant L . In this case, we have that the quadrature error $\mathcal{Q}$ is given by

$$\begin{aligned} \mathcal{Q} &= \left| \int_{\Omega} f(\mathbf{y}) d\mathbf{y} - \sum_{i=1}^k A_i f(\mathbf{z}_i) \right| \\ &= \left| \sum_{i=1}^k \int_{V_i} (f(\mathbf{y}) - f(\mathbf{z}_i)) d\mathbf{y} \right| \leq L \sum_{i=1}^k \int_{V_i} |\mathbf{y} - \mathbf{z}_i| d\mathbf{y}. \end{aligned} \quad (1.11)$$

Thus, it is natural to try to choose $\{\mathbf{z}_i\}_{i=1}^k$ and $\{V_i\}_{i=1}^k$ so that the right-hand side of (1.11) is minimized.

We may also consider quadrature rules that use the values of the function f and its first $p-1$ derivatives evaluated at the points $\mathbf{z}_i$. Then, if the $(p-1)$ -st derivative of f is Lipschitz continuous with Lipschitz constant L , we have that the quadrature error $\mathcal{Q}$ is now estimated by

$$\mathcal{Q} \leq L \sum_{i=1}^k \int_{V_i} |\mathbf{y} - \mathbf{z}_i|^p d\mathbf{y}. \quad (1.12)$$

Again, it is natural to try to choose $\{\mathbf{z}_i\}_{i=1}^k$ and $\{V_i\}_{i=1}^k$ so that the right-hand-side of (1.12) is minimized.

The minimization of the right-hand-side of (1.11) or (1.12) is accomplished when the V_i 's are the Voronoi sets for the $\mathbf{z}_i$'s and, *simultaneously*, the $\mathbf{z}_i$'s are the mass centroids

of the V_i 's, in the sense of (1.7)–(1.9), where $d(\cdot, \cdot)$ is given by (1.6). This may be shown using techniques similar to the ones discussed in §3.

The problem of finding an optimal numerical quadrature has been well-studied in the literature. For instance, the classical Gaussian integration rules (12) and the quasi-Monte Carlo rules based on number-theoretic methods all share optimal properties in some sense. We refer to (51) for a comprehensive treatment of the latter subject; see also (30; 58; 68) for some recent developments. In general, one can consider optimal quadrature rules over a given function space and for specific types of function evaluations. The appearance of the centroidal Voronoi diagrams is the result of the special function spaces we have chosen here. We note that for integrals over irregular domains, many sophisticated integration rules can provide estimates superior to that for centroidal Voronoi diagrams, but the former require partitioning or mapping into regular regions. On the other hand, the quadrature rules given by the centroidal Voronoi diagrams can be a convenient method to provide a crude estimate with very limited information known about the integrand.

1.2.3 Optimal representation, quantization, and clustering

A related application is the representation/interpolation of observed data. For example, in one of the earliest practical applications of Voronoi diagrams, Thiessen studied the problem of estimating the total precipitation in 1911 in a given geographical region. The mathematical problem can be described as follows. Given k locations of the pluviometers $\{\mathbf{x}_i\}$ and associated subregions $\{V_i\}$, if $p(\mathbf{x})$ denotes the precipitation at $\mathbf{x}$, then the total precipitation

$$P = \sum_{i=1}^k \int_{V_i} p(\mathbf{x}) d\mathbf{x}$$

can be estimated by

$$\sum_{i=1}^k p(\mathbf{x}_i) |V_i|,$$

where $|V_i|$ denotes the area of V_i . If we wish to find the best locations for the pluviometers in order to reduce the estimation error, we may follow the discussion in §1.2.2 to formulate this optimization problem as an optimal quadrature problem. Another formulation is documented in (52). Imagine the function $p(\mathbf{x})$ is a random variable at $\mathbf{x}$ which can be expressed as

$$p(\mathbf{x}) = m(\mathbf{x}) + \epsilon(\mathbf{x}),$$

where $m(\mathbf{x})$ is a deterministic continuous function of $\mathbf{x}$ and $\epsilon(\mathbf{x})$ is a random function having zero expected value. Then, the estimation error can be formulated as

$$\mathcal{E}(\mathbf{x}_i, i = 1, \dots, k) = E\left[\sum_{i=1}^k \int_{V_i} |p(\mathbf{x}) - p(\mathbf{x}_i)|^2 d\mathbf{x}\right].$$

In the case of constant variance $E[\epsilon(\mathbf{x})^2] = \sigma$ and if the change in $m(\mathbf{x})$ is small compared with the variance σ in the associated region, one can approximate the above error by

$$\mathcal{E}(\mathbf{x}_i, i = 1, \dots, k) = \sum_{i=1}^k \int_{V_i} 2\sigma^2 (1 - Cov(\mathbf{x}, \mathbf{x}_i)) d\mathbf{x}.$$

Here, $Cov(\mathbf{x}, \mathbf{x}_i)$ is the covariance. Empirically, $1 - Cov(\mathbf{x}, \mathbf{x}_i)$ may be treated as an increasing function $f(|\mathbf{x} - \mathbf{x}_i|)$ of the distance $|\mathbf{x} - \mathbf{x}_i|$; thus, the problem of finding the optimal locations for pluviometers can be formulated as the minimization of the functional

$$\mathcal{E}(\mathbf{x}_i, i = 1, \dots, k) = 2\sigma^2 \sum_{i=1}^k \int_{V_i} f(|\mathbf{x} - \mathbf{x}_i|) d\mathbf{x}$$

which is, again, a problem of finding the centroidal Voronoi diagram.

A similar and even earlier example is the question of round-off, i.e., to represent real numbers by the closest integers. This problem has been studied in (62) and the answer is obviously a simple example of one-dimensional centroidal Voronoi diagrams.

The representation of a given quantity with less information is often referred to as *quantization* and it is an important subject in information theory. The subject of vector, i.e, multidimensional, quantization has broad applications in signal processing

and telecommunications. The image compression example we discussed earlier belongs to this subject as well. We refer to (21; 22) for surveys on the subject and comprehensive lists of references to the literature.

In the same spirit, centroidal Voronoi diagrams also play a central role in clustering analysis; see, e.g., (26; 57). Clustering, as a tool to analyze similarities and dissimilarities between different objects, is fundamental and is used in various fields of statistical analysis, pattern recognition, learning theory, computer graphics, and combinatorial chemistry. Given a set W of objects, the aim is to partition the set into, say, k disjoint subsets, called clusters, that best classifies the objects according to some criteria that distinguishes between them. This is commonly referred as *k-clustering analysis*. For example, in combinatorial chemistry, *k*-clustering analysis is used in compound selection (10) where the similarity criteria may be related to the compound components as well as their structure. Clustering analysis provides a selection of a finite collection of templates that well represent, in some sense, a large collection of data. To illustrate the connection between centroidal Voronoi diagrams and the optimal *k*-clustering, let us consider a simple example where W contains m points in $\mathbb{R}^N$. Given a subset (cluster) V of W with n points, the cluster is to be represented by the arithmetic mean

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{\mathbf{x}_j \in V} \mathbf{x}_j \quad (1.13)$$

which corresponds to the mass centroid of V in the definition (1.5) with $V^* = \mathbb{R}^N$. The variance is given by

$$Var(V) = \sum_{\mathbf{x}_j \in V} |\mathbf{x}_j - \bar{\mathbf{x}}|^2$$

and for a *k*-clustering $\{V_i\}_{i=1}^k$ (a tessellation of W into k disjoint subsets), the total variance is given by

$$Var(W) = \sum_{i=1}^k Var(V_i) = \sum_{i=1}^k \sum_{\mathbf{x}_j \in V_i} |\mathbf{x}_j - \bar{\mathbf{x}}_i|^2. \quad (1.14)$$

As observed in (35; 24), the optimal k -clustering having the minimum total variance occurs when $\{V_i\}_{i=1}^k$ is the Voronoi partition of W with $\{\bar{\mathbf{x}}_i\}_{i=1}^k$ as the generators, i.e., the optimal k -clustering is a centroidal Voronoi diagram if we use the above variance-based criteria.

In computational geometry, criteria such as the diameter or radius of the subsets have been well studied; see, e.g., (3). Other criteria have also been proposed, e.g., L_1 -based (29) and variance-based (71). In some applications of statistical analysis, only the clusters are of interest and the cluster centers are not important themselves. In other applications, e.g., the color image compression problem, the cluster centers are the representative colors to be retained, i.e., they should be elements of W . Thus, it may be more appropriate to replace the simple arithmetic mean by the more general notion of the mass centroid given in §1.1. For discussions of complexity issues involved in k -clustering as well as related clustering algorithms, see (19; 24; 37; 36; 54) and the references cited therein. Additional references on clustering analysis are also provided in later sections.

1.2.4 Finite difference schemes having optimal truncation errors

For the sake of ease of presentation, we only consider the simple problem

$$-\operatorname{div} \mathbf{u} = f \quad \text{and} \quad \mathbf{u} = \operatorname{grad} \phi \quad \text{in } \Omega \subset \mathbb{R}^2; \quad (1.15)$$

for the same reason, we assume that Ω is a polygonal domain and we also ignore all effects due to boundaries and boundary conditions. (We make no judgment as to the usefulness of the algorithm discussed here; our goal is merely to illustrate the potential of centroidal Voronoi grids.) In (1.15), $\mathbf{u}$ and ϕ are the unknowns and f is a given function defined on $\bar{\Omega}$.

A common way to discretize (1.15) is based on Voronoi tessellations. Thus, let $\{V_i\}_{i=1}^k$ denote a Voronoi tessellation of Ω with respect to a given set of k points $\{\mathbf{z}_i\}_{i=1}^k$

belonging to $\overline{\Omega}$. Let $\{T_j\}_{j=1}^m$ denote the dual tessellation, i.e., the Delaunay triangulation of the points $\{\mathbf{z}_i\}_{i=1}^k$. Let A_i and B_j denote the area of V_i and T_j , respectively.

Let $\{\mathbf{t}_{js}\}_{s=1}^3$ denote the set of vertices of the triangle T_j . Of course, for any j and s , $\mathbf{t}_{js}$ is one of the given points $\mathbf{z}_i$. From the first equation in (1.15), we have that

$$-\int_{T_j} f = \int_{T_j} \operatorname{div} \mathbf{u} = \int_{\partial T_j} \mathbf{u} \cdot \mathbf{n} \quad \text{for } j = 1, \dots, m, \quad (1.16)$$

where ∂T_j denotes the boundary of T_j . We approximate these equations by

$$-B_j f(\mathbf{v}_j) = \sum_{s=1}^3 b_s \left(\frac{\mathbf{u}(\mathbf{t}_{js}) + \mathbf{u}(\mathbf{t}_{j,s+1})}{2} \right) \cdot \mathbf{n}_s \quad \text{for } j = 1, \dots, m, \quad (1.17)$$

where B_j denotes the area of the triangle T_j , $\mathbf{n}_s$ and b_s denote the outer unit normal vector to and the length of the edge joining the vertices $\mathbf{t}_{js}$, and $\mathbf{t}_{j,s+1}$, respectively, and where $\mathbf{t}_{j4} = \mathbf{t}_{j1}$. In (1.17), $\mathbf{v}_j$ is a convenient point in T_j at which the data f is evaluated. If this point is chosen so that the left-hand side of (1.17) is a second-order approximation to the left-hand side of (1.16), then the truncation error of the difference equation (1.17) is of order h^2 , where h is some measure of the size of the triangulation, e.g., the largest diameter of any of the triangles in $\{T_j\}_{j=1}^m$.

Next, let $\{\mathbf{v}_{ij}\}_{j=1}^{K_i}$ denote the set of vertices of the Voronoi polygon V_i . From the second equation in (1.15), we have that

$$\int_{V_i} \mathbf{u} = \int_{V_i} \operatorname{grad} \phi = \int_{\partial V_i} \phi \mathbf{n} \quad \text{for } i = 1, \dots, k, \quad (1.18)$$

where ∂V_i denotes the boundary of V_i . We approximate these equations by

$$A_i \mathbf{u}(\mathbf{z}_i) = \sum_{j=1}^{K_i} \frac{\phi(\mathbf{v}_{ij}) + \phi(\mathbf{v}_{i,j+1})}{2} a_j \mathbf{n}_j, \quad \text{for } i = 1, \dots, k, \quad (1.19)$$

where A_i denotes the area of V_i , $\mathbf{n}_j$ and a_j denote the outer unit normal vector to and the length of the edge joining the vertices $\mathbf{v}_{ij}$ and $\mathbf{v}_{i,j+1}$, respectively, and where $\mathbf{v}_{i,K_i+1} = \mathbf{v}_{i1}$. If each $\mathbf{z}_i$ is chosen to be the centroid of the region V_i , then the truncation error of the difference equations (1.19) is again of order h^2 . This follows because the

left hand side of (1.19) is a second-order approximation to the left-hand side of (1.18) whenever $\mathbf{z}_i$ is the centroid of V_i .

Thus, if we discretize (1.15) according to (1.17) and (1.19), we can guarantee a second-order truncation error for the difference equations by choosing $\{V_i\}_{i=1}^k$ to be a Voronoi tessellation of Ω with respect to the points $\{\mathbf{z}_i\}_{i=1}^k$, and, *simultaneously*, choosing the $\{\mathbf{z}_i\}_{i=1}^k$ to be the centroids of the corresponding regions $\{V_i\}_{i=1}^k$. (We note that, in general, difference equations need not have second-order truncation errors with respect to a given partial differential equation in order for the solution of the difference equations to be a second-order approximation to the solution of the differential equation. For example, there are many second-order accurate finite element schemes which, when interpreted as difference schemes, do not have second-order truncation errors.)

Related to our discussion here, we can consider *covolume methods* for the approximate solution of partial differential equations, making use of the dual Voronoi–Delaunay tessellations. Unlike the finite difference scheme discussed above, unknowns can be associated with edges of either or both the Voronoi polygons or the Delaunay triangles (50). Here, it is possible to use centroidal Voronoi grids to obtain schemes for which the L^2 -error in the approximate solution is of order h^2 and for which the error is only of order h for general grids.

1.2.5 Optimal placement of resources

We consider a typical example among problems dealing with the optimal placement of resources. We ask: what is the optimal placement of mailboxes in a city or neighborhood so as to make them most convenient for users? We make the following assumptions.

- A user will use the mailbox nearest to their home.
- The cost (to the user) of using a mailbox is a function of the distance from the user’s home to the mailbox.

- The total cost to users as a whole is measured by the distance to the nearest mailbox averaged over all users in the region.
- The optimal placement of mailboxes is defined to be the one that minimizes the total cost to the users.

The total cost function can be thus expressed as

$$\mathcal{E}(\mathbf{x}_i, i = 1, \dots, k) = \sum_{i=1}^k \int_{V_i} f(|\mathbf{x} - \mathbf{x}_i|) \phi(\mathbf{x}) d\mathbf{x}.$$

Then, it is not difficult to show that the optimal placement of the mailboxes is at the centroids of a centroidal Voronoi tessellation of the city, using the population density ϕ as a density function and $f(z) = z^2$. For other choices of f , we also get the centroidal Voronoi tessellation in the generalized sense. More details can be found in, e.g., (38; 52).

The same formulation can be generalized to other problems related to the optimal placement of resources, such as schools, distribution centers, mobile vendors, bus terminals, voting stations, service stops, etc.

1.2.6 Cell division

There are many examples of animal and plant cells, usually monolayered or columnar, whose geometrical shapes are polygonal. In many cases, they can be identified with a Voronoi tessellation and, indeed, with a centroidal Voronoi tessellation; see, e.g., (31; 32). Likewise, centroidal Voronoi tessellations can be used to model how cells are reshaped when they divide or when they are removed from a tissue. Here we consider one example provided in (31; 32), namely cell division in certain stages in the development of a starfish (*Asteria pectinifera*) embryo. During this stage, the cells are arranged in a hollow sphere consisting of a single layer of columnar cells. It is shown in (31; 32) that, viewed locally along the surface, the cells shapes closely match that of a Voronoi tessellation and, in fact, are that of a centroidal Voronoi tessellation.

The results of the cell division process can also be described using centroidal Voronoi tessellations; see (31; 32). We start with a configuration of cells that, by observation, is a Voronoi tessellation. In this geometrical description of the cell shapes, cell division can be modeled as the addition of another Voronoi generator, or more precisely, as the splitting of a Voronoi generator into two generators. Having added a generator (or more than one if more than one cell divides), how does one determine the shapes of the new cells and the (necessarily) different shapes of the remaining old cells? In (31; 32) it is shown that the new shapes are very closely approximated by a centroidal Voronoi tessellation corresponding to the increased number of generators.

The exact geometrical cell division algorithm described in (31; 32) is as follows. A photograph of an arrangement of polygonal cells is shown to be closely approximated by a Voronoi tessellation and is used to define data from which the corresponding Voronoi generators are determined. Additional photographs (taken at later times) are used to identify the parent cells that divide. The generator of a parent cell (now represented by a Voronoi polygon) is replaced by two points lying along a long axis of the cell; the exact placement of the two points turns out to be unimportant. A new Voronoi tessellation is determined using the generators of the cells that have not divided along with the points resulting from the replacement of the generators of the parent cells. In this manner, two daughter cells are introduced for each parent cell. The shape of these cells are now adjusted by first moving all the generators to the centroids of their corresponding Voronoi polygon and then recomputing the Voronoi tessellation. This procedure is repeated until the cell shapes, i.e, the Voronoi polygons, cease to change. The final arrangement of cells determined by this iterative process is a centroidal Voronoi tessellation and matches very well with photographs of the actual cells after the cell division process is completed.

1.2.7 Territorial behavior of animals

There are many species of animals that employ Voronoi tessellations to stake out territory. If the animals settle into a territory asynchronously, i.e., one or a few at a time, the distribution of Voronoi generators often resembles the centers of circles with fixed radius in a random circle packing problem. On the other hand, if the settling occurs synchronously, i.e., all the animals settle at the same time, then the distribution of Voronoi centers can be that for a centroidal Voronoi tessellation.

As an example of synchronous settling for which the territories can be visualized, consider the mouthbreeder fish (*Tilapia mossambica*). Territorial males of this species excavate breeding pits in sandy bottoms by spitting sand away from the pit centers towards their neighbors. For a high enough density of fishes, this reciprocal spitting results in sand parapets which are visible territorial boundaries. In (5), the results of a controlled experiment were given. Fishes were introduced into a large outdoor pool with a uniform sandy bottom. After the fishes had established their territories, i.e., after the final position of the breeding pits were established, the parapets separating the territories were photographed. In Fig. 1.4, the resulting photograph from (5) is reproduced. The territories are seen to be polygonal and, in (31; 65), it was shown that they are very closely approximated by a Voronoi tessellation.

A behavioral model for how the fishes establish their territories was given in (33; 34; 70). When the fishes enter a region, they first randomly select the centers of their breeding pits, i.e., the locations at which they will spit sand. Their desire to place the pit centers as far away as possible from their neighbors cause the fish to continuously adjust the position of the pit centers. This adjustment process is modeled as follows. The fishes, in their desire to be as far away as possible from their neighbors, tend to move their spitting location towards the centroid of their current territory; subsequently, the territorial boundaries must change since the fishes are spitting from different locations.

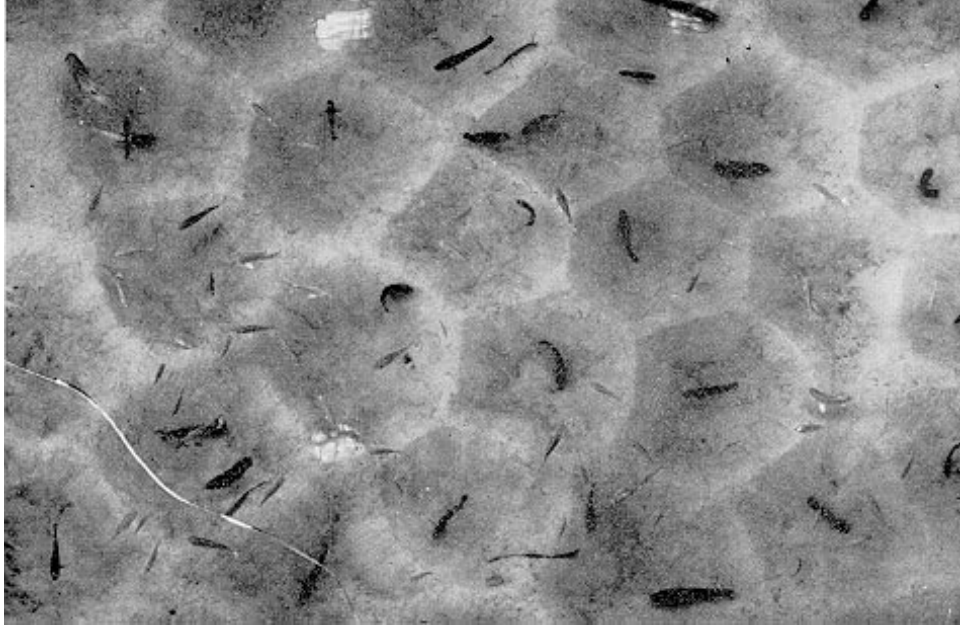


Figure 1.4 A top view photograph, using a polarizing filter, of the territories of the male *Tilapia mossambica*; each is a pit dug in the sand by its occupant. The boundaries of the territories, the rims of the pits, form a pattern of polygons. The breeding males are the black fish which range in size from about 15 to 20cm. The gray fish are the females, juveniles, and non-breeding males. The fish with a conspicuous spot in its tail, in the upper right corner, is a *Cichlasoma maculicauda*. Photograph and caption by permission of Academic Press, London.

Since all the fishes are assumed to be of equal strength, i.e., they all presumably have the same spitting ability, the new boundaries naturally define a Voronoi tessellation of the sandy bottom with the pit centers as the generators. The adjustment process, i.e., movement to centroids and subsequent redefinition of boundaries, continues until a steady state configuration is achieved. The final configuration is a centroidal Voronoi tessellation. If we denote the center of the pit belonging to the i -th fish by $\mathbf{z}_i$ and the territory staked out by that fish by V_i , remarkably, the V_i 's are the Voronoi regions for the $\mathbf{z}_i$'s and the $\mathbf{z}_i$'s are the mass centroids of the V_i 's.

1.3 Some results about centroidal Voronoi tessellations and their minimization properties

1.3.1 Results involving centroidal Voronoi tessellations as minimizers

For the sake of completeness, we provide the following result. The analogous result for other metrics or for discrete sets can be proven using only slightly more complicated arguments.

Proposition 1. *Given $\Omega \subset \mathbb{R}^N$, a positive integer k , and a density function $\rho(\cdot)$ defined on $\overline{\Omega}$. Let $\{\mathbf{z}_i\}_{i=1}^k$ denote any set of k points belonging to $\overline{\Omega}$ and let $\{V_i\}_{i=1}^k$ denote any tessellation of Ω into k regions. Let*

$$\mathcal{F}((\mathbf{z}_i, V_i), i = 1, \dots, k) = \sum_{i=1}^k \int_{\mathbf{y} \in V_i} \rho(\mathbf{y}) |\mathbf{y} - \mathbf{z}_i|^2 d\mathbf{y}. \quad (1.20)$$

A necessary condition for $\mathcal{F}$ to be minimized is that the V_i 's are the Voronoi regions corresponding to the $\mathbf{z}_i$'s [in the sense of (1.1)] and, simultaneously, the $\mathbf{z}_i$'s are the centroids of the corresponding V_i 's [in the sense of (1.2)].

Proof. See (13). □

Another interesting and useful point follows from the proof and the above proposition. Define the functional

$$\mathcal{K}((\mathbf{z}_i), i = 1, \dots, k) = \sum_{i=1}^k \int_{\mathbf{y} \in \widehat{V}_i} \rho(\mathbf{y}) |\mathbf{y} - \mathbf{z}_i|^2 d\mathbf{y}, \quad (1.21)$$

where $\widehat{V}_i$'s are the Voronoi regions corresponding to the $\mathbf{z}_i$'s. Note that the functional in (1.21) is only a function of the $\mathbf{z}_i$'s since, once these are fixed, the $\widehat{V}_i$'s are determined; for the functional in (1.20) it is assumed that the $\mathbf{z}_i$'s and V_i 's are independent. For minimizers of the functionals in (1.20) and (1.21), we have the following result

Proposition 2. *Given $\Omega \subset \mathbb{R}^N$, a positive integer k , and a density function $\rho(\cdot)$ defined on $\overline{\Omega}$. Then, $\mathcal{F}$ and $\mathcal{K}$ have the same minimizer.*

1.3.2 Existence of a minimizer

Given $\Omega \subset \mathbb{R}^N$, a positive integer k , and a density function $\rho(\cdot)$ defined on $\overline{\Omega}$. Let $\{\mathbf{z}_i\}_{i=1}^k$ denote any set of k points belonging to $\overline{\Omega}$ and let $\{V_i\}_{i=1}^k$ denote any tessellation of Ω into k regions. Let $K = \{\mathbf{Z} = (\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k), \mathbf{z}_j \in \overline{\Omega}\}$. Let $A_i = |\widehat{V}_i|$, the area of $\widehat{V}_i$, and $\mathbf{A} = (A_1, \dots, A_k)$. Then, one easily obtains the following results.

Lemma 1. *$\mathbf{A}$ is continuous.*

Lemma 2. *$\mathcal{K}$ is continuous and thus it possesses a global minimum.*

Proof. See (13). □

In general, $\mathcal{K}$ may also have some local minimizers; it also can be shown that a local minimizer of $\mathcal{K}$ will give non-degenerate Voronoi diagrams.

Proposition 3. *Assume that $\rho(\mathbf{y})$ is positive except on a set of measure zero in Ω . Then, local minimizers satisfy $\mathbf{z}_i \neq \mathbf{z}_j$ for $i \neq j$.*

Proof. See (13). □

Remark 1. For a general metric (assuming that ρ vanishes only on a set of zero measure), existence is provided by the compactness of the Voronoi regions. Uniqueness is also attained if the Voronoi regions are convex and for any $\mathbf{y}_1, \mathbf{y}_2 \in \widehat{V}_i$, $\mathbf{y}_1 \neq \mathbf{y}_2$, the inequality

$$d(\mathbf{z}, \lambda \mathbf{y}_1 + (1 - \lambda) \mathbf{y}_2) < \lambda d(\mathbf{z}, \mathbf{y}_1) + (1 - \lambda) d(\mathbf{z}, \mathbf{y}_2)$$

is valid for some $\lambda \in (0, 1)$ on a subset of $\widehat{V}_i$ with positive measure.

1.3.3 Equal partition of energy in one dimension

We consider the unit interval with constant density $\rho(x) = 1$. Then, for given k , the optimal centroids with k intervals are given by $x_i = (2i - 1)/2k$, $i = 1, \dots, k$. A simple

calculation shows that

$$\int_{V_i} |x - x_i|^2 dx = \frac{1}{12k^3} \quad \forall i,$$

i.e., the energy is equally partitioned over the k Voronoi regions. One naturally asks, does the centroidal Voronoi diagram for general densities share the equal partition of energy property? The answer is negative in general. For example, let the density function be given by $\rho(x) = x$ and $k = 2$. Then the two centroidal Voronoi regions are given by $(0, z)$ and $(z, 1)$ where $z = (\sqrt{5} - 1)/2$, and the centers of mass are given by $2z/3$ and $4z/3$, respectively. The values of energy on each region are given by $(7 - 3\sqrt{5})/72$ and $(83 - 37\sqrt{5})/72$.

An interesting question is related to the distribution of the energy (also referred to as the cost or variance or distortion error, etc., in some contexts) when the number of generators gets large. Let $h_i = |V_i|$. Under the condition that the density function $\rho(x)$ is bounded and strictly positive, we have $h = \max h_i \rightarrow 0$, as k , the number of generators, goes to infinity. Furthermore, we have the following result:

Proposition 4. *Let m_i be the midpoint of the interval V_i , then*

$$|V_i| \approx \frac{c}{\rho^{1/3}(m_i)}.$$

and

$$\int_{V_i} \rho(x)(x - x_i)^2 dx \approx \frac{1}{12} \rho(m_i) h_i^3 \approx \frac{c^3}{12} \quad \forall i.$$

for some constant c .

Proof. See (13). □

These two properties are the deterministic analog of the results in (73) for the probabilistic k -means clustering that we discussed in the previous section, i.e., under some assumptions on the density function, asymptotically speaking, the energy is equally distributed in the Voronoi intervals and the sizes of the Voronoi intervals are inversely

proportional to the one-third power of the underlying density at the midpoints of the intervals. In (20), an important conjecture is made which states that asymptotically, for the optimal centroidal Voronoi tessellation, all Voronoi regions are approximately congruent to the same basic cell that only depends on the dimension. The basic cell is shown to be the regular hexagon in two dimension (49) but the conjecture remains open for three and higher dimensions. The equidistribution of energy principle, however, can be established based on Gersho's conjecture (20; 22).

CHAPTER 2. PROBABILISTIC ALGORITHMS FOR CENTROIDAL VORONOI TESSELLATIONS AND THEIR PARALLEL IMPLEMENTATIONS

The plan of this chapter is as follows. First, we briefly describe the generation of random numbers according to a nonuniform probability density function. Then, in §2.2, we discuss known and new sequential probabilistic methods for determining centroidal Voronoi tessellations and compare their performance. In §2.3, we consider parallel implementations of the methods examined in §2.2 and study their performance through some parallel computational experiments using MPI (Message Passing Interface) (48; 66).

2.1 Determining random numbers from nonuniform densities

Random sampling points corresponding to a nonuniform density function may be determined from uniformly distributed points in a variety of ways. For example, in one dimension, given the interval (a, b) and the density function $\rho(x)$ defined on $[a, b]$, a random point x in $[a, b]$ may be determined from

$$\frac{\int_a^x \rho(s) ds}{\int_a^b \rho(s) ds} = X,$$

where X is a point in $[0, 1]$ sampled according to a uniform distribution. If many points need be sampled, this classical procedure is very computationally intensive, especially

in higher dimensions, due to the need to do repeated numerical integrations. It is also difficult to apply to complex geometries.

An alternate, completely probabilistic procedure is the *rejection method* (60). In this procedure, a random point x in $[a, b]$ is determined by first sampling two random points X' and U with constant density in $[0, 1]$ and then setting $X = a + (b - a)X'$. If $U < \rho(X)/\hat{\rho}$, we let $x = X$; otherwise, we begin again. Although we may need to call the random number generator many times (the number of times depends on the density function $\rho(x)$), the total computation time is in general trivial compared to the classical procedure using numerical integrations.

The rejection method can be extended to higher dimensions. For example, for two-dimensional domains, one can use the following completely probabilistic procedure (in the sense that no deterministic integrations are involved) for generating random points. Given the domain $\Omega \subset \mathbb{R}^2$, we determine an enclosing rectangle $D = [a, b] \times [c, d]$, i.e., a rectangle D such that $\overline{\Omega} \subset D$ and whose sides are parallel to the coordinate axes. We are also given the density function $\rho(x, y)$ defined on $\overline{\Omega}$; we set $\hat{\rho} = \max_{(x, y) \in \overline{\Omega}} \rho(x, y)$. Then, a random point $(x, y) \in \overline{\Omega}$ is determined as follows. First, we sample a random point X' with constant density in $[0, 1]$ and set $X = a + (b - a)X'$. Then, we sample a second random point Y' with constant density in $[0, 1]$ and set $Y = c + (d - c)Y'$. If $(X, Y) \notin \overline{\Omega}$, then we begin again. Otherwise, we next sample a random point U with constant density in $[0, 1]$. If $U < \rho(X, Y)/\hat{\rho}$, set $(x, y) = (X, Y)$; otherwise, start again. (The rejection of points in D which are outside of $\overline{\Omega}$ may also be effected by setting $\rho(x, y) = 0$ in $D \setminus \overline{\Omega}$.) This two-dimensional sampling algorithm can be obviously extended to higher dimensions.

In the sequel, we will refer to the rejection method as a *Monte Carlo method*; other sampling methods may be substituted without substantially affecting the relative efficiencies of the algorithms we consider. Sampling methods such as the Monte Carlo method play a crucial role in probabilistic algorithms for determining centroidal Voronoi

tessellations. Not only can Monte Carlo methods be used to generate initial conditions for the latter type of algorithms, but they will be used to generate the random sampling points needed during each iteration of those algorithms. In addition, Monte Carlo methods will provide results for comparisons to centroidal Voronoi tessellations.

2.2 Methods for determining centroidal Voronoi tessellations

In this section, we discuss sequential methods for determining centroidal Voronoi tessellations. The first two methods, one probabilistic and one deterministic, are well known in the literature. The third method is a new probabilistic method which may be viewed as a generalization of both known methods.

2.2.1 MacQueen's method

First, we introduce *MacQueen's method* (47), a very elegant random sequential sampling method which divides the samples into k sets or clusters by taking means of sampling points.

Algorithm 1. (MacQueen's method) *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, and a positive integer k ,*

0. *choose an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$ in Ω , e.g., by using a Monte Carlo method;
set $j_i = 1$ for $i = 1, \dots, k$;*
1. *determine a point $\mathbf{y}$ in Ω at random, e.g., by a Monte Carlo method, according to the probability density function $\rho(\mathbf{x})$;*
2. *find a $\mathbf{z}_{i^*}$ among $\{\mathbf{z}_i\}_{i=1}^k$ that is the closest to $\mathbf{y}$;*
3. *set*

$$\mathbf{z}_{i^*} \leftarrow \frac{j_{i^*}\mathbf{z}_{i^*} + \mathbf{y}}{j_{i^*} + 1} \quad \text{and} \quad j_{i^*} \leftarrow j_{i^*} + 1;$$

the new $\mathbf{z}_{i^}$, along with the unchanged $\{\mathbf{z}_j\}$, $j \neq i^*$, form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;*

- 4. if the new points meet some convergence criterion, terminate; otherwise, return to step 1.*

In all our numerical experiments for MacQueen's method and other iterative methods, we will terminate at a fixed number of iterations. In practice, other stopping criteria, e.g., a sufficiently small difference in successive energies or in the successive positions of the generators, may be used instead.

Clearly, at any stage of MacQueen's method, each point $\mathbf{z}_i$, $i = 1, \dots, k$, is the mean of all sampling points that have been found to be closest to the point $\mathbf{z}_i$ that was in effect when each point was sampled. Thus, this subset of all sampling points forms a cluster and the point $\mathbf{z}_i$ is the mean of its cluster.

The almost sure convergence of the energy for the random MacQueen's method has been proved, although not in all cases does the method converge to a centroidal Voronoi tessellation; see (47; 26). As is the case for most probabilistic methods, Monte Carlo methods play a key role in MacQueen's method. Of course, many variations of MacQueen's method have been developed, such as using bootstrapping techniques (39; 55). Here, we focus on Algorithm 1 since this is the method we will generalize.

One-dimensional numerical experiments for Algorithm 1 (MacQueen's method) were performed on a CRAY T3E-600 (11) located at the CRAY Inc. facility in Eagan, Minnesota. In all cases, 128 generators in the interval $(-1, 1)$ were used. Three different density functions were considered: a constant density function; e^{-10x^2} which has a huge variation, i.e., its values range from 1 to e^{-10} ; and $e^{-20x^2} + 0.05 \sin^2(\pi x)$ which has a large peak at the center of the interval but also varies with small amplitude away from the peak. Since MacQueen's method is a probabilistic one, four experiments for each situation were performed. Different seeds (time-based) were used to initialize the random

number generator for each experiment. Since only one closest point search and only one update of the search space are needed at each iteration, there is not much motivation for efficiently determining closest neighbors. Thus, no special algorithm was used for the one-dimensional experiments. The results of the one-dimensional computational experiments for Algorithm 1 (MacQueen’s method) are given in Table 2.1 and Figure 2.1. The data given in Table 2.1 are the average of the energies and the maximum running times (in seconds) over the four realizations vs. the number of iterations of MacQueen’s method. In Figure 2.1, the energy vs. the number of iterations is displayed for the three choices for the density function.

Table 2.1 Maximum running times and average energies over four realizations of MacQueen’s method vs. number of iterations for three density functions.

Algorithm 1 (MacQueen’s method) 128 generators on $(-1, 1)$						
	1		e^{-10x^2}		$e^{-20x^2} + \frac{1}{20} \sin^2(\pi x)$	
no. of iterations	average energy	maximum time (sec)	average energy	maximum time (sec)	average energy	maximum time (sec)
0	22.514E-5	0.00	8.211E-5	0.00	9.323E-5	0.00
200,000	5.888E-5	2.93	2.285E-5	3.62	2.802E-5	3.83
800,000	5.658E-5	11.59	2.159E-5	13.94	2.702E-5	16.48
3,200,000	5.475E-5	46.34	2.059E-5	55.21	2.622E-5	63.44

Table 2.1 and Figure 2.1 illustrate the slow convergence of MacQueen’s method; the energy is reduced very little during each step of the algorithm and, even after a large number of iterations, one does not achieve a good approximation. For a constant density function, the energy of the true minimizer for 128 generators in $(-1, 1)$ can be computed exactly and is found to be 4.069E-5. We see from Table 2.1 and Figure 2.1 for a constant density function that even after 3,200,000 iterations, we are not very close to the minimum energy; the lowest energy achieved after that many steps is over 15% larger than the energy of the minimizer. (This does not imply that we are necessarily “far away” from the optimal set of points; we will return to this point in Section 2.3.3.)

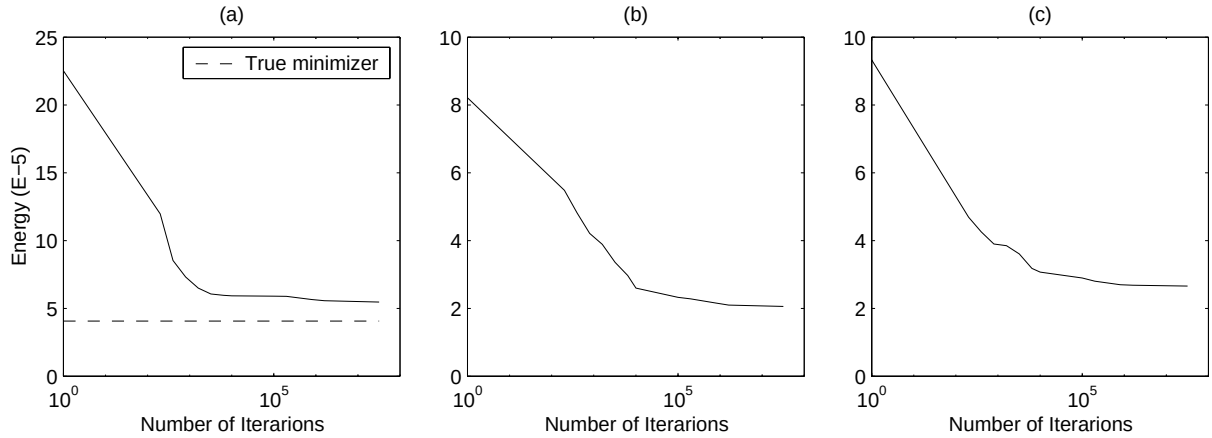


Figure 2.1 Energy vs. number of iterations for MacQueen's method. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$.

One reason for the inefficiency of MacQueen's algorithm is that only a single random sampling point is used and only a single point $\mathbf{z}_i$ is updated at each iteration. We also notice that, using the rejection method for generating nonuniformly distributed sampling points, the running times for the nonconstant density functions are only slightly higher than the corresponding running times for the constant density function.

One observation that can be gleaned from a detailed examination of the results of the computational experiments (although not definitively from the data given in the tables) is that the energy of the final set of points is closely correlated to the energy of the corresponding initial set of points, i.e., in general, the smaller the initial energy, the smaller the final energy.

2.2.2 Lloyd's method

In the discussion in Section 2.2.3 about the new probabilistic algorithm, we will need to refer to a deterministic algorithm for determining centroidal Voronoi tessellations that is known in some circles as *Lloyd's method* (13; 45) and which is the obvious iteration between computing Voronoi diagrams and mass centroids.

Algorithm 2. (Lloyd’s Method) *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, and a positive integer k ,*

0. *select an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$, e.g., by using a Monte Carlo method;*
1. *construct the Voronoi sets $\{V_i\}_{i=1}^k$ associated with $\{\mathbf{z}_i\}_{i=1}^k$;*
2. *determine the mass centroids of the Voronoi sets $\{V_i\}_{i=1}^k$; these centroids form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;*
3. *if the new points meet some convergence criterion, terminate; otherwise, return to step 1.*

Generally, compared to MacQueen’s method, Lloyd’s method requires substantially fewer iterations but has a much higher cost per iteration. In step 1 of the Lloyd’s method, a Voronoi tessellation must be explicitly constructed and in step 2, numerical integrations over polyhedral domains must be employed to calculate centroids. A virtue of MacQueen’s method is that it does not require the construction of Voronoi tessellations. Furthermore, if a completely probabilistic procedure such as the rejection method is used to determine random sampling points, then MacQueen’s method also does not involve any numerical integrations.

2.2.3 A new probabilistic method

Although MacQueen’s method is a very elegant random sampling method for determining centroidal Voronoi tessellations, its slow convergence provides motivation for developing alternative methods that are more efficient. One could, of course, look for parallel implementations of MacQueen’s method; we consider this approach in Section 2.3.1. Here, we present a new probabilistic method that can be viewed as a probabilistic version of Lloyd’s method and/or a generalization of the random MacQueen’s method.

Algorithm 3. *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, and a positive integer k ,*

0. *choose a positive integer q and constants $\{\alpha_i, \beta_i\}_{i=1}^2$ such that $\alpha_2 > 0$, $\beta_2 > 0$, $\alpha_1 + \alpha_2 = 1$, and $\beta_1 + \beta_2 = 1$; choose an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$, e.g., by using a Monte Carlo method; set $j_i = 1$ for $i = 1, \dots, k$;*
1. *choose q points $\{\mathbf{y}_r\}_{r=1}^q$ in Ω at random, e.g., by a Monte Carlo method, according to the probability density function $\rho(\mathbf{x})$;*
2. *for $i = 1, \dots, k$, gather together in the set W_i all sampling points $\mathbf{y}_r$ closest to $\mathbf{z}_i$ among $\{\mathbf{z}_i\}_{i=1}^k$ (i.e., in the Voronoi region of $\mathbf{z}_i$); if the set W_i is empty, do nothing; otherwise, compute the average $\mathbf{u}_i$ of the set W_i and set*

$$\mathbf{z}_i \leftarrow \frac{(\alpha_1 j_i + \beta_1) \mathbf{z}_i + (\alpha_2 j_i + \beta_2) \mathbf{u}_i}{j_i + 1} \quad \text{and} \quad j_i \leftarrow j_i + 1;$$

the new set of $\{\mathbf{z}_i\}$, along with the unchanged $\{\mathbf{z}_j\}$ (i.e. W_j is empty), form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;

3. *if the new points meet some convergence criterion, terminate; otherwise, return to step 1.*

For $q = 1$, $\alpha_1 = \beta_2 = 1$, and $\alpha_2 = \beta_1 = 0$, this method reduces to MacQueen's method. If $\alpha_1 = \beta_1 = 0$, $\alpha_2 = \beta_2 = 1$, then $\mathbf{z}_i = \mathbf{u}_i$, the average of the points in the set W_i ; then, since the points in W_i are randomly selected points in the Voronoi region corresponding to $\mathbf{z}_i$, one may view $\mathbf{u}_i$ as a probabilistic approximation to the centroid of V_i . Thus, for $\alpha_1 = \beta_1 = 0$, $\alpha_2 = \beta_2 = 1$, this method is *a probabilistic version of Lloyd's method*; the larger q is, the better the centroid approximations. This justifies our view of Algorithm 3 as both a probabilistic Lloyd's method and a generalization of MacQueen's method. Other choices for $\{\alpha_j, \beta_j\}_{j=1}^2$ define other methods.

At every iteration of Algorithm 3, we sample a set of q points in the region Ω and, at least if q is large, we may update, perhaps all, of the generators $\{\mathbf{z}_i\}_{i=1}^k$. Furthermore, again if q is large, it is likely that many sampling points are used to update each generator. This gives us hope that this algorithm is more efficient than Algorithm 1 (MacQueen's method) for which only a single point is sampled and only a single generator is updated at each iteration.

The updating of the generators effected in Step 3 of Algorithm 3 may be rewritten in the form

$$\mathbf{z}_i \leftarrow C_{1i}\mathbf{z}_i + C_{2i}\mathbf{u}_i, \quad (2.1)$$

where

$$C_{1i} = \frac{\alpha_1 j_i + \beta_1}{j_i + 1} \quad \text{and} \quad C_{2i} = \frac{\alpha_2 j_i + \beta_2}{j_i + 1}.$$

This clearly shows that the new position of each generator is a linear combination of the old position and the average of the points sampled in the Voronoi region of the old generator, see Figure 2.2 for a visual description. Note that since $\alpha_1 + \alpha_2 = 1$ and $\beta_1 + \beta_2 = 1$, we have that $C_{1i} + C_{2i} = 1$. From (2.1) and the fact that α_1 and β_1 may be negative, one can see that under and over-relaxation updating methods can be defined; see (13; 42).

Algorithm 3 represents a two parameter family of algorithms, e.g., each choice of α_2 and β_2 defines a new algorithm. (One also has to choose q , the number of points sampled at each iteration.) From (2.1), it is clear that Algorithm 3 will converge (in a probabilistic sense) only if $C_{1i} \rightarrow 1$ and $C_{2i} \rightarrow 0$ as the number of iterations increases. This holds for MacQueen's method for which $C_{1i} = j_i/(j_i + 1)$ and $C_{2i} = 1/(j_i + 1)$ but not for the probabilistic Lloyd's method for which $C_{1i} = 0$ and $C_{2i} = 1$. The reason why the probabilistic Lloyd's method cannot converge is that, for a fixed number of sampling points q per iteration, the average of the sampling points in the Voronoi region of a generator cannot be the true centroid of that region. However, this does not mean that

methods such as the probabilistic Lloyd’s method are useless. Quite the contrary, as we shall see, they can be quite effective. One need only realize that one cannot hope to get better accuracy than that allowed for by the amount of sampling done at each iteration. In turn, this means that such methods require “large” q in order to be very effective.

We now report on the results of some one-dimensional computational experiments with 128 generators in the interval $(-1, 1)$. Since we want q to be large, it is useful to do the closest point searches efficiently. In the one-dimensional setting, these searches were done by first quickly sorting the set of k points $\{z_i\}_{i=1}^k$, then, for each y_r , $r = 1, \dots, q$, using binary searching to find the z_{i_r} in the set $\{z_i\}_{i=1}^k$ which is closest to y_r . For the results for Algorithm 3 given in Table 2.2, we chose five values for q (500, 1,000, 2,000, 8000, and 16,000), two sets of parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ ($\alpha_1 = \beta_1 = 0, \alpha_2 = \beta_2 = 1$ and $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$), and the same three density functions as were used for Table 2.1. For each value of q , the number of random points sampled per iteration, the number of iterations of Algorithm 3 performed was chosen so that the total number of random points sampled (other than for initialization) was 3,200,000. For reference, the initial energies for the runs reported in Table 2.2 are in the range $[1.598\text{E-}4, 3.113\text{E-}4]$ for $\rho = 1$, $[4.363\text{E-}5, 1.482\text{E-}4]$ for $\rho = e^{-10x^2}$, and $[4.682\text{E-}5, 1.679\text{E-}4]$ for $\rho = e^{-20x^2} + \frac{1}{20} \sin^2(\pi x)$. Comparing with Table 2.1 for the random MacQueen’s method, we see that for the same density function and the same number (3,200,000) of random sampling points, the running times and the energies for Algorithm 3 are substantially lower than for Algorithm 1. The reason the running times are lower even though the number of sampling points are the same is that it is difficult to do efficient searching with MacQueen’s algorithm.

It is also interesting to examine the effect of changing the number of points sampled at each iteration. In Table 2.3, we give the average running times and average energies over four realizations of Algorithm 3 for a fixed number of iterations but for different numbers of points sampled during each iteration. The ranges of the initial energies are again as given above. We can conclude from the table that q , the number of points sampled at

Table 2.2 Maximum running times and average energies over four realizations of Algorithm 3 for a fixed number (3,200,000) of random points sampled.

Algorithm 3			128 generators on $(-1, 1)$		3,200,000 sampling points	
			$\alpha_1 = 0, \alpha_2 = 1$ $\beta_1 = 0, \beta_2 = 1$		$\alpha_1 = 1/2, \alpha_2 = 1/2$ $\beta_1 = 1/2, \beta_2 = 1/2$	
$\rho(x)$	q	no. of iterations	average energy	maximum time (sec)	average energy	maximum time (sec)
1	500	6,400	5.112E-5	36.01	4.488E-5	38.03
	1,000	3,200	4.472E-5	38.78	4.299E-5	39.23
	2,000	1,600	4.343E-5	40.07	4.341E-5	40.64
	8,000	400	4.267E-5	36.72	4.322E-5	36.14
	16,000	200	4.424E-5	37.36	4.366E-5	37.97
e^{-10x^2}	500	6,400	0.606E-5	43.99	0.603E-5	45.37
	1,000	3,200	0.593E-5	43.15	0.533E-5	44.25
	2,000	1,600	0.582E-5	45.60	0.634E-5	45.38
	8,000	400	0.720E-5	45.91	0.827E-5	43.46
	16,000	200	0.933E-5	44.84	0.916E-5	43.75
$e^{-20x^2} + \frac{1}{20} \sin^2(\pi x)$	500	6,400	0.605E-5	54.88	0.594E-5	55.38
	1,000	3,200	0.593E-5	54.65	0.533E-5	56.71
	2,000	1,600	0.590E-5	53.87	0.709E-5	54.74
	8,000	400	0.822E-5	54.24	1.013E-5	54.30
	16,000	200	1.153E-5	54.21	1.319E-5	55.69

each iteration, does not need to be too large. For example, in our experiments for 128 generators and 400 iterations, $q = 8,000$ seems to be good enough; choosing $q = 16,000$ effects almost no improvement, i.e., almost no further reduction in the energy, but requires much higher running times. We note that q should scale with the number of generators. Also, from Table 2.3, we see that, as expected, the running times scale linearly with the total number of points sampled.

Figure 2.3 provides a display of the energy versus number of iterations for Algorithm 3 with $q = 8000$ and $\{\alpha_1 = 0, \alpha_2 = 1, \beta_1 = 0, \beta_2 = 1\}$. Comparing with Figure 2.1, we see that Algorithm 3 converges much faster, i.e., in much fewer iterations, than does Algorithm 1. Of course, the work per iteration is higher for Algorithm 3 but, as can

Table 2.3 Average running times and average energies over four realizations of Algorithm 3 for a fixed number (400) of iterations.

Algorithm 3		128 generators on $(-1, 1)$		400 iterations		
			$\alpha_1 = 0, \alpha_2 = 1$ $\beta_1 = 0, \beta_2 = 1$	$\alpha_1 = 1/2, \alpha_2 = 1/2$ $\beta_1 = 1/2, \beta_2 = 1/2$		
$\rho(x)$	q	total points sampled	average energy	average time (sec)	average energy	average time (sec)
1	500	200,000	5.156E-5	2.36	4.678E-5	2.38
	1,000	400,000	4.748E-5	4.66	4.433E-5	4.69
	2,000	800,000	4.397E-5	9.87	4.542E-5	9.30
	8,000	3,200,000	4.267E-5	36.28	4.322E-5	35.69
	16,000	6,400,000	4.282E-5	72.64	4.257E-5	73.54
e^{-10x^2}	500	200,000	0.978E-5	2.83	1.028E-5	2.93
	1,000	400,000	1.003E-5	5.41	1.077E-5	5.70
	2,000	800,000	0.921E-5	10.81	0.926E-5	11.16
	8,000	3,200,000	0.720E-5	43.22	0.827E-5	42.23
	16,000	6,400,000	0.755E-5	85.04	0.786E-5	84.45
$e^{-20x^2} + \frac{1}{20} \sin^2(\pi x)$	500	200,000	1.313E-5	3.48	1.405E-5	3.51
	1,000	400,000	1.244E-5	6.84	1.410E-5	6.96
	2,000	800,000	0.958E-5	13.30	1.213E-5	13.71
	8,000	3,200,000	0.822E-5	53.06	1.013E-5	52.79
	16,000	6,400,000	0.841E-5	104.61	1.017E-5	106.73

be seen from Tables 2.1–2.3, for the same amount of total work, Algorithm 3 produces much better results than does Algorithm 1. It also should be pointed out that when the variations of the density functions become larger and more complex, more iterations are needed to obtain good results.

2.3 Parallel implementations

We now consider parallel implementations on distributed memory systems of the sequential probabilistic algorithms discussed in Section 2.2. For convinience of description, given two positive integers k and p and a non-negative integer i such as $p \leq k$ and $i < p$,

we define

$$M(i, p, k) = \begin{cases} \left\lfloor \frac{k}{p} \right\rfloor & \text{if } i < k \bmod(p) \\ \left\lfloor \frac{k}{p} \right\rfloor + 1 & \text{otherwise,} \end{cases}$$

$$G(0, p, k) = 0, \quad \text{and} \quad G(i, p, k) = \sum_{j=0}^{i-1} M(j, p, k),$$

where $\lfloor \cdot \rfloor$ denotes the greatest integer function. We begin with parallelizations of Algorithm 1, the random MacQueen's method.

2.3.1 Parallel MacQueen's methods

An obvious parallelization of MacQueen's method is given by the following algorithm.

Algorithm 4. (A parallel MacQueen's method) *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, a positive integer k , and p processors with rank = $0, 1, \dots, p-1$;*

0. *for rank = $0, \dots, p-1$, set $m_{rank} = M(rank, p, k)$; then, each processor independently chooses its own initial set of m_{rank} points $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$, e.g., by using a Monte Carlo method; set $j_i^{rank} = 1$ for $i = 1, \dots, m_{rank}$;*
1. *processor 0 selects a $\mathbf{y} \in \Omega$ at random, e.g., by a Monte Carlo method, according to the probability density function $\rho(\mathbf{x})$ and then broadcasts it to all other processors;*
2. *each processor finds the $\mathbf{z}_{i*}^{rank}$ which is the closest to $\mathbf{y}$ in $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$ respectively; determine the corresponding distance by d_{i*}^{rank} ;*
3. *compare them by communication to obtain the d^* , where d^* is the minimum among the d_{i*}^{rank} of all processors;*
4. *on each processor, if $d_{i*}^{rank} \neq d^*$, do nothing; otherwise, set*

$$\mathbf{z}_{i*}^{rank} \leftarrow \frac{j_{i*}^{rank} \mathbf{z}_{i*}^{rank} + \mathbf{y}}{j_{i*}^{rank} + 1} \quad \text{and} \quad j_{i*}^{rank} \leftarrow j_{i*}^{rank} + 1;$$

this new $\mathbf{z}_{i*}^{rank}$, along with the unchanged $\mathbf{z}_i^{rank}$, form the new set of points $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$;

5. if the new points meet some convergence criterion, terminate; otherwise, return to step 1.

In practice, Algorithm 4 is not a good one because in step 3 it requires the global transmission of a small message, i.e., a single floating point number. It is wasteful with regard to network bandwidth and this kind of communication is also inefficient because the hardware start-up time is much larger (generally by a factor of about 1000) than the communication time to send one floating point number through the network. Thus, we need to make some modifications to improve the communication efficiency. We accomplish this in the following algorithm by updating multiple points at each iteration; the algorithm is clearly based on MacQueen's method, but it also has some differences with the direct parallel implementation of the latter method.

Algorithm 5. (A modified parallel MacQueen's method) *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, a positive integer k , and p processors with $rank = 0, 1, \dots, p-1$;*

0. *choose a positive integer s ; for $rank = 0, \dots, p-1$, set $m_{rank} = M(rank, p, k)$; then, each processor independently chooses its own initial set of m_{rank} points $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$, e.g., by using a Monte Carlo method; set $j_i^{rank} = 1$ for $i = 1, \dots, m_{rank}$;*
1. *each processor independently selects s points in Ω at random, according to the probability density function $\rho(\mathbf{x})$; combine them together by communication to form a set of sampling points $\{\mathbf{y}_r\}_{r=1}^q$ ($q = sp$);*
2. *on each processor, for $r = 1, \dots, q$, find a $\mathbf{z}_{i_r}^{rank}$ among $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$ that is closest to $\mathbf{y}_r$; determine the corresponding distance by d_r^{rank} ;*

3. compare them by communication to obtain the set $\{d_r\}_{r=1}^q$, where d_r is the minimum among d_r^{rank} on all processors;
4. for $r = 1, \dots, q$, on each processor, if $d_r^{rank} \neq d_r$, do nothing; otherwise, set

$$\mathbf{z}_{i_r}^{rank} \leftarrow \frac{j_{i_r}^{rank} \mathbf{z}_{i_r}^{rank} + \mathbf{y}_r}{j_{i_r}^{rank} + 1} \quad \text{and} \quad j_{i_r}^{rank} \leftarrow j_{i_r}^{rank} + 1;$$
 this new set of $\{\mathbf{z}_{i_r}^{rank}\}$, along with the unchanged $\mathbf{z}_j^{rank}$ ($j \neq i_r$), form the new set of points $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$;
5. if the new points meet some convergence criterion, terminate; otherwise, return to step 1.

We expect that this modified parallel MacQueen's algorithm will be more efficient than Algorithm 4. Note that here $q = sp$ should not be large; thus, there is no need to do fast closest point searches because the preparation job (such as building the data structure) prior to any fast search also needs much time and therefore there is a trade off. We coded Algorithm 5 using MPI (Message Passing Interface) (48) and ran the code using up to 16 processors on the CRAY T3E-600; the results of some computational experiments are given in Table 2.4 for 128 generators on $(-1, 1)$ and the same three density functions used previously. In the table, p is the number of processors and s is the number of random points sampled by each processor at each iteration. Different values of p and s were used such that $q = ps = 160$ in all cases, i.e., the number of points sampled at each iteration over all processors is fixed. With 20,000 iterations being carried, the total number of points sampled in each case was 3,200,000.

Comparing Table 2.4 for the modified parallel MacQueen's method with Table 2.1 for the sequential MacQueen's method ($p = 1$), we see that for 3,200,000 random sampling points the average energies obtained by the two methods are very similar, i.e., independent of the number of processors. However, the running times are very much lower for the parallel method. This is seen more clearly in Figure 2.4 which gives, for each of the

Table 2.4 Maximum running times and average energies over four realizations of Algorithm 5 for a fixed number of iterations (20,000) and a fixed number (3,200,000) of random points sampled.

Algorithm 5 20,000 iterations				128 generators on (-1,1) 3,200,000 sampling points			
		density function					
		1		e^{-10x^2}		$e^{-20x^2} + \frac{1}{20} \sin^2(\pi x)$	
p	s	average energy	max time	average energy	max time	average energy	max time
1	160	5.563E-5	39.94	2.221E-5	47.91	2.812E-5	57.73
2	80	5.421E-5	26.22	2.172E-5	30.30	2.789E-5	34.88
4	40	4.976E-5	16.24	1.914E-5	18.39	2.755E-5	20.94
8	20	5.112E-5	12.17	1.837E-5	13.42	2.710E-5	14.86
16	10	5.259E-5	12.63	1.875E-5	13.30	2.715E-5	14.24

three density function, the speed up obtained by parallelization. (The average time of all similar experiments was used to compute the parallel speed ups and the results for $p = 10, \dots, 15$ are also included in this figure although they were not given in Table 2.4.) Note that the speed up appears to become sublinear as the number of processors increases, and it becomes even worse as p goes from 10 to 16 because the loss caused by the cost for communication exceeds the gain obtained from the distribution of computations. It is also worth noting that the speed up of Algorithm 5 depends strongly on the number of sampling points q used at each iteration: the larger q is, the better the speed. However, q should not be too large in this modified parallel MacQueen's method as was pointed out in the last paragraph.

2.3.2 Parallelization of the new algorithm

A direct parallel implementation of Algorithm 3 is given in the following algorithm.

Algorithm 6. (A parallel version of Algorithm 3) *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, a positive integer k , and p processors with rank $= 0, 1, \dots, p-1$;*

0. choose a positive integer s and constants $\{\alpha_i, \beta_i\}_{i=1}^2$ such that $\alpha_1 > 0$, $\beta_1 > 0$, $\alpha_1 + \alpha_2 = 1$ and $\beta_1 + \beta_2 = 1$; for $rank = 0, \dots, p-1$, set $m_{rank} = M(rank, p, k)$, $g_{rank} = G(rank, p, k)$ and $g_p = G(p, p, k)$; then, each processor independently chooses its own initial set of m_{rank} points $\{\mathbf{z}_i^{rank}\}_{i=1}^{m_{rank}}$, e.g., by using a Monte Carlo method;
1. combine them together by communication to form a complete initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$; set $j_i = 1$ for $i = 1, \dots, k$;
2. each processor selects its own set of s points $\{\mathbf{y}_r^{rank}\}_{r=1}^s$ in Ω at random according to the probability density function $\rho(\mathbf{x})$;
3. on each processor, for each generator $\mathbf{z}_i$, gather together all sampling points $\mathbf{y}_r^{rank}$ closest to $\mathbf{z}_i$, (i.e., in the Voronoi region of $\mathbf{z}_i$) in the set W_i^{rank} respectively, and denote the cardinality of W_i^{rank} by w_i^{rank} ;
4. if the set W_i^{rank} is empty, set $\mathbf{v}_i^{rank} = \mathbf{0}$; otherwise, determine the sum $\mathbf{v}_i^{rank}$ of the members of the set W_i^{rank} ;
5. on each processor, by communication, obtain the set $\{\mathbf{v}_i\}_{i=g_{rank}+1}^{g_{rank}+1}$ respectively, where $\mathbf{v}_i$ is the sum of $\mathbf{v}_i^{rank}$ over all processors, and the corresponding set $\{w_i\}_{i=g_{rank}+1}^{g_{rank}+1}$, where w_i is the sum of w_i^{rank} over all processors.
6. on each processor, for $i = g_{rank} + 1, \dots, g_{rank}+1$, if $w_i \neq 0$, compute the average $\mathbf{u}_i = \frac{1}{w_i} \mathbf{v}_i$, and then set

$$\mathbf{z}_i \leftarrow \frac{(\alpha_1 j_i + \beta_1) \mathbf{z}_i + (\alpha_2 j_i + \beta_2) \mathbf{u}_i}{j_i + 1} \quad \text{and} \quad j_i \leftarrow j_i + 1;$$
 otherwise, do nothing; by communication, this new set of $\mathbf{z}_i$, along with the unchanged $\mathbf{z}_j$ (i.e. $w_j = 0$), form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;
7. if the new points meet some convergence criterion, terminate; otherwise, return to step 1.

Note that, in step 3 of the above algorithm, searching for closest points on each processor is required. However, sequential fast searching algorithms can still be directly applied since the searching is local, i.e., totally independent between processors, and s is generally a large number in order to make the algorithm effective as was mentioned in Section 2.2.3, i.e., $q = ps$. We coded Algorithm 6, also using MPI, and ran the code using up to 16 processors of a CRAY T3E-600; the results of some computational experiments are given in Table 2.5 for 128 generators on $(-1, 1)$ and the same three density functions used previously. The same two sets of parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ used for Tables 2.2 and 2.3 are used for Table 2.5. Different values of p and s were used such that $q = ps = 8,000$, i.e., the number of points sampled at each iteration over all processors is fixed; 400 iterations were carried out so that the total number of points sampled in each case was again 3,200,000.

Comparing Table 2.4 for the modified parallel MacQueen's method with Table 2.5 for the parallel version of the new algorithm shows the clear superiority of the latter method; for the same number of sampling points and for the same number of processors, the new method results in a substantial lower energy and a substantial reduction in the running times. This is especially true as the number of processors increases.

Comparing Table 2.5 for the parallel version of the new algorithm with Table 2.2 for the sequential version ($p = 1$) shows that for the same number of sampling points, the new algorithm results in lower energies and requires much lower running times. The parallel speed up for each of the three density functions is given in Figure 2.5. Note that for Algorithm 6 the speed up in all cases is almost perfectly linear in the number of processors, and there are even some superlinear speed ups for $p = 2$ which are probably mainly caused by the cache.

Table 2.5 Average running times and average energies over four realizations of Algorithm 6 for a fixed number (400) of iterations and a fixed number (3,200,000) of random points sampled;

Algorithm 6			128 generators on $(-1, 1)$					
400 iterations			3,200,000 sampling points					
			density function					
			1		e^{-10x^2}		$e^{-20x^2} + 0.05 \sin^2(\pi x)$	
parameters values	p	s	average energy	average time	average energy	average time	average energy	average time
$\alpha_1 = 0, \alpha_2 = 1$ $\beta_1 = 0, \beta_2 = 1$	1	8,000	4.274E-5	38.85	0.706E-5	44.82	0.809E-5	55.69
	2	4,000	4.235E-5	19.77	0.751E-5	21.59	0.906E-5	26.26
	4	2,000	4.234E-5	9.63	0.722E-5	10.86	0.846E-5	13.62
	8	1,000	4.159E-5	4.99	0.795E-5	5.80	0.840E-5	6.96
	16	500	4.155E-5	2.70	0.753E-5	2.97	0.879E-5	3.64
$\alpha_1 = \alpha_2 = 1/2$ $\beta_1 = \beta_2 = 1/2$	1	8,000	4.413E-5	38.84	0.901E-5	44.37	0.977E-5	56.76
	2	4,000	4.337E-5	19.76	0.897E-5	21.88	1.085E-5	27.40
	4	2,000	4.328E-5	9.52	0.851E-5	10.90	1.085E-5	13.43
	8	1,000	4.228E-5	4.95	0.885E-5	5.79	1.015E-5	6.98
	16	500	4.150E-5	2.67	0.812E-5	3.10	1.075E-5	3.64

2.3.3 Two-dimensional computational experiments

We also carried out some experiments with Algorithms 5 and 6 for 256 generators on the two-dimensional domains $\Omega = (-1, 1)^2$ and $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$ using 32 processors on a CRAY T3E-600. There are many algorithms to efficiently compute closest neighbors using some data structures such as KD tree or binary search tree (see, e.g., (1; 4; 61)) for centroidal Voronoi generators in two dimensions; however, this is not our focus here so we did not employ them.

For Algorithm 6, the experiments use the same two sets of parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ used for the one-dimensional experiments. For both Algorithms 5 and 6, the density functions used were similar to those used in the one-dimensional experiments. Specifically, for $\Omega = (-1, 1)^2$, the density functions used were a constant, $e^{-10(x^2+y^2)}$, and $e^{-20(x^2+y^2)} + 0.05 \sin^2(\pi x) \sin^2(\pi y)$; for $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$, the density functions used were

a constant and $e^{-5(1-x^2-y^2)}$. For each iteration of Algorithm 5, 10 random points were sampled per processor; a total of 30,000 iterations were performed. For each iteration of Algorithm 6, 375 random points were sampled per processor and a total of 800 iterations were performed. Thus, for each experiment, a total of 9.6×10^6 random points were sampled. Thus, the sequential ($p = 1$) running times for the two algorithms should be about the same. The parallel running times are reported in Table 2.6. Again, s denotes the number of random points sampled by each processor at each iteration. We easily see that for all three density functions and for the same number of total sampling points, the running times for the new method (Algorithm 6) are substantially lower than for Algorithm 5, a parallel MacQueen's algorithm. Also, there is little difference between the running times for each of the two sets of parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ tested for Algorithm 6.

Naturally, one would like to know what is the quality of the centroidal Voronoi generators determined by the experiments with Algorithms 5 and 6. This is especially important in view of the observation made previously for the one-dimensional experiments with a constant density for which, even after 3,200,000 points were sampled, the energies were not always close to the energy of the true minimizer. Here, in Figures 2.6–2.9, we give visual evidence of the relative quality of solutions found using Algorithms 5 and 6 with 9.6×10^6 sampling points in $\Omega = (-1, 1)^2$. A baseline to compare with for all three density functions is provided by (very expensive) converged (with respect to the energies) deterministic solutions obtained using the deterministic Lloyd's method (Algorithm 2). Then, comparing Figure 2.6 with Figures 2.7–2.9 shows that, for the same total number of sampling points, Algorithm 6 with either choice for the parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ provides substantially superior results than does Algorithm 5. Of course, the latter algorithm provides vastly better results than does just a single random sampling of points according the density function. Thus, it seems that Algorithm 6 is superior to Algorithm 5 with respect to both speed and accuracy. We also provide, in

Table 2.6 Maximum running times for four realizations of Algorithms 5 and 6 for a fixed number (16) of processors and a fixed number (9.6×10^6) of random points sampled.

Two-dimensional simulations				
Algorithm 5				
Algorithm 6a ($\alpha_1 = 0, \alpha_2 = 1, \beta_1 = 0, \beta_2 = 1$)				
Algorithm 6b ($\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 1/2$)				
32 processors	256 generators	9,600,000	sampling points	

$\Omega = (-1, 1)^2$				
density	algorithm	s	iterations	time
1	5	10	30,000	867.3
	6a	375	800	329.6
	6b	375	800	329.1
$e^{-10(x^2+y^2)}$	5	10	30,000	1012.6
	6a	375	800	436.7
	6b	375	800	435.9
$e^{-20(x^2+y^2)} + \frac{1}{20} \sin^2(\pi x) \sin^2(\pi y)$	5	10	30,000	1246.2
	6a	375	800	680.4
	6b	375	800	684.1

$\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$				
density	algorithm	s	iterations	time
1	5	10	30,000	869.5
	6a	375	800	332.2
	6b	375	800	331.2
$e^{-5(1-x^2-y^2)}$	5	10	30,000	953.1
	6a	375	800	381.8
	6b	375	800	380.9

Figures 2.10 and 2.11, some centroidal Voronoi diagrams found using Algorithms 5 and 6 with 9.6×10^6 sampling points in the circle $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$. The results again show that Algorithm 6 performs much better than Algorithm 5.

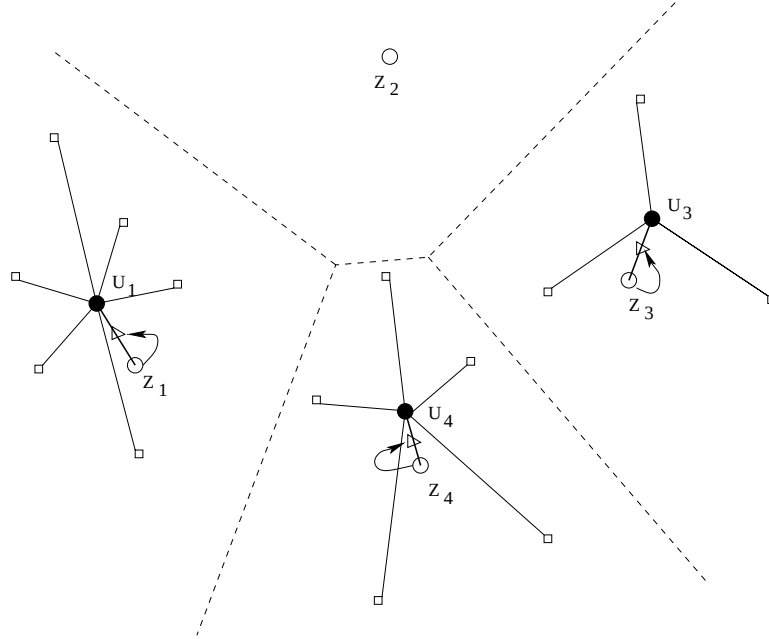


Figure 2.2 A visual description of Algorithm 3. The open circles denote the current positions of the generators and the dashed lines denote their corresponding Voronoi tessellation. The small squares denote the points sampled during a single iteration of the algorithm. These are grouped according to which Voronoi cell they belong to and then all sample points in each cell are averaged to produce the filled circles. A triangle denotes the new position of a generator which is determined by taking a linear combination of the old position (the open circle) and the local sample average (the filled circle). Note that since no points are sampled in the Voronoi region for z_2 , the position of that generator remains unchanged. The Voronoi regions as depicted only for illustrative purposes; Algorithm 3 does not require the construction of the Voronoi tessellation.

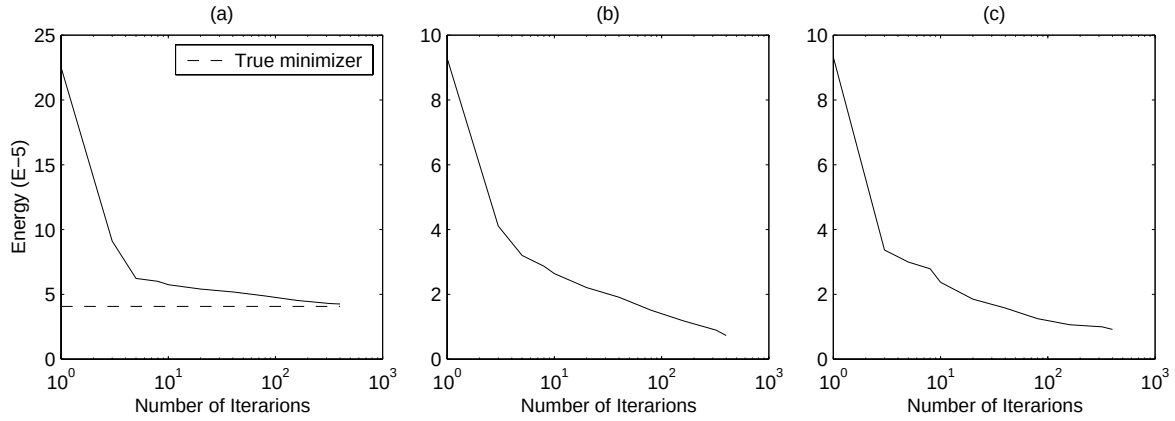


Figure 2.3 Energy vs. number of iterations for Algorithm 3 with $q = 8000$ and $\{\alpha_1 = 0, \alpha_2 = 1, \beta_1 = 0, \beta_2 = 1\}$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$.

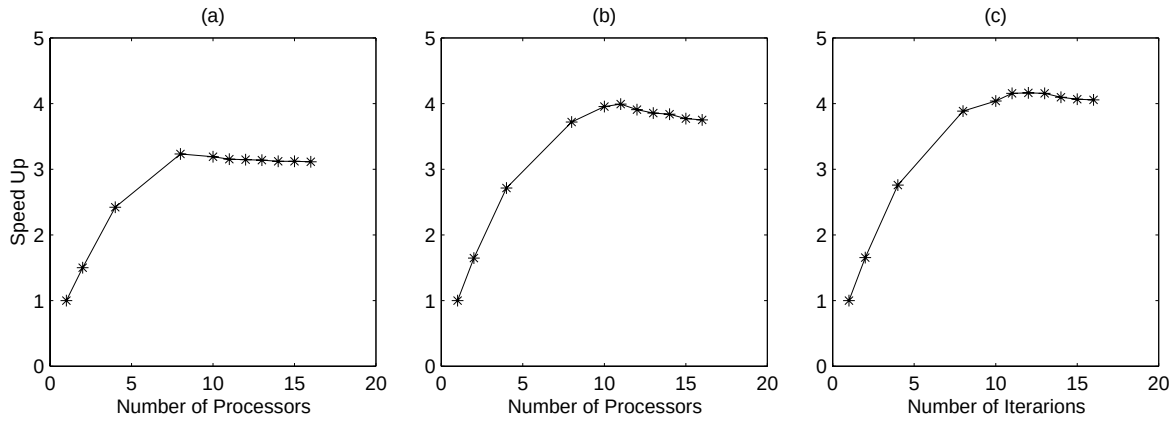


Figure 2.4 Speed up of Algorithm 5 with $q = ps = 160$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$.

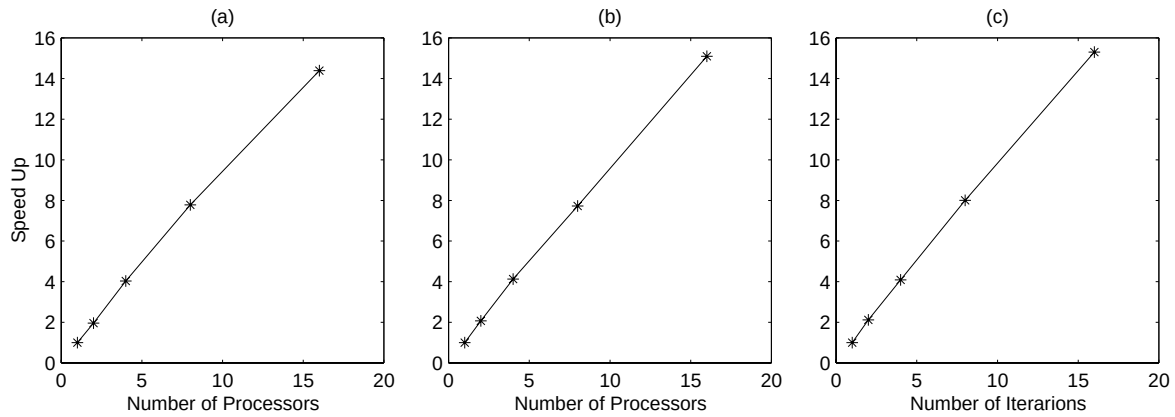


Figure 2.5 Speed up of Algorithm 6 with $q = ps = 8,000$. (a) $\rho(x) = 1$; (b) $\rho(x) = e^{-10x^2}$; (c) $\rho(x) = e^{-20x^2} + 0.05 \sin^2(\pi x)$.

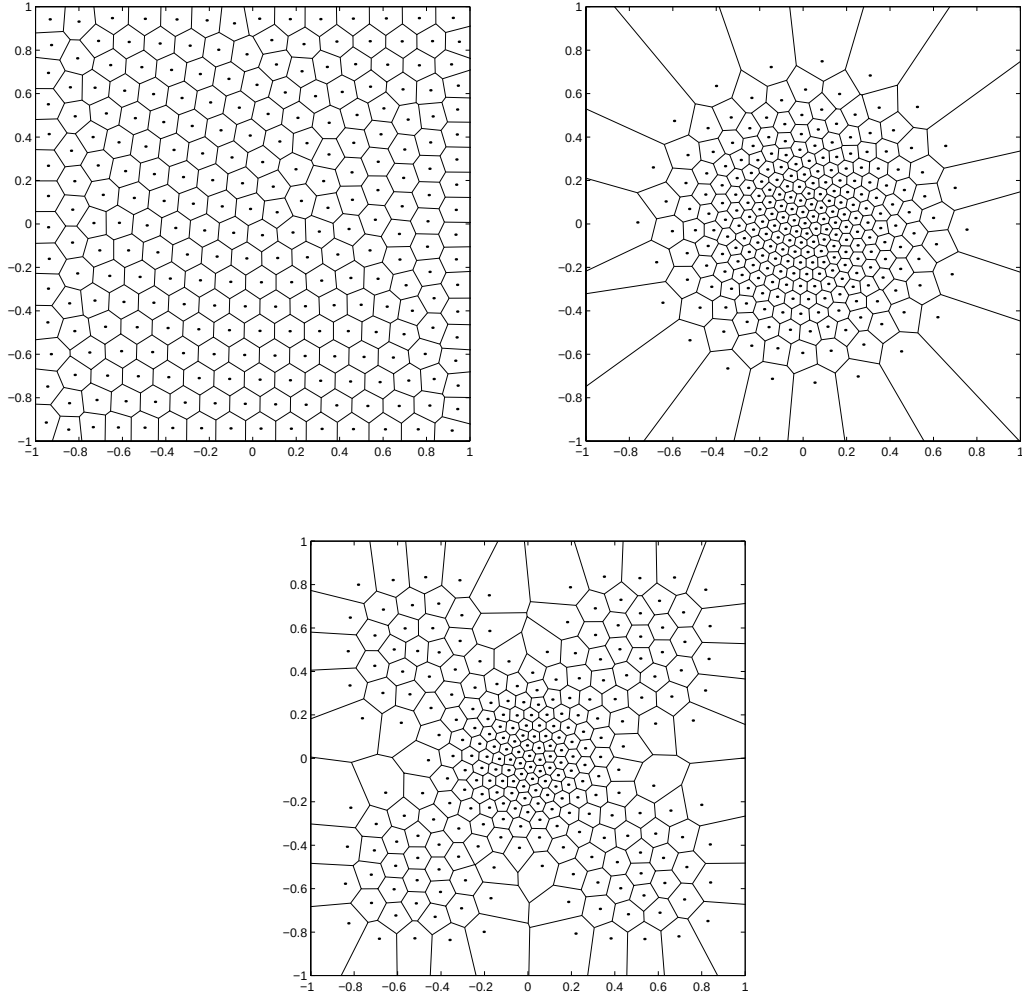


Figure 2.6 Two-dimensional centroidal Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ found by the deterministic Lloyd's method (Algorithm 2); top-left: $\rho(x, y) = 1$; top-right: $\rho(x, y) = e^{-10(x^2+y^2)}$; bottom: $\rho(x, y) = e^{-20(x^2+y^2)} + 0.05 \sin^2(\pi x) \sin^2(\pi y)$.

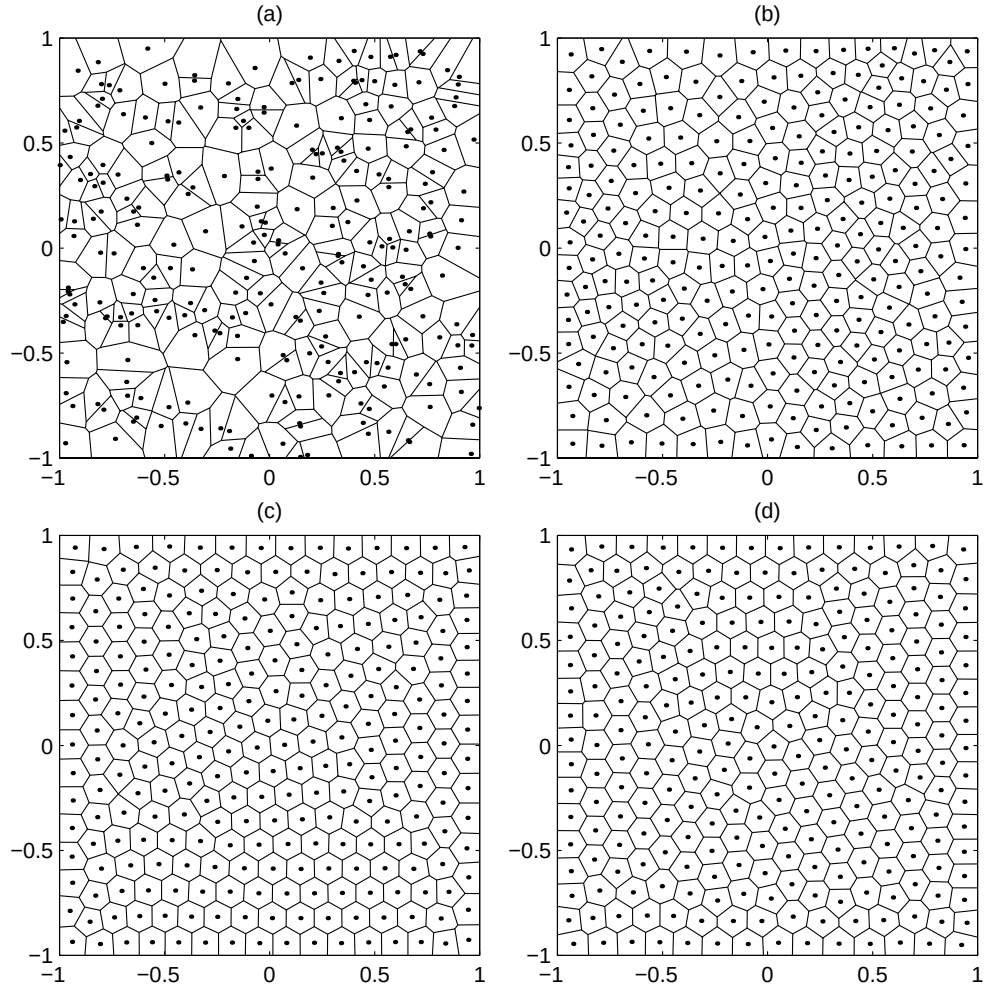


Figure 2.7 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = 1$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).

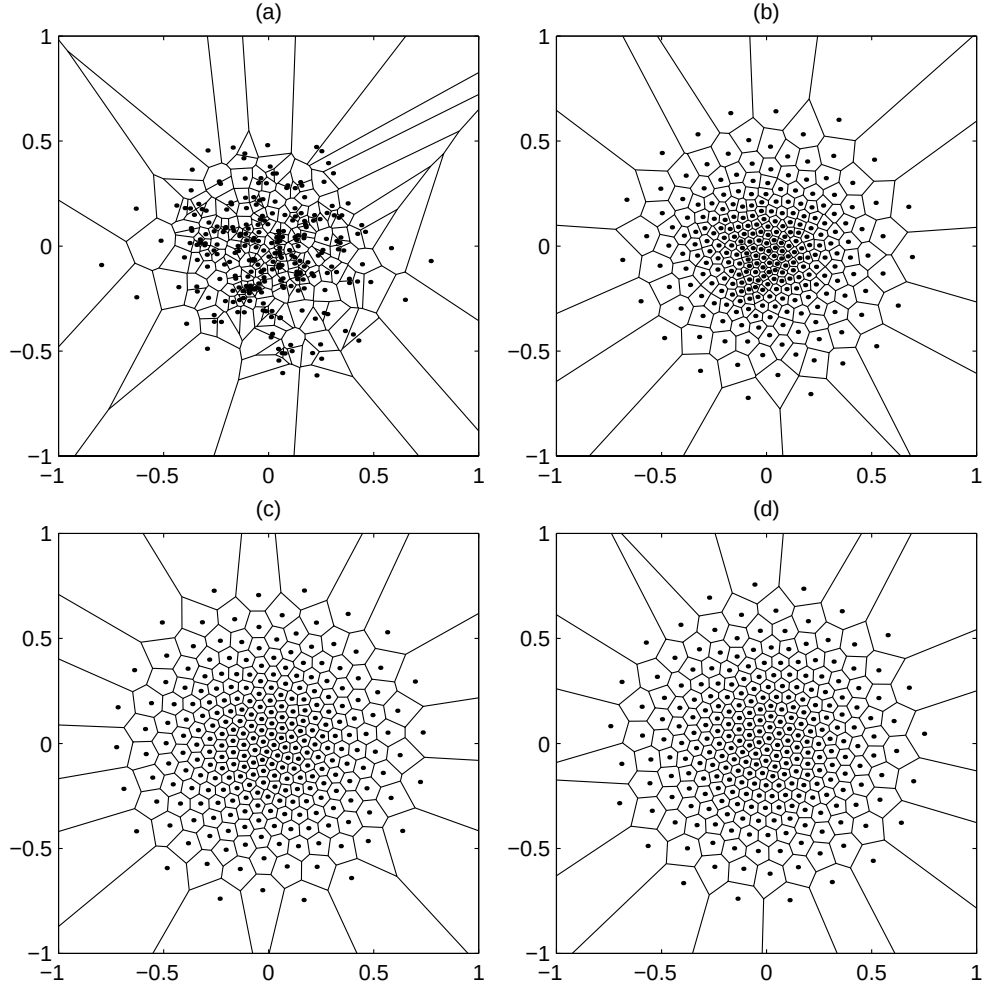


Figure 2.8 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = e^{-10(x^2+y^2)}$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).

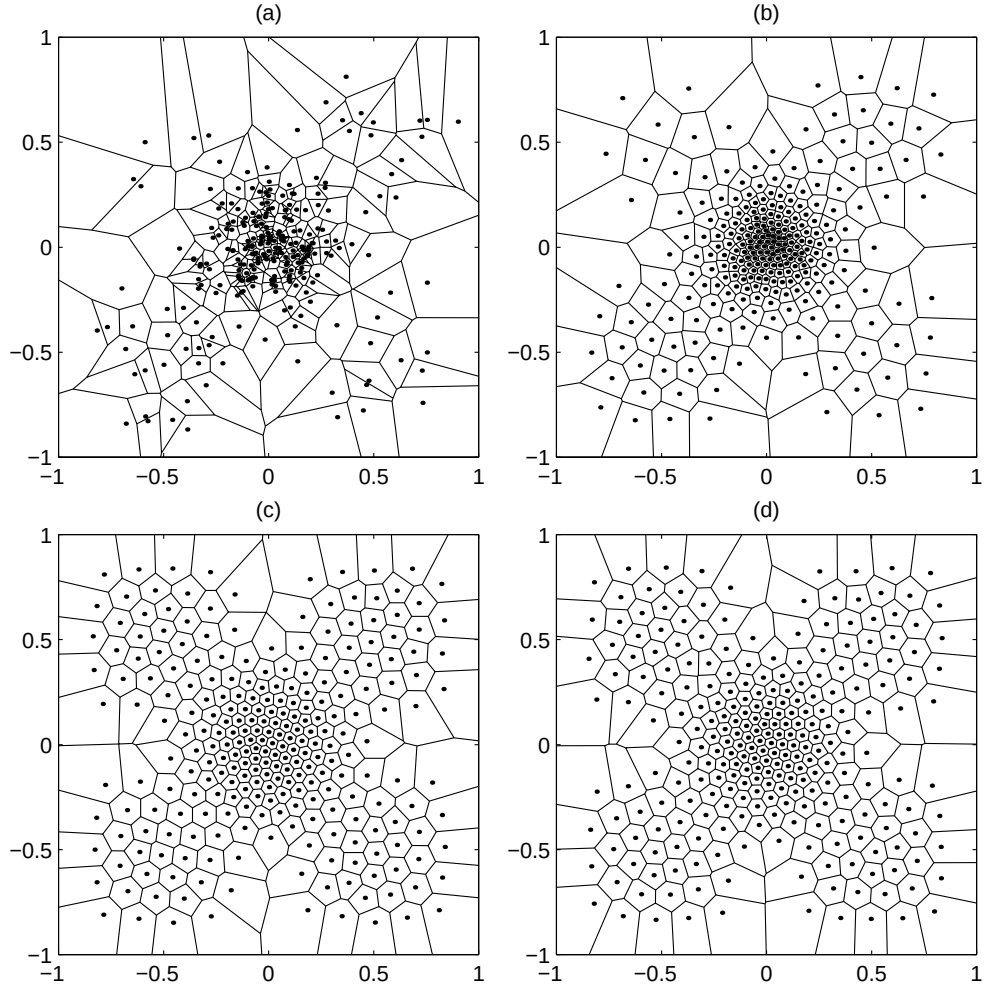


Figure 2.9 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = (-1, 1)^2$ with the density function $\rho(x, y) = e^{-20(x^2+y^2)} + 0.05 \sin^2(\pi x) \sin^2(\pi y)$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).

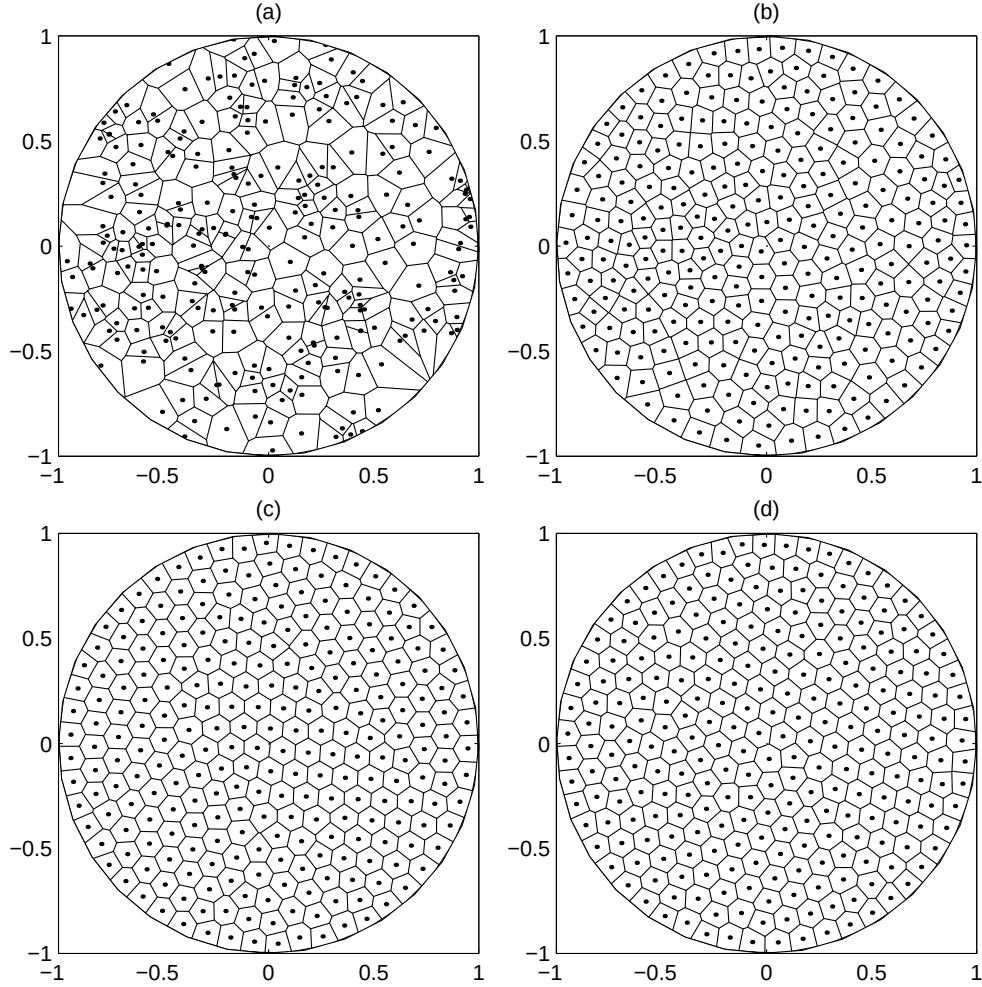


Figure 2.10 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = \{(x, y) \mid (x^2 + y^2 < 1)\}$ with the density function $\rho(x, y) = 1$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).

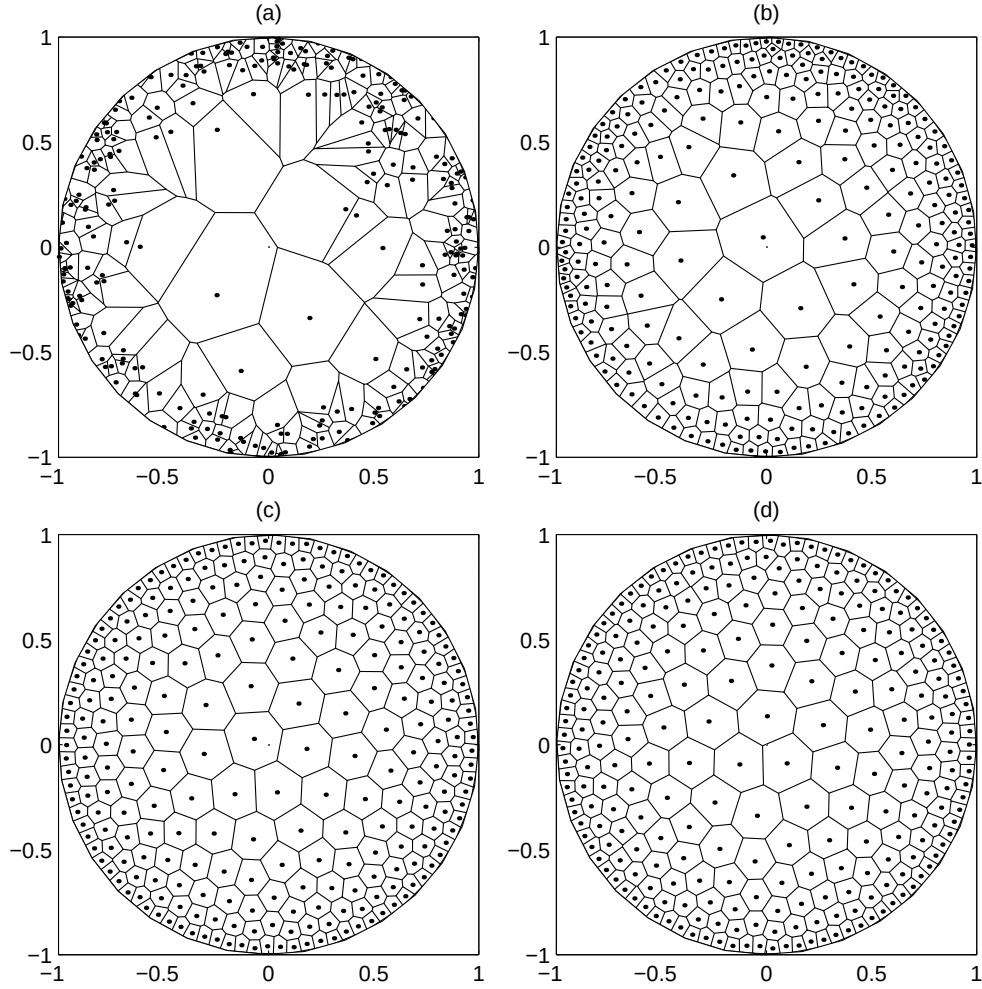


Figure 2.11 Two-dimensional Voronoi diagrams for 256 generators in $\Omega = \{(x, y) \mid (x^2 + y^2) < 1\}$ with the density function $\rho(x, y) = e^{-5(1-x^2-y^2)}$; (a) Monte Carlo simulation; (b) approximate centroidal Voronoi diagram using Algorithm 5; (c) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$; (d) approximate centroidal Voronoi diagram using Algorithm 6 with $\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = 0.5$; 9.6×10^6 sampling points were used for (b), (c), and (d).

CHAPTER 3. GENERATION OF POINT SETS AND SUPPORT REGIONS FOR MESHLESS COMPUTING

In this chapter, we present probabilistic (and totally messfree) algorithms for point placement and support sphere determination in any domain; the algorithms are very well suited for adaptation and parallelization and produce very high-quality point sets. First we give a short introduction about the current status of meshless computing. In §3.2 and §3.3, we respectively discuss point placements using centroidal Voronoi tessellations and the open covering of the domain using spheres. (Spherical coverings are considered only for ease of exposition; our methodologies apply just as well to other support region geometries.) In §3.4, some numerical experiments are performed to compare our methodologies with the Monte Carlo method for uniform and nonuniform point distributions; in addition, for uniform point distributions, we compare with point sets generated via Halton sequences.

3.1 Introduction

It is well known that mesh generation is a very difficult endeavor for problems with complicated geometries. As a result, there is growing interest in seeking alternative approaches for numerical simulations which involve no mesh at all. These approaches are called *meshless* or *meshfree* or *gridless* methods. Of course, gridless methods are not a new notion; for example, Monte Carlo and particle methods and even polynomial approximation methods for numerical integration, function approximation, or for ap-

proximating the solutions of differential equations are meshless-type methods that have been around for decades. The newer meshless methods that are of interest here take a different approach. The main ideas are to generate a set of points in a given domain; attach a patch to each point such that the union of the patches covers the domain; construct local shape functions having support over the patches; and then, using some data fitting method, set up an algebraic system of equations. Several realizations of these approaches using different data fitting methods have been proposed, including partition of unity methods (PUM) (8), element-free Galerkin methods (7), hp-clouds (15), and reproducing kernel methods (44).

The generation of a point set and the associated patches is always the first step in efficient meshless methods. Many previous algorithms for these steps are complicated since they involve the point set of a mesh, i.e., they are not truly meshless. There exist several algorithms for point set generation which avoid this flaw. In (9), an approach is proposed that uses Voronoi diagrams and weighted bubble packing. Another good simplicial algorithm for the generation of point sets and the associated patches is the biting method introduced in (46) which uses an advancing front strategy for generating the spherical patches. In (23), a meshless discretization technique is suggested for nonstationary convection-diffusion problems where the point set is formed via a Halton sequence instead of by sampling uniformly distributed points. However, all the above methods do not lend themselves well to adaptive techniques so that the implementation of local refinement or coarsening strategies of point sets based on these methods is not a simple matter. For example, in the method of (23), coarser approximations in specific regions are effected just by reducing the number of points used; since no natural multilevel Halton sequence can be given, this leads to a multilevel approach in which non-nested spaces arise. Another example is provided by moving least square methods (MLSM) (6) which are used to build shape functions on the domain of interest and result in the need for multiple coverings of the domain so that the biting method cannot be

used. Still another example is provided by Monte Carlo methods associated with some density function; these are natural approaches for adaptive techniques, but the results are not good in general because the points placement is not sufficiently regular and it also requires a very good random number generator.

3.2 Point placement using centroidal Voronoi tessellations

As stated in Chapter 1, for an arbitrary set of points $\{\mathbf{x}_i\}_{i=1}^n$ in some domain Ω with some density function $\rho(\mathbf{x})$, these points generally are not the mass centroids of the corresponding Voronoi regions (13). Thus, we are faced with the construction problem: given a positive integer n , a domain Ω , and a density function $\rho(\mathbf{x})$ defined on $\overline{\Omega}$, find a centroidal Voronoi tessellation of Ω . Several good algorithms for determining centroidal Voronoi tessellations for both discrete and continuous sets Ω have been proposed and discussed in Chapter 2, including MacQueen's method (47) which is a random sampling algorithm and Lloyd's method (45) which is the obvious (deterministic) iteration between finding centroids and Voronoi tessellations. However, MacQueen's method converges very slowly and even fails to give the optimal centroidal Voronoi regions in some situations. Lloyd's method can generally obtain good results but requires accurate calculations of Voronoi diagrams and volumes of Voronoi regions; these are difficult operations for complicated geometries in three dimensions and, in any case, are relatively expensive.

Another probabilistic method, Algorithm 3 stems from a combination of the MacQueen and Lloyd methods. The Monte Carlo method, i.e., random sampling according to a given density function, plays a key role in this algorithm; we use the rejection method for random point generation; it is much faster than the classical Monte Carlo procedure using numerical integrations and is also much better for dealing with complicated regions; see Chapter 2. There also are a number of algorithms for efficiently determining

closest neighbors which use some data structures for Algorithm 3; see (1; 4; 61). The complexity of Algorithm 3 is $O(q \log(n))$ per iteration. The results of numerical experiments in Chapter 2 also showed that Algorithm 3 can produce almost the same good results as does the deterministic (and certainly not meshless) Lloyd's method but without requiring the calculation of Voronoi regions. Note that in the context of probabilistic algorithms such as Monte Carlo methods and Algorithm 3, uniform point distributions mean that the density function is constant.

We will take the generators $\{\mathbf{x}_i\}_{i=1}^n$ of centroidal Voronoi tessellations as point sets for meshless discretizations. In fact, they can be regarded as improvements on point sets generated via Monte Carlo, e.g., random sampling, methods.

In practical applications, many meshless methods require the imposition of certain constraints on the positions of the points; for example, it may be required that some of the points lie on the geometrical boundary (or more generally, on surfaces or lines in three dimensions) so that boundary conditions can be handled. To satisfy such constraints on a point set, a constrained CVT algorithm is required. There are different approaches for constructing constrained CVT's; see (14). One approach is to predetermine a subset of points along the boundary via a lower-dimensional CVT construction based on another pre-selected density function defined on the boundary. A second approach is to construct the CVT using a standard algorithm with the modification that at each iteration, the set of generators whose corresponding Voronoi regions contain a portion of the boundary of the domain are projected onto the geometrical boundary. A drawback of the first approach is that, in practice, it may not be a simple matter to determine a priori the number of generating points on the boundary and the corresponding boundary density function. The second approach, on the other hand, is totally automatic and is free from the need to predetermine the number of boundary points or boundary densities. In (14), a variational formulation is also proposed which covers both approaches mentioned above.

For the numerical experiments of this paper, the second approach is taken since it is a simple matter to probabilistically determine the set of generators whose corresponding Voronoi regions contain a portion of the boundary of the domain. For example, one could a priori sample a set of boundary points; then, for each point on the boundary point set, one searches for the closest generators; then one knows that the corresponding Voronoi region will contain a portion of the boundary of the domain.

3.3 Sphere supports and overlaps

Suppose that a collection of n points $\{\mathbf{x}_i\}_{i=1}^n$ has been chosen in the domain $\overline{\Omega}$ according to a given density function $\rho(\mathbf{x})$ using the generators of the centroidal Voronoi tessellation of $\overline{\Omega}$. Then, to each point $\mathbf{x}_i$, a spherical patch

$$S_i = \{\mathbf{y} \in \mathbb{R}^N \mid \|\mathbf{y} - \mathbf{x}_i\| \leq h_i\} \quad (3.1)$$

is attached, where $h_i \in \mathbb{R}^+$ is the radius of the sphere S_i . In our methodology, the patches can also be easily chosen to have other shapes. e.g., ellipsoidal, rectangular, or brick type objects; for simplicity of exposition, here we restrict ourselves to spherical patches. The construction of the patches S_i , e.g., the covering of the domain $\overline{\Omega}$ by spherical regions, is a very important step in the discretization process because, e.g., the S_i 's define the supports of the trial and test functions of a Galerkin method. The S_i 's have to satisfy the covering condition $\overline{\Omega} \subset \cup_{i=1}^k S_i$. Furthermore, the intersection of $S_i \cap S_j$ of two patches should be sufficiently large if it is not empty, e.g., if the intersection is too small, it may be mistakenly identified to be zero when using finite precision arithmetic. An algorithm for determining the size of these patches was proposed in (23). Although the algorithm there applies to square patches, it can easily be changed for use for spherical patches. That algorithm involves a coarse mesh which often has to be refined locally; this is not so simple when the points $\{\mathbf{x}_i\}_{i=1}^n$ are nonuniformly distributed. Thus, we define a new algorithm to avoid these flaws and which take advantage of properties of

centroidal Voronoi tessellations such as the fact each point has an associated Voronoi region which is generally a polyhedron.

We use the algorithm given in (23) as a basis for an algorithm which is totally meshfree.

Algorithm 7. *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, and a set of points $\{\mathbf{x}_i\}_{i=1}^n$ in Ω ;*

0. *choose positive integers m_1 and m_2 and a constant $\gamma > 1$ and set $h_i = 0$ for all $i = 1, 2, \dots, n$;*
1. *select a set of m_1 points $\{\xi_i\}_{i=1}^{m_1}$ uniformly distributed over Ω by, e.g., a Monte Carlo method; select another set of m_2 points $\{\eta_j\}_{j=1}^{m_2}$ distributed over Ω according to the density function $\rho(x)$, e.g., again by a Monte Carlo method; let $\mathcal{P} = \{\xi_i\}_{i=1}^{m_1} \cup \{\eta_j\}_{j=1}^{m_2}$;*
2. *for all $\mathbf{y} \in \mathcal{P}$:*
 - *find a $\mathbf{x}_{i^*}$ in $\{\mathbf{x}_i\}_{i=1}^n$ which is closest to $\mathbf{y}$;*
 - *compute the distance $d_{i^*} = \|\mathbf{y} - \mathbf{x}_{i^*}\|$;*
 - *if $h_{i^*} < d_{i^*}$, then set $h_{i^*} = d_{i^*}$;*
3. *set $h_i = \gamma h_i$ for all $i = 1, 2, \dots, n$.*

The points $\{\xi_i\}_{i=1}^{m_1}$ represent the domain Ω and the points $\{\eta_j\}_{j=1}^{m_2}$ provide a further refinement according to the density function $\rho(\mathbf{x})$. Jointly, the points $\{\xi_i\}_{i=1}^{m_1} \cup \{\eta_j\}_{j=1}^{m_2}$ are used as pseudo-points to guarantee that the patches, i.e., the spheres, $\{S_i\}_{i=1}^n$ cover the entire domain Ω . The parameter γ is motivated by the requirement that the overlaps of intersecting patches be sufficiently large and is also used to control the smoothness of the shape functions and the density of the stiffness matrix. The overall complexity of Algorithm 7 is $O(\text{card}(\mathcal{P}) \log(n))$.

The ideas behind Algorithm 7 are as follows. First, we find a discrete approximation of the Voronoi regions corresponding to $\{\mathbf{x}_i\}_{i=1}^n$. Then, for each $i = 1, 2, \dots, n$, we determine the distance between the generator $\{\mathbf{x}_i\}$ and the farthest point in its region. If the number of the pseudo-points is large, then the sphere with $\{\mathbf{x}_i\}$ as its center and the furthest point distance as its radius should almost cover the Voronoi region associated with $\mathbf{x}_i$. Then, the union of all these spheres should come close to covering the domain Ω . Since the Voronoi regions are polyhedra in general, the covering is optimal in some sense. The parameter γ is then used to guarantee the complete covering of the entire domain Ω .

Algorithm 7 is generally not suitable if moving least square methods are used to construct the shape functions and if one wishes to obtain more than one order of consistency; this is the case for hp-clouds (15), the element-free Galerkin method (7), and reproducing kernel methods (44). The reason for this is that, in order to make the method work, one needs the points in Ω to be covered multiple times. For example, for a given interger $p > 1$, which depends on the basis space used, it is required that for any $\mathbf{y} \in \overline{\Omega}$, there exist indices $i_1, i_2, \dots, i_k$, $k \geq 2 - p$, such that $\mathbf{y} \in \cap_{j=1}^k S_{i_j}$, i.e., each point in $\overline{\Omega}$ belongs to at least p of the support sets S_i . An algorithm for determining a covering of square support sets satisfying this requirement is given in (15); it may be easily ammended to apply to the case of spherical support sets. Here, we suggest a meshless variant of that algorithm.

Algorithm 8. *Given a region Ω , a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \overline{\Omega}$, an integer $p > 1$, and a set of points $\{\mathbf{x}_i\}_{i=1}^n$ in Ω ;*

0. *choose positive integers m_1 and m_2 and a constant $\gamma > 1$ and set $h_i = 0$ for all $i = 1, 2, \dots, n$;*
1. *select a set of m_1 points $\{\xi_i\}_{i=1}^{m_1}$ uniformly distributed over Ω by, e.g., a Monte Carlo method; select another set of m_2 points $\{\eta_j\}_{j=1}^{m_2}$ distributed over Ω according to the*

density function $\rho(x)$, e.g., again by a Monte Carlo method; let $\mathcal{Q} = \{\xi_i\}_{i=1}^{m_1} \cup \{\eta_j\}_{j=1}^{m_2} \cup \{\mathbf{x}_i\}_{i=1}^n$;

2. For all $\mathbf{y} \in \mathcal{Q}$:

- determine the set $S_{y,R}$ of all points $\mathbf{x}_i$ that fall within a searching sphere B_R which is centered at $\mathbf{y}$ and whose radius is equal to R ; if $\text{card}(S_{y,R}) < p$, then increase R and try again; denote the points in the set $S_{y,R}$ by $\mathbf{x}_{k(j)}$, $j = 1, 2, \dots, \text{card}(S_{y,R})$;
- determine the distance $d_{k(j)} = \|\mathbf{y} - \mathbf{x}_{k(j)}\|$ for all $\mathbf{x}_{k(j)} \in S_{y,R}$, i.e., for $j = 1, 2, \dots, \text{card}(S_{y,R})$;
- sort the set $\{d_{k(j)}\}_{j=1}^{\text{card}(S_{y,R})}$ in increasing order;
- for $j = 1, 2, \dots, p$, if $h_{k(j)} < d_{k(j)}$, then set $h_{k(j)} = d_{k(j)}$;

3. set $h_i = \gamma h_i$ for all $i = 1, 2, \dots, n$.

An efficient implementation of the evaluation of the set $S_{y,R}$, which is a crucial step in this algorithm, is given in (63). The overall complexity of Algorithm 8 is $O(\text{card}(\mathcal{Q}) \log(n))$. Note that in Algorithm 7, the set $\mathcal{P} = \{\xi_i\}_{i=1}^{m_1} \cup \{\eta_j\}_{j=1}^{m_2}$ was used because $\mathbf{x}_i$ is the center of S_i , i.e., we already have $\mathbf{x}_i \in S_i$ for each i , but $\{\mathbf{x}_i\}_{i=1}^n$ is added to $\mathcal{Q}$ in Algorithm 8 since now $p > 1$ and each $\mathbf{x}_i$ also has to be covered, in addition to S_i , by at least $p - 1$ other support regions.

3.4 Numerical Experiments

We now provide the results of some numerical experiments for square, circular, and non-convex regions with both uniform and nonuniform density functions. We will compare centroidal Voronoi point placements (obtained using Algorithm 3) with those obtained using the Monte Carlo method, i.e., random sampling. In addition, for a constant

density function, i.e., for a uniform distribution of points, we also make comparisons with the point sets generated via a (2,3) Halton sequence.¹ In each case, we will use sphere coverings resulting from the application of both Algorithms 7 and 8, the latter with $p = 3$; this is a typical requirement for the cover when using moving least square methods to construct shape functions in two dimensions. We choose $\gamma = 1.3$ throughout.

For convenience of description, the characteristic matrix $M = (M_{ij})$ of a sphere covering $\{S_i\}_{i=1}^n$ of the domain Ω is defined by

$$M_{ij} = \begin{cases} 1 & \text{if } S_i \cap S_j \neq \emptyset \\ 0 & \text{otherwise.} \end{cases}$$

Clearly, the density of M , i.e., the percentage of non-zero entries, is in general proportional to the density of the stiffness matrix which, in practice, one hopes is a sparse matrix.

Except where noted, for all the figures we chose the number of points $n = 128$. However, we also examined the cases $n = 256$ and $n = 512$ in order to gather statistics about the densities of the corresponding characteristic matrices.

3.4.1 Uniform point distributions

We give a visual comparison of three different point placement schemes for uniform point distributions. For the Monte Carlo (random point placement) and centroidal Voronoi point placements, uniform point distributions means that the density function is constant.

First, we choose the square region $\Omega = [-1, 1] \times [-1, 1]$. In Fig. 3.1, the top row corresponds to a Monte Carlo point set, the middle row corresponds to a (2,3) Halton

¹Halton sequences are pseudo-Monte Carlo sequences which can be described as follows. Given a prime number p , for $k \in \mathbb{N}$, k can be represented as $k = \sum_i k_i p^i$. We then define a mapping H_p from $\mathbb{N}$ to $[0, 1]$ by $H_p(k) = \sum_i k_i / p^{i+1}$. Then, the (p, q) Halton sequence of n points in two dimensions is defined as $\{H_p(k), H_q(k)\}_{k=1}^n$. To generate points in a unit circle through Halton sequences, the following mapping, which is a variant of the mapping in (64), can be used: $(H_p(k), H_q(k)) \mapsto (\sqrt{1 - H_p(k)^2} \cos(2\pi H_q(k)), \sqrt{1 - H_p(k)^2} \sin(2\pi H_q(k)))$.

sequence point set, and the bottom row to a centroidal Voronoi point set, the first and third of these generated from the density function $\rho(x) = 1$. The first column shows the placement of the 128 points and the second and third columns the coverings of Ω by circular support regions determined by Algorithms 7 and 8, respectively, with $p = 3$ for the latter. We then choose the circular region $\Omega = \{(x_1, x_2) \mid x_1^2 + x_2^2 \leq 1\}$, i.e., the unit circle, for which Fig. 3.2 provides the same information as did Figures 3.1 for the square domain. Then, in Table 3.1, we provide some information about the sphere coverings. Specifically, we provide, for 128, 256, and 512 point sets, for both the square and circular domains, for the three point placement methods (Monte Carlo, Halton sequence, and centroidal Voronoi), and for the two sphere radii selection methods (Algorithms 7 and 8 with $p = 3$), the densities, i.e., the percentage of nonzero entries, of the characteristic matrices of the corresponding coverings.

Table 3.1 Densities of nonzero entries of the characteristic matrices of coverings of square and circular domains with uniform point distributions.

No. of points	Algorithm	Point selection method		
		Monte Carlo	Halton sequence	CVT
$\Omega = [-1, 1] \times [-1, 1]$				
128	7	12.02%	10.04%	5.05%
	8 (with $p = 3$)	25.41%	21.63%	18.68%
256	7	6.25%	4.95%	2.61%
	8 (with $p = 3$)	14.23%	10.97%	9.53%
512	7	3.29%	2.72%	1.33%
	8 (with $p = 3$)	7.30%	5.91%	4.72%
$\Omega = \{(x_1, x_2) \mid x_1^2 + x_2^2 \leq 1\}$				
128	7	11.90%	9.03%	5.22%
	8 (with $p = 3$)	24.87%	19.12%	20.94%
256	7	6.85%	4.20%	2.61%
	8 (with $p = 3$)	14.99%	11.11%	9.58%
512	7	3.47%	2.34%	1.33%
	8 (with $p = 3$)	7.64%	5.50%	4.62%

From Figures 3.1 and 3.2 and Table 3.1, one easily concludes that point placement

using centroidal Voronoi tessellations results in better point distributions and sphere coverings than do the other methods. Specifically, the point distributions and associated circular patches are much more uniform and the densities of the corresponding characteristic matrices are lower. Note that the densities for Algorithm 8 are higher than for Algorithm 7; this is to be expected since with $p = 3$ Algorithm 8 guarantees (in a probabilistic sense) that each point in Ω is covered by at least three of the circular patches. Note also the inverse linear dependence of the densities on the number of points. We further observe (this is easiest to see for the square domain) that the characteristic matrix densities obtained for the centroidal Voronoi point selection method along with Algorithm 7 for determining the radii for the circular patches are very much the same as that which one would obtain for a piecewise linear finite element grid. If Algorithm 8 with $p = 3$ is used instead, then the characteristic matrix densities are very much the same as for a piecewise quadratic finite element grid. If either of the other point selection methods are used, the corresponding matrix densities are substantially higher.

We describe how centroidal Voronoi point placements were obtained for circular domains since it helps to point out a very favorable feature of our completely probabilistic method of determining point placements and support radii. We embed the circular domain into a square domain and sample points in the square; if a sampled point lies outside the circle, we simply reject it. This is equivalent to defining the density function to vanish outside the circle. This procedure can be generalized to arbitrary convex domains; one simply embeds the domain into a rectangular domain, then one samples on the rectangular domain and rejects points lying outside the given domain (or, equivalently, one extends the density function by zero outside of the given domain and then samples on the rectangular domain.) Ultimately, we only need the ability to determine if a point is inside or outside of the given domain. Non-convex domains need an additional rejection step; see Section 3.4.3.

3.4.2 Nonuniform point distributions

In Figures 3.3–3.7, we present, for Monte Carlo and centroidal Voronoi point placements, the same information as is found in Figures 3.1 and 3.2, except that now we consider nonuniform point distributions. Specifically, for Figures 3.3–3.5, which are for the square domain $[-1, 1] \times [-1, 1]$, the points are respectively distributed according to the density functions $e^{-3(x^2+y^2)}$ (which has a peak at the center of the square), $e^{-1.5(2+x+y)}$ (which has a peak at the bottom-left corner of the square), and $e^{-(1+x)^2}$ (which peaks at the left side of the square.) For Figures 3.6 and 3.7, which are for the circular domain $\{(x_1, x_2) \mid x_1^2 + x_2^2 \leq 1\}$, the points are respectively distributed according to the density functions $e^{-4(x^2+y^2)}$ (which has a peak at the center of the circle) and $e^{-3(1-x^2-y^2)}$ (which peaks at the circumference of the circle).

Table 3.2 provides the densities, i.e., the percentage of nonzero entries, of the characteristic matrices for circular coverings associated with 128, 256, and 512 points generated for the two domains (square and circle), the two point placement methods (Monte Carlo and centroidal Voronoi), the two radii selection methods (Algorithms 7 and Algorithm 8 with $p = 3$), and for the nonuniform density functions given above. From Figures 3.3–3.7 and Table 3.2, we again see the clear superiority of the centroidal Voronoi point placement method.

3.4.3 Non-convex domains

Our point placement and support region selection methods are easily adapted to non-convex domains with relatively well-behaved boundaries. Only an additional rejection step is necessary in Algorithm 3 and a change in the closest point determination may be necessary; otherwise, all the algorithms we have described remain unchanged. The change to Algorithm 3 is simply that whenever the point location $\mathbf{x}_i^*$ computed according to step 3 of that algorithm lies outside the domain, then it is rejected. (This cannot

occur for convex domains but is possible for non-convex domains.) Equivalently, if the density function at that point vanishes, i.e., if $\rho(\mathbf{x}_i^*) = 0$, then the point $\mathbf{x}_i^*$ is rejected.

For nonconvex domains having ill-behaved boundaries such as slits or cracks, it is possible that the line segment connecting a sampled point to the nearest (in terms of Euclidean distance) generator crosses the boundary of the domain. This situation should be rejected, i.e., Voronoi regions should not have part of the domain boundary traversing through them. The rejection of such situations can be effected automatically through a simple redefinition of the notion of distance. If $\partial\Omega$ denotes the boundary of Ω , we redefine the distance used for searching for closest points in all algorithms as follows. Given two points $\mathbf{y}_1$ and $\mathbf{y}_2$ in Ω , let $\overline{\mathbf{y}_1\mathbf{y}_2}$ denote the line segment between $\mathbf{y}_1$ and $\mathbf{y}_2$. Then, the distance between $\mathbf{y}_1$ and $\mathbf{y}_2$ is given by

$$d(\mathbf{y}_1, \mathbf{y}_2) = \begin{cases} \infty & \text{if } \partial\Omega \cap \overline{\mathbf{y}_1\mathbf{y}_2} \neq \emptyset \\ \|\mathbf{y}_1 - \mathbf{y}_2\| & \text{otherwise.} \end{cases}$$

(Note again that for convex domains, we always have that $\partial\Omega \cap \overline{\mathbf{y}_1\mathbf{y}_2} = \emptyset$.)

We first illustrate the superior performance of our algorithms for point placement and support radii determination for non-convex domains in Figures 3.8–3.9 where for three different non-convex domains, we provide (for the Monte Carlo and centroidal Voronoi point placement methods) the same information as found in Figures 3.1 and 3.2 for a square and circle. Figures 3.8–3.9 are for a uniform density function; there is no difficulty applying our methods to nonuniform density functions over these non-convex domains.

As a further illustration, we next consider the square region $\Omega = [-1, 1] \times [-1, 1]$ with a slit $L = \{(0, y) | y \in [-0.5, 0.5]\}$. As noted above, the Voronoi region of any point cannot cross over the slit, i.e., any two points should not influence each other if the line segment connecting them intersects the slit. Numerical experiments were performed for both a uniform and a nonuniform density function; see Figures 3.10 and 3.11 for a

depiction of the placement of points and support radii; here 256 points are used. The density function for Fig. 3.11 is given by

$$\rho(x, y) = \begin{cases} e^{-10r} & \text{if } r < 0.6 \\ e^{-6} & \text{otherwise} \end{cases} \quad \text{where} \quad r = \begin{cases} \sqrt{x^2 + (y - 0.5)^2} & \text{if } y > 0 \\ \sqrt{x^2 + (y + 0.5)^2} & \text{otherwise.} \end{cases}$$

The choice of the density function reflects the desire to pack more points near the two ends of the slit to resolve any possible singularities.

3.4.4 Point placement under constraints

It is possible to incorporate the techniques for constrained CVT discussed in the section 3.2 in order to fulfill the requirement of placing some points on the boundary as is required in many current meshless methods. Fig. 3.12 provides a test example for a square domain. Here, we construct the CVT using Algorithm 3 as usual, but whenever a new set of generators were computed (after step 3) at each iteration, the points whose Voronoi regions include part of a side of the square were projected onto the side along its normal direction and points whose Voronoi regions involve one of the four corners of the square were replaced by the corresponding corner point. Fig. 3.13 illustrates the same situation for the unit circle where the projection procedure is even simpler in the sense that the points whose Voronoi regions include part of the boundary are automatically projected onto the boundary along the radial direction. Although only uniform density functions are used in Fig. 3.12 and 3.13, the same projection procedure can be applied to any nonuniform density function. For example, in Fig. 3.14, we provide an example of a constrained CVT for a circular domain for the density function $\rho(x, y) = e^{-2.5(1+y)}$.

3.4.5 Three-dimensional domains

Algorithms 3, 7 and 8 can be easily applied to any space dimension since the procedure is not related to the dimension of the space except for the concept of distances and

the sorting process. Some numerical experiments with $n = 512$ points were performed in $\Omega = ([-1, 1] \times [-0.5, 0.5] \times [-1, 1]) \cup ([0, 1] \times [-0.5, 0.5] \times [0, 1])$. Since Ω is also non-convex, we used a rejection technique similar to that described in the section 3.4.3 for domains with relatively well-behaved boundaries. Due to the difficulty in visualizing the spherical patches in three dimensionas, we here only give the scatter plots of the 512 points. Fig. 3.15 provides the plots for a uniform density function and Fig. 3.16 for the nonuniform the density function

$$\rho(x, y, z) = \begin{cases} e^{-20r} & \text{if } r < 0.4 \\ e^{-8} & \text{otherwise,} \end{cases}$$

where $r = \sqrt{x^2 + z^2}$. The centroidal Voronoi point placement method again provides point sets of superior quality.

Table 3.2 Densities of nonzero entries of the characteristic matrices of coverings of square and circular domains with nonuniform point distributions.

Density function	No. of points	Algorithm	Point placement method	
			Monte Carlo	CVT
$\Omega = [-1, 1] \times [-1, 1]$				
$e^{-3(x^2+y^2)}$	128	7	18.58%	6.50%
		8 (with $p = 3$)	48.70%	28.36%
	256	7	7.87%	2.91%
		8 (with $p = 3$)	21.04%	12.46%
	512	7	3.97%	1.44%
		8 (with $p = 3$)	9.96%	5.58%
$e^{-1.5(2+x+y)}$	128	7	12.57%	5.27%
		8 (with $p = 3$)	32.80%	21.74%
	256	7	7.18%	2.67%
		8 (with $p = 3$)	16.01%	10.48%
	512	7	3.25%	1.36%
		8 (with $p = 3$)	7.38%	5.12%
$e^{-(1+x)^2}$	128	7	15.85%	5.26%
		8 (with $p = 3$)	39.53%	22.05%
	256	7	7.84%	2.72%
		8 (with $p = 3$)	18.17%	10.47%
	512	7	3.36%	1.35%
		8 (with $p = 3$)	7.60%	5.05%
$\Omega = \{(x_1, x_2) \mid x_1^2 + x_2^2 \leq 1\}$				
$e^{-4(x^2+y^2)}$	128	7	20.75%	5.34%
		8 (with $p = 3$)	48.06%	26.41%
	256	7	9.13%	2.94%
		8 (with $p = 3$)	23.49%	12.11%
	512	7	3.90%	1.42%
		8 (with $p = 3$)	10.01%	5.61%
$e^{-3(1-x^2-y^2)}$	128	7	12.23%	5.34%
		8 (with $p = 3$)	25.26%	18.59%
	256	7	7.22%	2.67%
		8 (with $p = 3$)	15.78%	10.12%
	512	7	3.27%	1.35%
		8 (with $p = 3$)	7.48%	5.01%

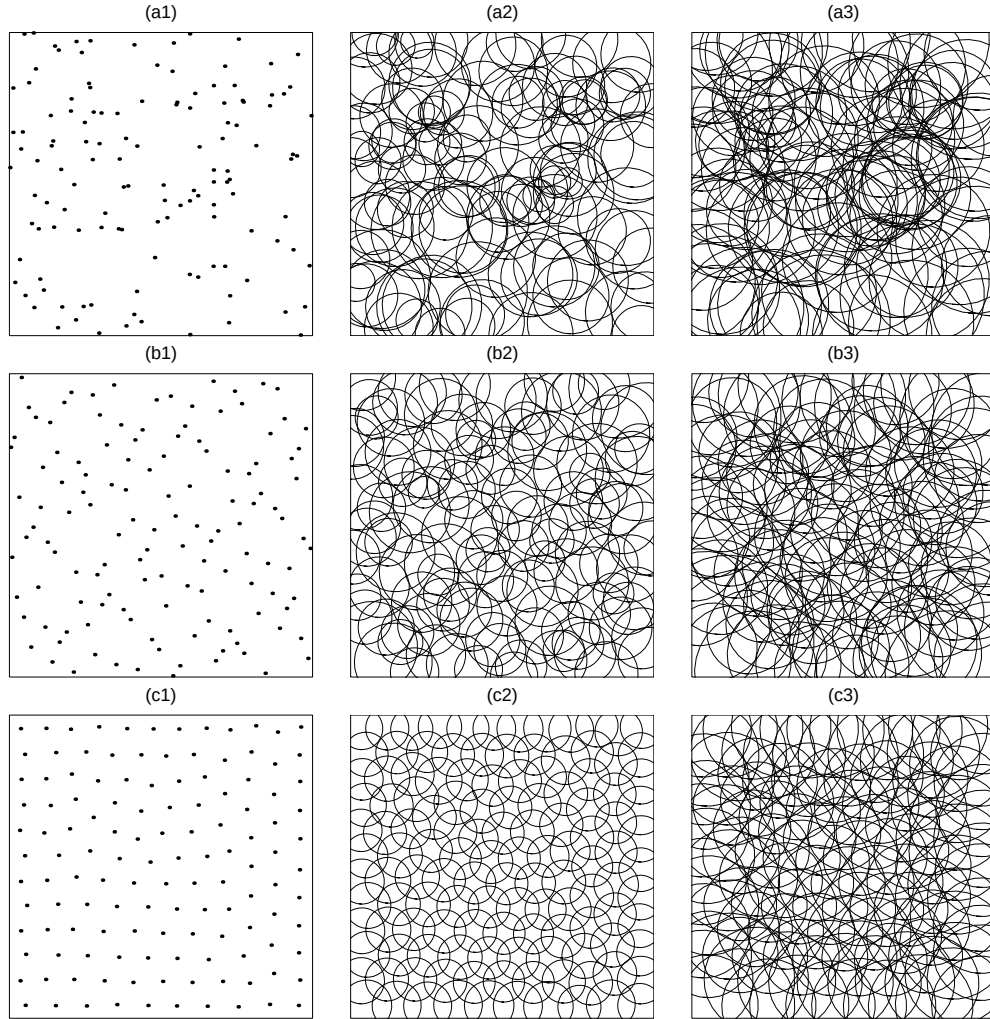


Figure 3.1 The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top), (2,3) Halton sequence (middle), and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

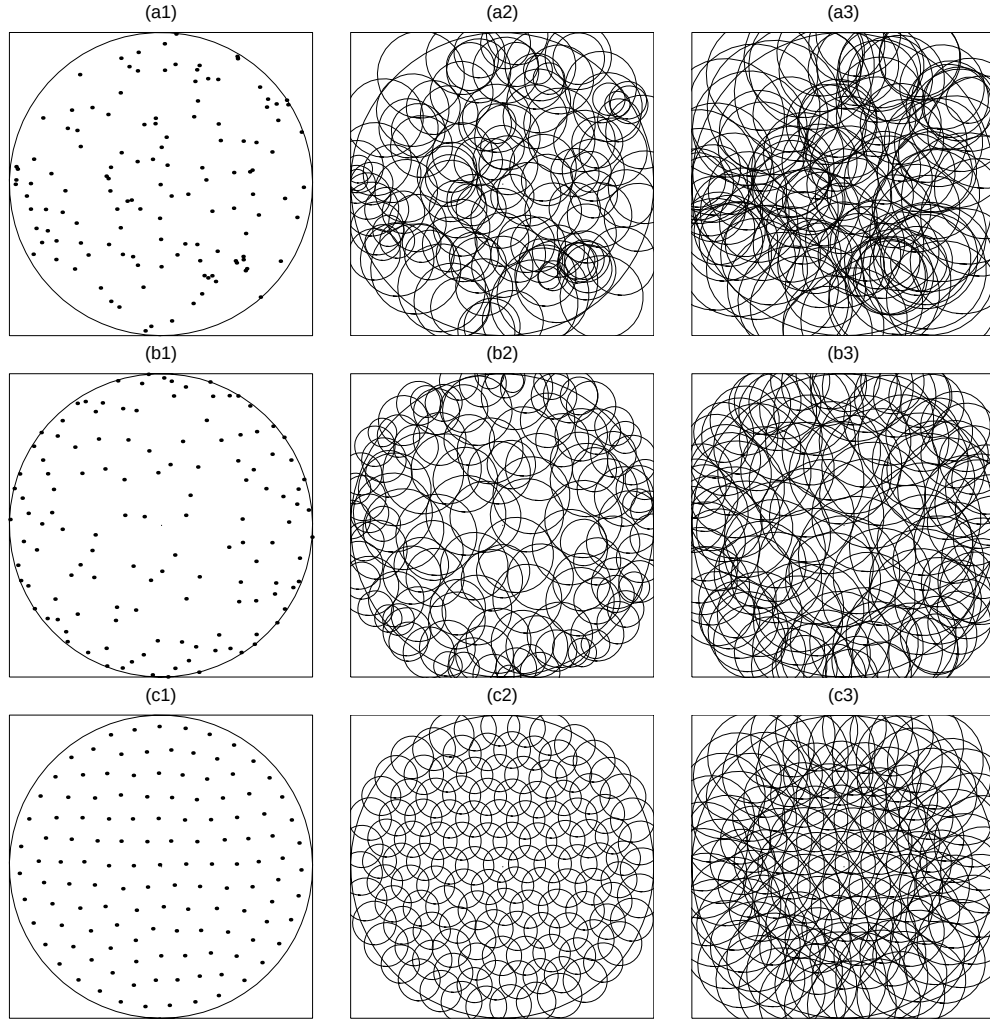


Figure 3.2 The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top), (2,3) Halton sequence (middle), and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

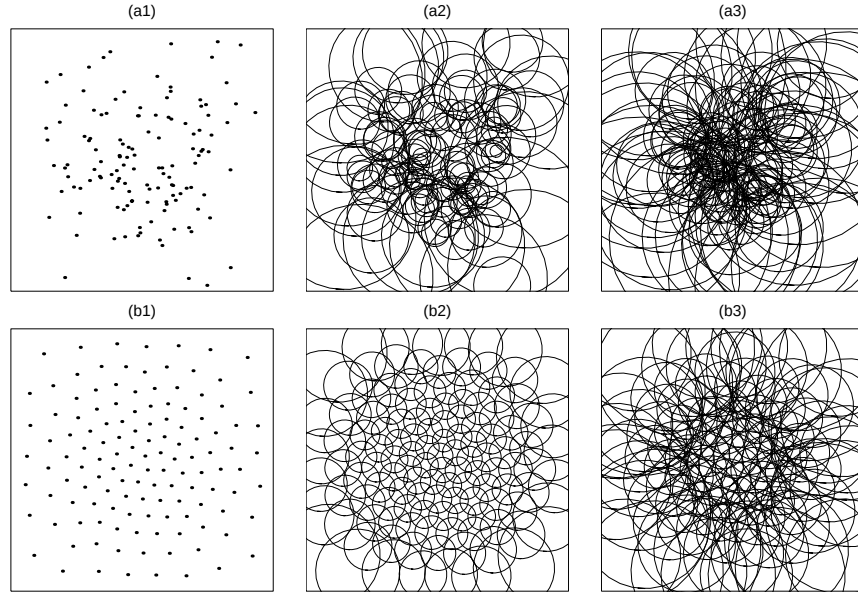


Figure 3.3 The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-3(x^2+y^2)}$.

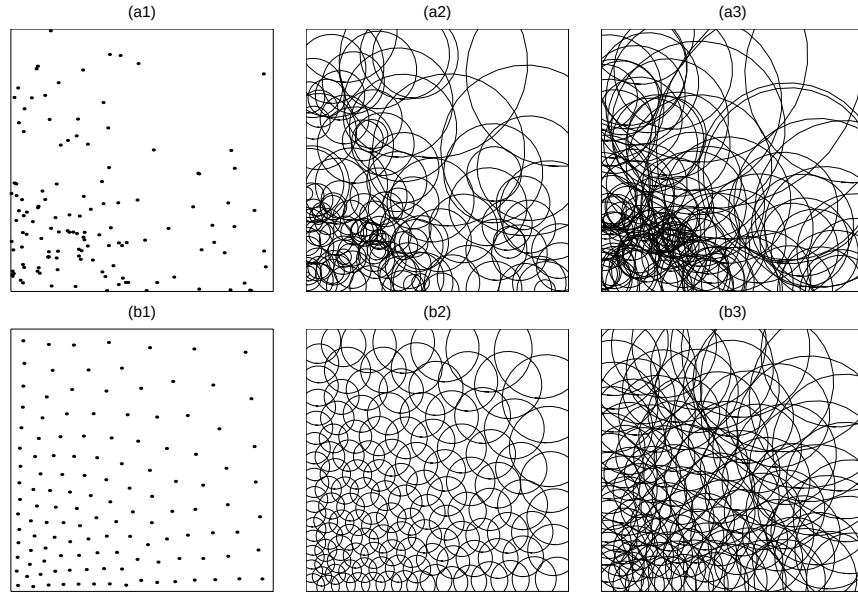


Figure 3.4 The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-1.5(2+x+y)}$.

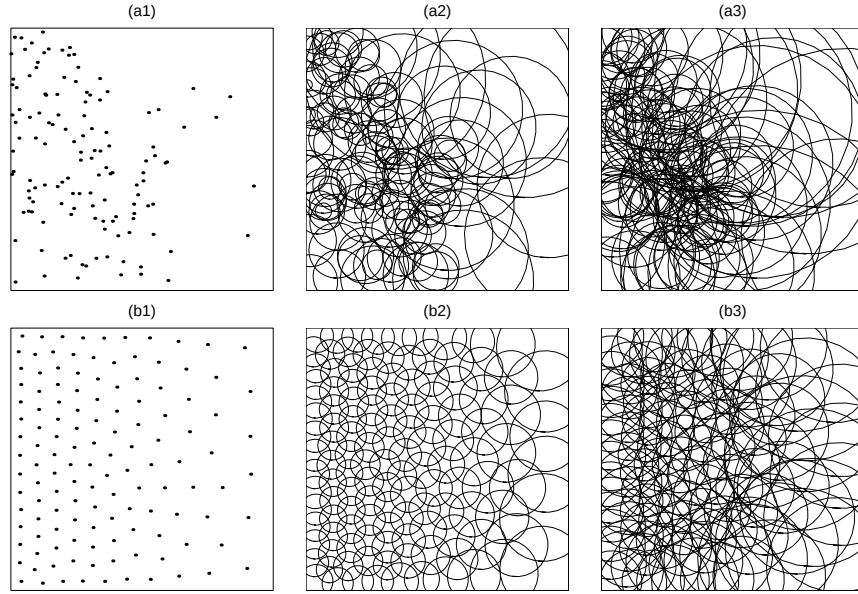


Figure 3.5 The sets of 128 points in a square (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-(1+x)^2}$.

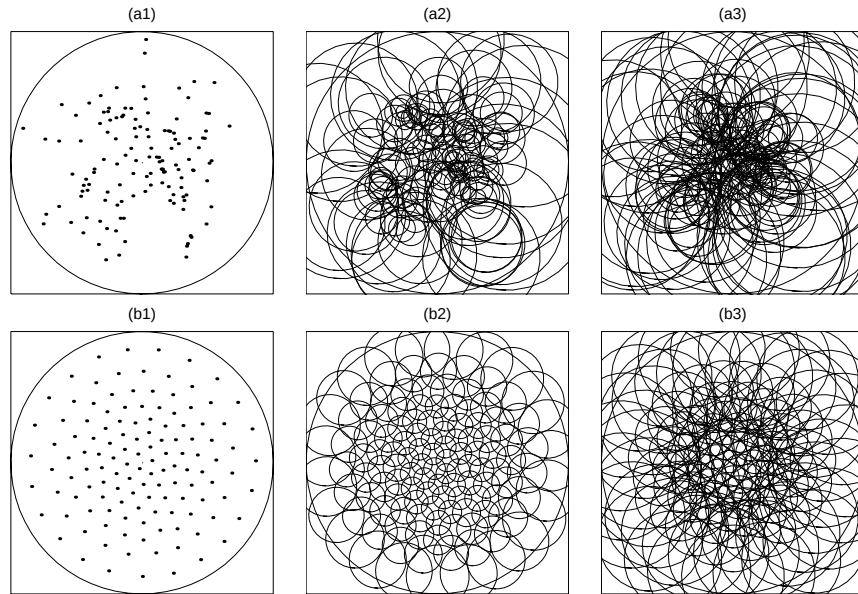


Figure 3.6 The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-4(x^2+y^2)}$.

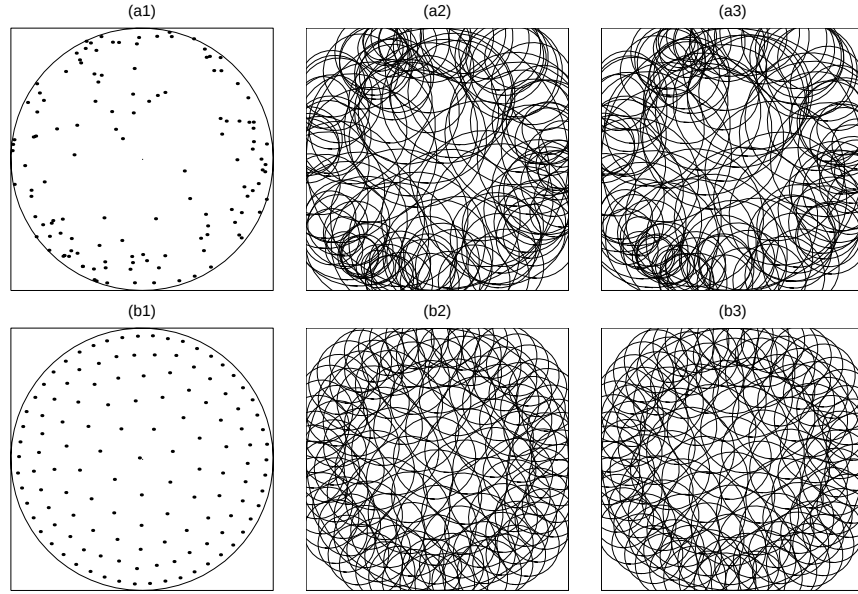


Figure 3.7 The sets of 128 points in a circle (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $e^{-3(1-x^2-y^2)}$.

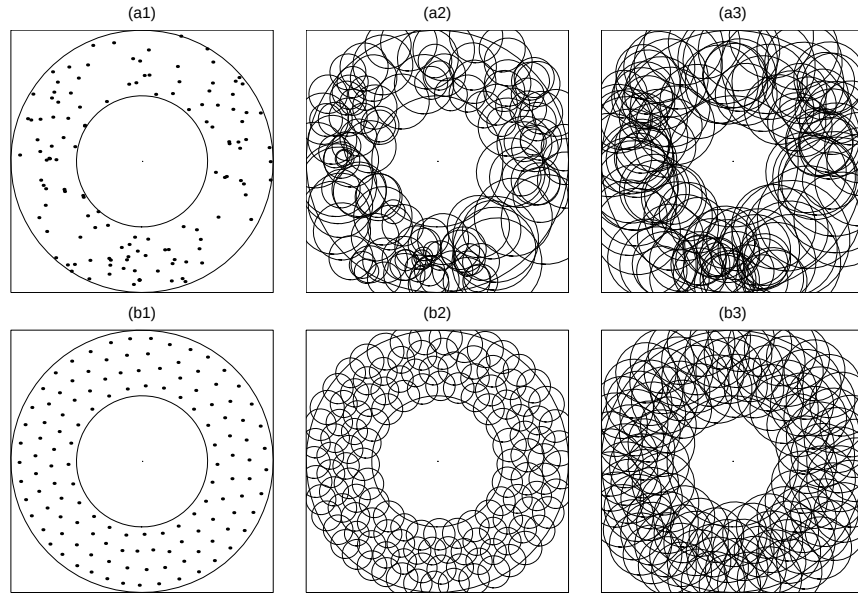


Figure 3.8 The sets of 128 points in a non-convex domain (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

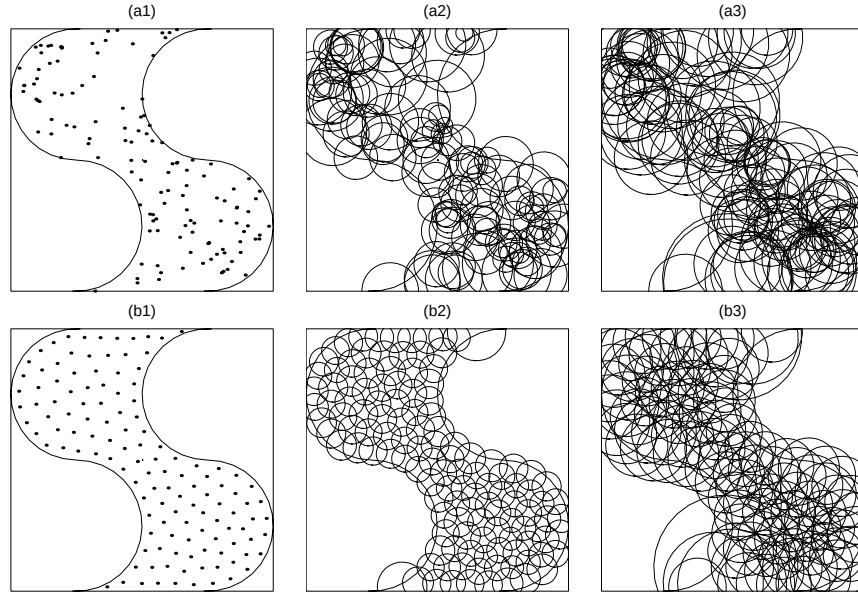


Figure 3.9 The sets of 128 points in a non-convex domain (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

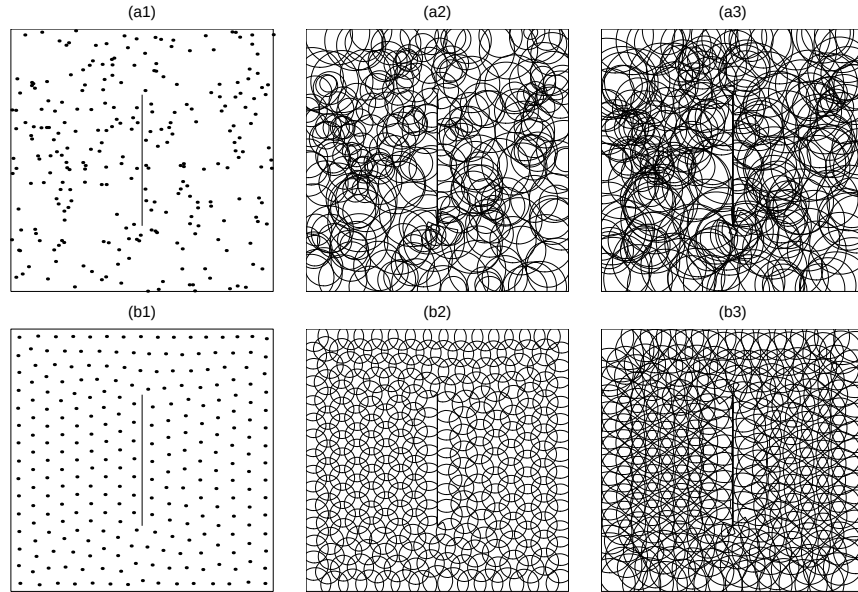


Figure 3.10 The sets of 256 points in a domain with a slit (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

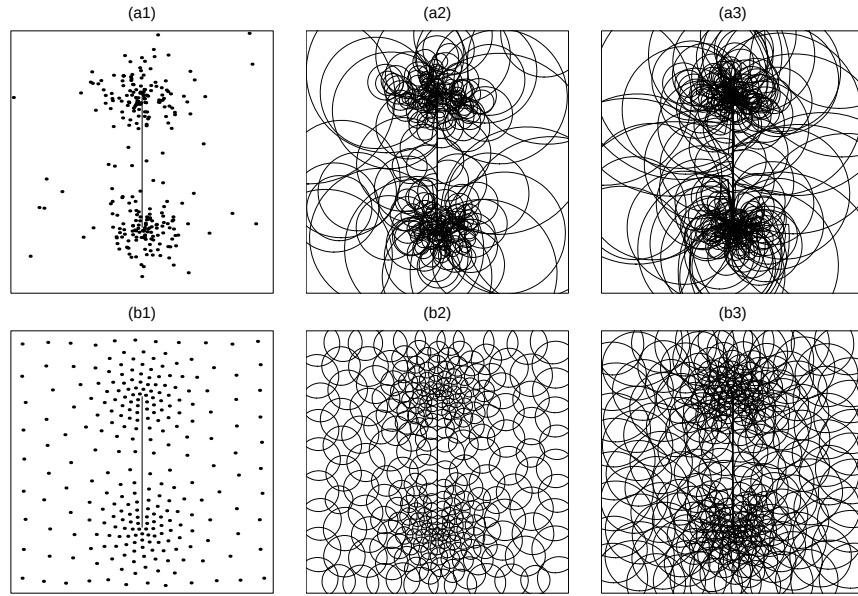


Figure 3.11 The sets of 256 points in a domain with a slit (left) and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a nonuniform density function.

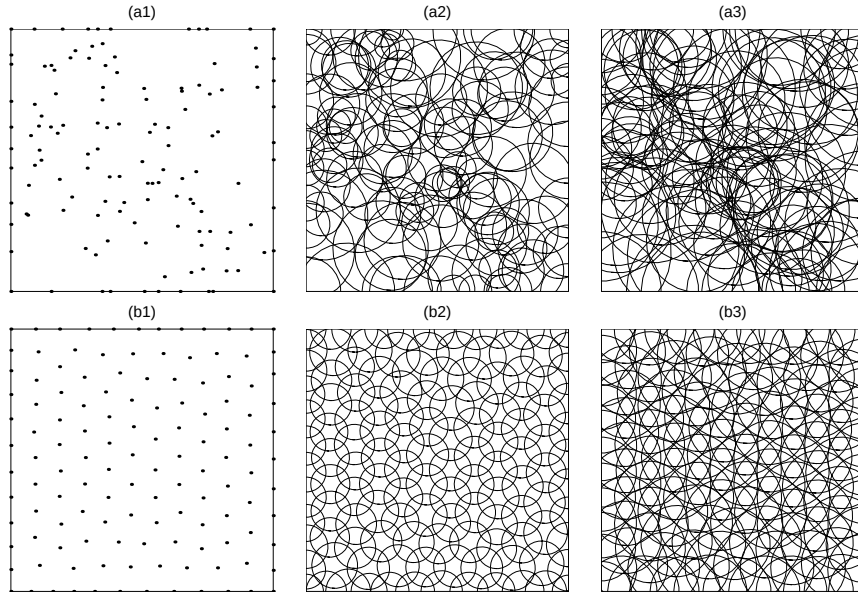


Figure 3.12 The sets of 128 points in a square (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

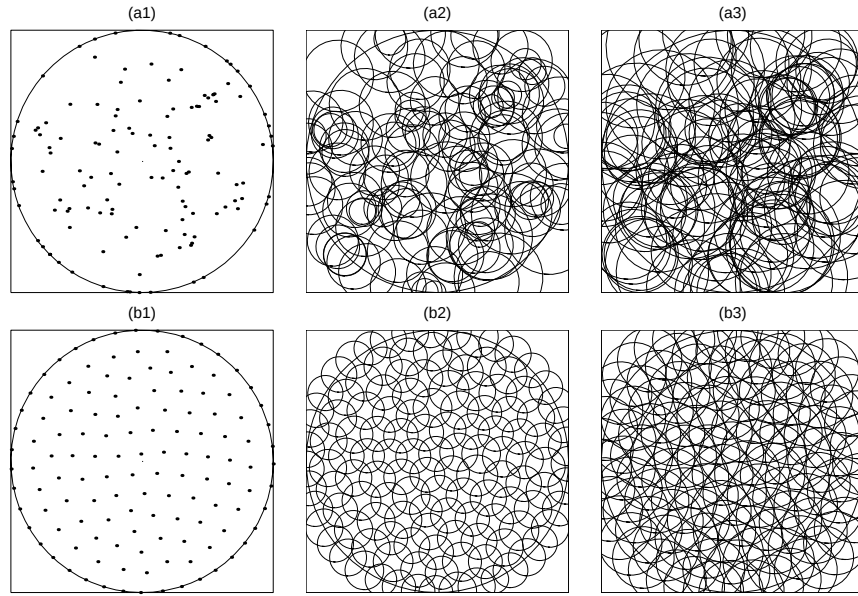


Figure 3.13 The sets of 128 points in a circle (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for a uniform density function.

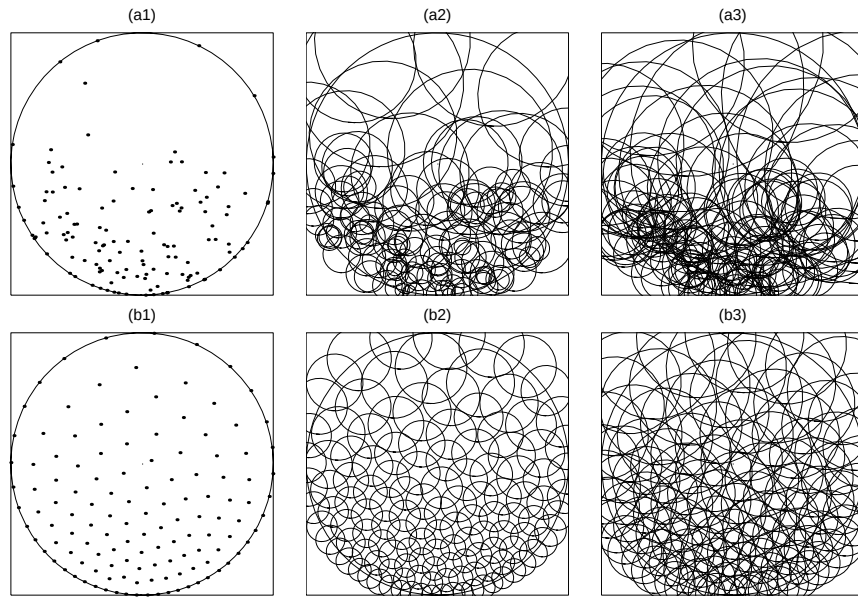


Figure 3.14 The sets of 128 points in a circle (left) under constraint and the associated spherical patches determined by Algorithms 7 (middle) and 8 (with $p = 3$) (right) for the Monte Carlo (top) and centroidal Voronoi tessellation (bottom) point selection methods for the density function $\rho(x, y) = e^{-2.5(1+y)}$.

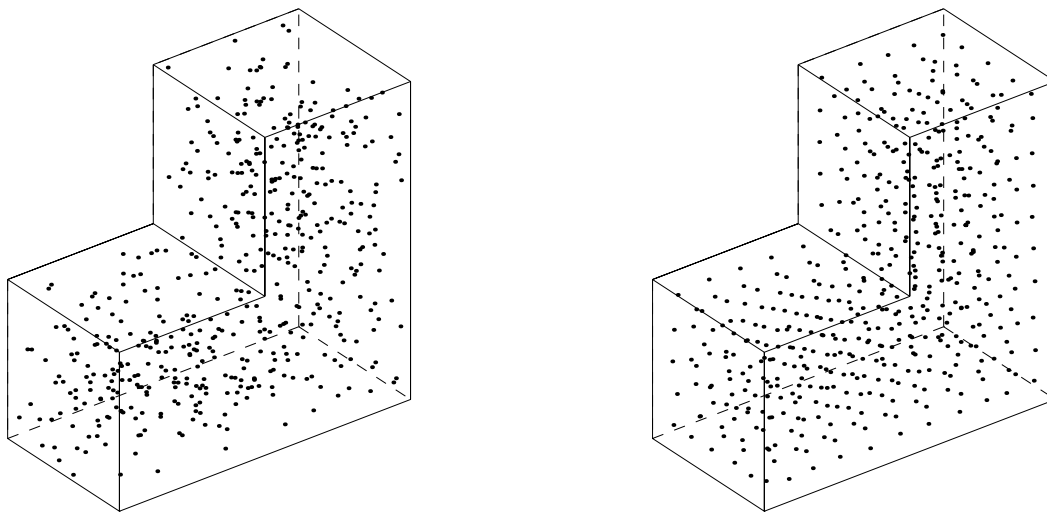


Figure 3.15 The sets of 512 points in a three-dimensional non-convex domain for the Monte Carlo (left) and centroidal Voronoi tessellation (right) point selection methods for a uniform density function.

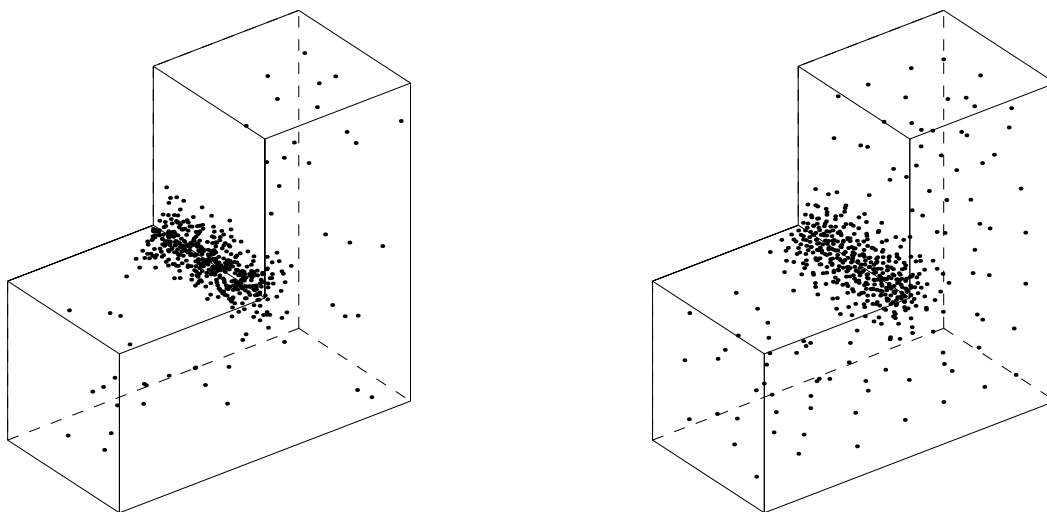


Figure 3.16 The sets of 512 points in a three-dimensional non-convex domain for the Monte Carlo (left) and centroidal Voronoi tessellation (right) point selection methods for a non uniform density function.

CHAPTER 4. CONSTRAINED CENTROIDAL VORONOI TESSELLATIONS ON GENERAL SURFACES

The plan of this chapter is as follows. We develop and analyze, in §4.2, the notion of *constrained* CVT's. Then, in §4.3, we discuss some deterministic and probabilistic algorithms for the construction of constrained CVT's. In §4.4, some computational examples are provided that illustrate the high quality of constrained CVT point sets. Finally, in §4.5, we discuss the use of constrained CVT point sets for polynomial interpolation and numerical integration on a sphere.

4.1 Introduction

In (13; 14; 16) and Chapter 2, a methodology for point placement in regions, i.e., volumes, in $\mathbb{R}^N$ has been developed. The methodology is based on the notion of *centroidal Voronoi tessellations* (CVT's) which is explained in §1.1. The ensuing methodology produces high-quality point distributions which may themselves be of interest or may be used as a basis for triangulations or Voronoi tessellations of the region. Among the advantages of the CVT methodology is that points may easily be distributed according to a prescribed nonuniform density function and the algorithms which make up the methodology are amenable to parallelization. In addition, CVT's enjoy an optimization characterization so that they themselves turn out to be useful in many applications such as image and data analysis, vector quantization, resource optimization, optimal placement of sensors and actuators for control, cell biology, territorial behavior of animals, nu-

merical partial differential equations, meshless computing, etc.; see, e.g., (13; 14; 17; 52) and Chapter 3.

The basic definition of the CVT in Chapter 1 can be generalized to very broad settings that range from abstract spaces to discrete point sets (13). The purpose of this paper is to study the CVT methodology that was developed in (13; 14; 16) and Chapter 2 in the case where the point sets are constrained to lie on surfaces in $\mathbb{R}^N$. There are many instances in which point distributions lying on surfaces or triangulations or more general subdivisions of surfaces are needed. Just to mention a few important examples, there are geophysical calculations on the surface of the earth, i.e., on nearly a sphere, collocation or nodal points for boundary finite element methods, and the geometric representation of surfaces by panels or other simple objects. Also, many mesh generation methods in three dimensions require that first a surface grid be developed.

4.2 Constrained centroidal Voronoi tessellations

Now consider a compact and continuous surface $\mathbf{S} \subset \mathbb{R}^N$ defined by

$$\mathbf{S} = \{\mathbf{x} \in \mathbb{R}^N \mid g_0(\mathbf{x}) = 0 \text{ and } g_j(\mathbf{x}) \leq 0 \text{ for } j = 1, \dots, m\} \quad (4.1)$$

for some continuous functions g_0 and $\{g_j\}_{j=1}^m$. Similar to (1.1), given a set of points $\{\mathbf{z}_i\}_{i=1}^k \in \mathbf{S}$, one may define their corresponding Voronoi regions on $\mathbf{S}$ by

$$V_i = \{\mathbf{x} \in \mathbf{S} \mid |\mathbf{x} - \mathbf{z}_i| < |\mathbf{x} - \mathbf{z}_j| \text{ for } j = 1, \dots, k, j \neq i\} \quad i = 1, \dots, k. \quad (4.2)$$

For a density function $\rho(\mathbf{x})$ defined on the surface $\mathbf{S}$ and positive almost everywhere, one may encounter a problem with the definition (4.2) when one defines centroidal Voronoi tessellations $\{(\mathbf{z}_i, V_i)\}_{i=1}^k$ of $\mathbf{S}$: the mass centroids $\{\mathbf{z}_i^*\}_{i=1}^k$ of $\{V_i\}_{i=1}^k$ as defined by (1.2) do not, in general, belong to $\mathbf{S}$. For example, the mass centroid of any region on the surface of a sphere is always located in the interior of the sphere. Therefore, we must first use a generalized definition of a mass centroid on surfaces.

For each Voronoi region V_i , we call $\mathbf{z}_i^c$ the *constrained mass centroid* of V_i on $\mathbf{S}$ if $\mathbf{z}_i^c$ is a solution of the following problem:

$$\min_{\mathbf{z} \in \mathbf{S}} F_i(\mathbf{z}), \quad \text{where} \quad F_i(\mathbf{z}) = \int_{V_i} \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}|^2 d\mathbf{x}. \quad (4.3)$$

The integral over $\{V_i\}$ is understood as standard surface integration on $\mathbf{S}$. Clearly, for each $i = 1, \dots, k$, $F_i(\cdot)$ is convex. Assume that $\mathbf{z}_1, \mathbf{z}_2 \in \mathbf{S}$; then we have

$$F_i(\mathbf{z}_1) - F_i(\mathbf{z}_2) = \int_{V_i} \rho(\mathbf{x}) (|\mathbf{x} - \mathbf{z}_1|^2 - |\mathbf{x} - \mathbf{z}_2|^2) d\mathbf{x}.$$

Since $\mathbf{S}$ is compact and $\rho(\cdot)$ is continuous almost everywhere, there exists a constant C such that

$$|F_i(\mathbf{z}_1) - F_i(\mathbf{z}_2)| \leq C |\mathbf{z}_1 - \mathbf{z}_2|.$$

Thus, we know that F_i is continuous and compact, and consequently we obtain the existence of solutions of (4.3); however, the solution may not be unique.

We call the tessellation defined by (4.2) a *constrained centroidal Voronoi tessellation* (CCVT) if and only if the points $\{\mathbf{z}_i\}_{i=1}^k$ which serve as the generators associated with the Voronoi regions $\{V_i\}_{i=1}^k$ are the constrained mass centroids of those regions. Note that although the definition of CCVT conforms with that of CVT for general spaces, it is important to emphasize the main features of the former, namely, the generators are constrained to the surfaces while the distances are still the standard Euclidean distances, not the more general geodesic distances.

For practical applications, it is natural to ask: for a given V_i , how does one find its constrained mass centroid on $\mathbf{S}$, i.e., how does one solve the constrained optimization problem (4.3)? We rewrite (4.3) in the more standard form

$$\begin{cases} \min_{\mathbf{z} \in \mathbb{R}^N} F_i(\mathbf{z}) \\ g_0(\mathbf{z}) = 0 \\ g_j(\mathbf{z}) \leq 0 \quad \text{for } j = 1, \dots, m. \end{cases} \quad (4.4)$$

If $\mathbf{z}_i^c$ is a solution of (4.4), then the Lagrange multiplier rule states that $\mathbf{z}_i^c$ is a stationary point of

$$L_i(\mathbf{z}) = F(\mathbf{z}) - \sum_{j=0}^m \lambda_j g_j(\mathbf{z}) = \int_{V_i} \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}|^2 d\mathbf{x} - \sum_{j=0}^m \lambda_j g_j(\mathbf{z}), \quad (4.5)$$

where $\lambda_j \in \mathbb{R}$. Under smoothness assumptions on the Lagrange functional, the necessary condition can then be written as

$$\begin{cases} L_{i,z}(\mathbf{z}_i^c) = 0 \\ g_0(\mathbf{z}_i^c) = 0 \\ \lambda_j \geq 0, \quad \lambda_j g_j(\mathbf{z}_i^c) = 0 \quad \text{for } j = 1, \dots, m. \end{cases} \quad (4.6)$$

Suppose $\mathbf{z}_i^c$ is not located on the boundary of $\mathbf{S}$, i.e.,

$$g_j(\mathbf{z}_i^c) < 0 \quad \text{for } j = 1, \dots, m.$$

Then, we know that λ_j must be 0 for all $j = 1, \dots, m$, and so (4.6) simplifies to

$$2 \int_{V_i} \rho(\mathbf{x})(\mathbf{x} - \mathbf{z}_i^c) d\mathbf{x} - \lambda_0 \nabla_z(g_0(\mathbf{z}_i^c)) = 0. \quad (4.7)$$

Furthermore, if $\bar{\lambda} = \frac{\lambda_0}{2}$, we have

$$\int_{V_i} \mathbf{x} \rho(\mathbf{x}) d\mathbf{x} = \mathbf{z}_i^c \int_{V_i} \rho(\mathbf{x}) d\mathbf{x} + \bar{\lambda} \nabla_z(g_0(\mathbf{z}_i^c))$$

or

$$\frac{\int_{V_i} \mathbf{x} \rho(\mathbf{x}) d\mathbf{x}}{\int_{V_i} \rho(\mathbf{x}) d\mathbf{x}} = \mathbf{z}_i^c + \frac{\bar{\lambda}}{\int_{V_i} \rho(\mathbf{x}) d\mathbf{x}} \nabla_z(g_0(\mathbf{z}_i^c)).$$

Using (1.2), we then obtain

$$\mathbf{z}_i^* - \mathbf{z}_i^c = \frac{\bar{\lambda}}{\int_{V_i} \rho(\mathbf{x}) d\mathbf{x}} \nabla_z(g_0(\mathbf{z}_i^c)) \quad (4.8)$$

and $g(\mathbf{z}_i^c) = 0$. Thus, we have obtained the following result.

Theorem 1. *For each $i = 1, \dots, k$, the constrained mass centroids of V_i exists. Furthermore, if $\mathbf{z}_i^c \in \mathbf{S} - \partial\mathbf{S}$ is a constrained centroid of V_i , then $\mathbf{z}_i^* - \mathbf{z}_i^c$ is a vector normal to the surface $\mathbf{S}$ at $\mathbf{z}_i^c$, where $\mathbf{z}_i^*$ is defined by (1.2), i.e., $\mathbf{z}_i^c$ is the projection of $\mathbf{z}_i^*$ onto $\mathbf{S}$ along the normal direction at $\mathbf{z}_i^c$.*

From Theorem 1, it is easy to see that the constrained centroidal Voronoi tessellation of a flat surface, i.e., of a surface $\mathbf{S}$ having zero curvature almost everywhere, is reduced to the centroidal Voronoi tessellation in the Euclidean space.

There are many available theoretical results for centroidal Voronoi tessellations; see (13). Similar analyses can be applied to constrained centroidal Voronoi tessellations. For example, we have the following results.

Proposition 5. *Given a compact surface $\mathbf{S} \subset \mathbb{R}^N$ defined by (4.1), a positive integer k , and a positive and measurable density function $\rho(\cdot)$ defined on $\mathbf{S}$. Let $\{\mathbf{z}_i\}_{i=1}^k$ denote any set of k points belonging to $\mathbf{S}$ and let $\{V_i\}_{i=1}^k$ denote any tessellation of $\mathbf{S}$ into k regions. Define the energy functional or the distortion value for $\{(\mathbf{z}_i, V_i)\}_{i=1}^k$ by*

$$\mathcal{F}(\{(\mathbf{z}_i, V_i)\}_{i=1}^k) = \sum_{i=1}^k \int_{\mathbf{x} \in V_i} \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_i|^2 d\mathbf{x}. \quad (4.9)$$

A necessary condition for $\mathcal{F}$ to be minimized is that the V_i 's are the Voronoi regions corresponding to the $\mathbf{z}_i$'s and, simultaneously, the $\mathbf{z}_i$'s are the constrained centroids of the corresponding V_i 's, i.e., $\{(\mathbf{z}_i, V_i)\}_{i=1}^k$ is a constrained centroidal Voronoi tessellation of $\mathbf{S}$.

Proof. First examine the first variation of $\mathcal{F}$ with respect to a single point, say $\mathbf{z}_j$. We then have that

$$\mathcal{F}(\mathbf{z}_j + \mathbf{v}) - \mathcal{F}(\mathbf{z}_j) = \int_{\mathbf{x} \in V_j} \rho(\mathbf{x}) (|\mathbf{x} - \mathbf{z}_j - \mathbf{v}|^2 - |\mathbf{x} - \mathbf{z}_j|^2) d\mathbf{x}, \quad (4.10)$$

where we have not listed the fixed variables in the argument of $\mathcal{F}$ and where $\mathbf{v}$ is arbitrary such that $\mathbf{z}_j + \mathbf{v} \in \mathbf{S}$. From (4.10), we see that minimizers of $\mathcal{F}$ with respect to $\mathbf{z}_j$ have

the same characterization as minimizers of F_j given in (4.3). Thus, the minimizing points $\{\mathbf{z}_j\}_{j=1}^k$ are the constrained centroids of the corresponding regions $\{V_j\}_{j=1}^k$ on $\mathbf{S}$.

Next, for given points $\{\mathbf{z}_i\}_{i=1}^k$ and a tessellation $\{V_i\}_{i=1}^k$ other than the Voronoi tessellation $\{\tilde{V}_i\}_{i=1}^k$ corresponding to $\{\mathbf{z}_i\}_{i=1}^k$, we may compare the value of $\mathcal{F}(\{(\mathbf{z}_i, V_i)\}_{i=1}^k)$ given by (4.9) with

$$\mathcal{F}(\{(\mathbf{z}_i, \tilde{V}_i)\}_{i=1}^k) = \sum_{i=1}^k \int_{\mathbf{x} \in \tilde{V}_i} \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_i|^2 d\mathbf{x}. \quad (4.11)$$

For a point $\mathbf{x} \in \mathbf{S}$ that belongs to the Voronoi region $\tilde{V}_j$ we have that

$$\rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_j|^2 \leq \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_i|^2 \quad \text{for } i = 1, \dots, k \quad (4.12)$$

since possibly $\mathbf{x}$ does not belong to the Voronoi region corresponding to $\mathbf{z}_i$. However, since $\{V_i\}_{i=1}^k$ is *not* a Voronoi tessellation of $\mathbf{S}$ corresponding to $\{\mathbf{z}_i\}_{i=1}^k$, (4.12) must hold with strict inequality over some measure non-zero set of $\mathbf{S}$. Thus,

$$\mathcal{F}(\{(\mathbf{z}_i, \tilde{V}_i)\}_{i=1}^k) < \mathcal{F}(\{(\mathbf{z}_i, V_i)\}_{i=1}^k)$$

so that $\mathcal{F}$ is minimized when the $\{V_j\}_{j=1}^k$ are chosen to be the Voronoi regions associated with the points $\{\mathbf{z}_j\}_{j=1}^k$. $\square$

Define the functional

$$\mathcal{K}(\{\mathbf{z}_j\}_{j=1}^k) = \sum_{i=1}^k \int_{\mathbf{x} \in \tilde{V}_i} \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_i|^2 d\mathbf{x}, \quad (4.13)$$

where $\mathbf{z}_i$'s belong to $\mathbf{S}$ and $\tilde{V}_i$'s are the corresponding Voronoi regions on $\mathbf{S}$. We call $\mathcal{K}$ the *energy* of $\{\mathbf{z}_j\}_{j=1}^k$ on the surface $\mathbf{S}$. From the above proof, we have the following result.

Proposition 6. *Given a surface $\mathbf{S} \in \mathbb{R}^N$, a positive integer k , and a positive density function $\rho(\cdot)$ defined on $\mathbf{S}$. Then, $\mathcal{F}$ and $\mathcal{K}$ have the same minimizers.*

Let $K = \{\mathbf{Z} = (\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k), \mathbf{z}_j \in \mathbf{S}\}$. Let $A_i = |\tilde{V}_i|$ denote the area of $\tilde{V}_i$ and let $\mathbf{A} = (A_1, A_2, \dots, A_k)$. It is obvious that $\mathbf{A}$ is continuous. Furthermore, the following result holds.

Theorem 2. *Let $\rho(\mathbf{x})$ be continuous on Ω . Then, $\mathcal{K}$ is continuous and possesses a global minimum.*

Proof. Let $\mathbf{Z}, \mathbf{Z}' \in K$. Then,

$$|\mathcal{K}(\mathbf{Z}) - \mathcal{K}(\mathbf{Z}')| = \left| \sum_{i=1}^k \left\{ \left(\int_{\mathbf{x} \in \tilde{V}_i} - \int_{\mathbf{x} \in \tilde{V}'_i} \right) \rho(\mathbf{x}) |\mathbf{x} - \mathbf{z}_i|^2 d\mathbf{x} + \int_{\mathbf{x} \in \tilde{V}'_i} \rho(\mathbf{x}) (|\mathbf{x} - \mathbf{z}_i|^2 - |\mathbf{x} - \mathbf{z}'_i|^2) d\mathbf{x} \right\} \right|. \quad (4.14)$$

Since $\mathbf{S}$ is compact and $\rho(\cdot)$ is continuous, there exists a constant C such that

$$|\mathcal{K}(\mathbf{Z}) - \mathcal{K}(\mathbf{Z}')| \leq C \sum_{i=1}^k (|A_i - A'_i| + |\mathbf{z}_i - \mathbf{z}'_i|).$$

Then, the continuity of $\mathcal{K}$ follows from the continuity of $\mathbf{A}$ and the existence of the global minimizer follows from the compactness of $\mathcal{K}$. $\square$

4.3 Algorithms for determining CCVT's

An arbitrary choice of generating points $\{\mathbf{z}_i\}_{i=1}^k$ on a surface are not, in general, the constrained mass centroids of the corresponding Voronoi regions on that surface. As a result, one is left with the following construction problem: *given a surface $\mathbf{S} \subset \mathbb{R}^N$, a positive integer k , and a density function $\rho(\mathbf{x})$ defined for $\mathbf{x} \in \mathbf{S}$, determine a k -point constrained centroidal Voronoi tessellation of $\mathbf{S}$ with respect to the given density function.*

Similar to centroidal Voronoi tessellations of a region, constrained centroidal Voronoi tessellations of a given surface also are not uniquely defined in general. For example, given a constrained centroidal Voronoi tessellation $\{(\mathbf{z}_i, V_i)\}_{i=1}^k$ on the surface of a sphere with respect to a constant density function, then $\{(\mathbf{z}'_i, V'_i)\}_{i=1}^k$, a rotation of $\{(\mathbf{z}_i, V_i)\}_{i=1}^k$, is still a constrained centroidal Voronoi tessellation.

4.3.1 Some probabilistic and deterministic methods for determining CCVT's

There are several algorithms known for constructing centroidal Voronoi tessellations of a given set and discussed in Chapter 2; see (13; 45; 47). Here, we consider three methods. The first is *MacQueen's method* (13; 47), a very elegant probabilistic algorithm which divides sampling points into k sets or clusters by taking means of clusters. The second method is a deterministic algorithm that is known in some circles as *Lloyd's method* (13; 45) and which is the obvious iteration between computing Voronoi diagrams and mass centroids. The third method is a probabilistic method suggested in Chapter 2 which may be viewed as a generalization of the known MacQueen and Lloyd methods. Using Theorem 1, we modify the three methods for the construction of constrained centroidal Voronoi tessellations.

The modified version of MacQueen's method is given as follows. Throughout, Monte Carlo sampling simply means random sampling according to the given density function.

Algorithm 9. (MacQueen's method for CCVT's) *Given a surface $\mathbf{S}$, a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \mathbf{S}$, and a positive integer k ;*

0. *choose an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$ on $\mathbf{S}$, e.g., by using a Monte Carlo method;
set $j_i = 1$ for $i = 1, \dots, k$;*
1. *determine a point $\mathbf{y}$ in $\mathbf{S}$ at random, e.g., by a Monte Carlo method, according to the probability density function $\rho(\mathbf{x})$;*
2. *find a $\mathbf{z}_{i^*}$ among $\{\mathbf{z}_i\}_{i=1}^k$ that is the closest to $\mathbf{y}$;*
3. *set*

$$\mathbf{z}'_{i^*} \leftarrow \frac{j_{i^*}\mathbf{z}_{i^*} + \mathbf{y}}{j_{i^*} + 1}, \quad j_{i^*} \leftarrow j_{i^*} + 1, \quad \text{and} \quad \mathbf{z}_{i^*} = \mathbf{proj}(\mathbf{z}'_{i^*});$$

the new $\mathbf{z}_{i^}$, along with the unchanged $\{\mathbf{z}_j\}, j \neq i^*$, form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;*

4. *if the new points meet some convergence criterion, terminate; otherwise, return to step 1.*

In Algorithm 9, $\mathbf{z}_{i*} = \mathbf{proj}(\mathbf{z}'_{i*})$ denotes the following process. If there exists a set $\{\mathbf{z}_{i*}^j\}_{j=1}^r \in \mathbf{S} - \partial\mathbf{S}$ which are projection points of $\mathbf{z}'_{i*}$ onto $\mathbf{S}$ along the normal direction to $\mathbf{S}$ at $\mathbf{z}_{i*}^j$, choose a $\mathbf{z}_{i*}^c$ satisfying $|\mathbf{z}_{i*}^c - \mathbf{z}'_{i*}| = \min_{j=1, \dots, r} |\mathbf{z}_{i*}^j - \mathbf{z}'_{i*}|$ and then replace $\mathbf{z}_{i*}$ by $\mathbf{z}_{i*}^c$; otherwise, keep $\mathbf{z}_{i*}$ unchanged.

There are two key issues in Algorithm 9 that need to be addressed. The first is how to sample points on a given surface by a Monte Carlo method, i.e., how to do random sampling on a surface according to a given density function. The second concerns the implementation of the projection process. These issues are discussed in Section 4.4.

The almost sure convergence of the energy for the random MacQueen method has been proved in the CVT case; note that in some cases this method fails to converge to a centroidal Voronoi tessellation; see (26; 47). It also has been observed that MacQueen's method converges very slowly and that, in practical computations, the energy of the final set of points is closely correlated to the energy of the corresponding initial set of points; see Chapter 2.

The following algorithm is the modification to the CCVT case of Lloyd's method (13; 45).

Algorithm 10. (Lloyd's Method for CCVT's) *Given a surface $\mathbf{S}$, a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \mathbf{S}$, and a positive integer k ,*

0. *select an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$ on $\mathbf{S}$, e.g., by using a Monte Carlo method;*
1. *construct the Voronoi sets $\{V_i\}_{i=1}^k$ of $\mathbf{S}$ associated with $\{\mathbf{z}_i\}_{i=1}^k$;*
2. *determine the constrained mass centroids of the Voronoi sets $\{V_i\}_{i=1}^k$; these constrained centroids form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;*

3. *if the new points meet some convergence criterion, terminate; otherwise, return to step 1.*

In Algorithm 10, the determination of constrained mass centroids can be done in two steps: first find the mass centroids and then project them onto the surface $\mathbf{S}$ using the projection process **proj**. Note that Algorithm 10 requires the explicit construction of Voronoi tessellations of $\mathbf{S}$ and of the constrained mass centroids of the Voronoi regions.

In the CVT setting, Lloyd's method converges much faster than does MacQueen's method. However, in the CCVT case, one is faced with the difficult task of constructing Voronoi tessellations corresponding to a given set of points on a surface. For a sphere, the STRIPACK package for constructing Voronoi diagrams (59) is available, but similar software is not currently available for general surfaces. This difficulty represents a serious obstacle to the use of the deterministic Lloyd method for determining constrained centroidal Voronoi tessellations of general surfaces. A variant scheme is to approximate the Voronoi sets using background grids, but the local refinement of background grids for complex density functions is still a substantial problem. Despite this difficulty, Lloyd's method retains considerable theoretical interest.

We next present a modified version, applicable to the CCVT case, of a probabilistic method given in Chapter 2 for CVT's. This method can be viewed as both a probabilistic version of Lloyd's method and as a generalization of the random MacQueen method.

Algorithm 11. *Given a surface $\mathbf{S}$, a density function $\rho(\mathbf{x})$ defined for all $\mathbf{x} \in \mathbf{S}$, and a positive integer k ,*

0. *choose a positive integer q and non-negative constants $\{\alpha_i, \beta_i\}_{i=1}^2$ such that $\alpha_1 + \alpha_2 = 1$; choose an initial set of k points $\{\mathbf{z}_i\}_{i=1}^k$ on $\mathbf{S}$, e.g., by using a Monte Carlo method; set $j_i = 1$ for $i = 1, \dots, k$;*

1. choose q points $\{\mathbf{y}_r\}_{r=1}^q$ in Ω at random, e.g., by a Monte Carlo method, according to the probability density function $\rho(\mathbf{x})$;
2. for $r = 1, \dots, q$, determine a $\mathbf{z}_{i_r}^*$ among $\{\mathbf{z}_i\}_{i=1}^k$ that is closest to $\mathbf{y}_r$;
3. for $i = 1, \dots, k$, gather together in the set W_i all sampling points $\mathbf{y}_r$ closest to $\mathbf{z}_i$ (i.e., in the Voronoi region of $\mathbf{z}_i$); if the set W_i is empty, do nothing; otherwise, compute the average $\mathbf{y}_i^*$ of the set W_i and set
$$\mathbf{z}_i^{*'} \leftarrow \frac{(\alpha_1 j_i + \beta_1) \mathbf{z}_i + (\alpha_2 j_i + \beta_2) \mathbf{y}_i^*}{j_i + 1}, \quad j_i \leftarrow j_i + 1, \quad \text{and} \quad \mathbf{z}_i^* = \mathbf{proj}(\mathbf{z}_i^{*'});$$
the new set of $\{\mathbf{z}_i^*\}$, along with the unchanged $\{\mathbf{z}_j\}, j \neq i$, form the new set of points $\{\mathbf{z}_i\}_{i=1}^k$;
4. if the new points meet some convergence criterion, terminate; otherwise, return to step 1.

Algorithm 11, like MacQueen's method and unlike Lloyd's method, does not require the calculation of Voronoi diagrams on a surface. Note that since two of the α_i 's and β_i 's are free, Algorithm 11 is actually a two-parameter family of methods. Furthermore, α_1 and β_1 may actually be chosen to be negative, yielding an under-relaxation method. However, for surfaces of closed bodies, e.g., a sphere, one should be careful when using negative values for α_1 and β_1 because then it is possible for the average point $\mathbf{y}_i^*$ to lie outside the body. This, in turn, could complicate the **proj** operation.

Some theoretical and computational analyses of Algorithm 11 (for the CVT case) can be found in Chapter 2. We also note that Algorithm 11 can be parallelized in much the same way it was described for the standard for the CVT case.

4.3.2 Some results about Lloyd's method

Let us consider the Lloyd map $\mathbf{Z}_{n+1} = \mathbf{T}(\mathbf{Z}_n)$ on a surface $\mathbf{S}$. From Propositions 5 and 6, we have

$$\begin{aligned}
 \mathcal{K}(\mathbf{Z}_{n+1}) &= \mathcal{F}(\mathbf{Z}_{n+1}, V(\mathbf{Z}_{n+1})) \\
 &\leq \mathcal{F}(\mathbf{Z}_{n+1}, V(\mathbf{Z}_n)) \\
 &\leq \mathcal{F}(\mathbf{Z}_n, V(\mathbf{Z}_n)) \\
 &= \mathcal{K}(\mathbf{Z}_n),
 \end{aligned} \tag{4.15}$$

where $V(\mathbf{Z})$ denotes the Voronoi regions of $\mathbf{S}$ associated with $\mathbf{Z}$. From (4.15), we know the Lloyd iteration produces a sequence of points $\{\mathbf{Z}_n\}$ on $\mathbf{S}$ that is bounded and that has monotonically decreasing energy. Applying the above results and a similar proof in (16), we can obtain the following result.

Proposition 7. *The set of limit points of a given Lloyd iteration share the same energy $\mathcal{K}$. Consequently, if the fixed point with the same energy is unique, then the Lloyd iteration converges globally.*

It is difficult in general to formulate verifiable conditions on the density function and the geometry of the surface that lead to global convergence. In the following, we will consider a special case.

Let us consider using Lloyd's method to construct constrained Voronoi tessellations on the unit circle $\mathbf{S} = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\}$. Let the density function $\rho(\cdot)$ be smooth and strictly positive. For convenience, we represent the unit circle by $\mathbf{S} = \{(\cos(\theta), \sin(\theta)) \mid 0 \leq \theta < 2\pi\}$ and $\rho(\theta) = \rho(\cos(\theta), \sin(\theta))$. In addition, $\rho(\cdot)$ is assumed to satisfy

$$\left(\frac{\rho'(\beta)}{\rho(\beta)} - \frac{\rho'(\alpha)}{\rho(\alpha)} \right) \sin(\alpha - \beta) > 0 \quad \forall \alpha \neq \beta. \tag{4.16}$$

Given k points $\mathbf{X} = \{(\cos(\theta_i), \sin(\theta_i)), i = 1, \dots, k\}$ such that $0 \leq \theta_1 < \theta_2 < \dots < \theta_k < 2\pi$, it is clear that the corresponding Voronoi regions are $V_i = \{(\cos(\theta), \sin(\theta)) \mid \theta \in$

$\triangle\theta_i\}$, where

$$\begin{aligned}\triangle\theta_1 &= \left(\frac{\theta_k + \theta_1}{2} - \pi, \frac{\theta_1 + \theta_2}{2} \right) \\ \triangle\theta_i &= \left(\frac{\theta_{i-1} + \theta_i}{2}, \frac{\theta_i + \theta_{i+1}}{2} \right) \quad \text{for } i = 2, \dots, k-1 \\ \triangle\theta_k &= \left(\frac{\theta_{k-1} + \theta_k}{2}, \frac{\theta_k + \theta_1}{2} + \pi \right).\end{aligned}\tag{4.17}$$

Then, the Lloyd map $\mathbf{T} = (T_1, \dots, T_k)$ can be defined as

$$T_i(\mathbf{X}) = \frac{\mathbf{y}_i}{|\mathbf{y}_i|}, \quad \text{where} \quad \mathbf{y}_i = \frac{\int_{V_i} \mathbf{x} \rho(\mathbf{x}) d\mathbf{x}}{\int_{V_i} \rho(\mathbf{x}) d\mathbf{x}}.\tag{4.18}$$

Using (4.17), the $\mathbf{y}_i$'s of (4.18) can be rewritten as

$$\mathbf{y}_i = \left(\frac{\int_{\triangle\theta_i} \rho(\theta) \cos(\theta) d\theta}{\int_{\triangle\theta_i} \rho(\theta) d\theta}, \quad \frac{\int_{\triangle\theta_i} \rho(\theta) \sin(\theta) d\theta}{\int_{\triangle\theta_i} \rho(\theta) d\theta} \right).\tag{4.19}$$

Letting $\Theta = \{\theta_1, \dots, \theta_k\}$, we can redefine the Lloyd map $\mathbf{T} : [0, 2\pi)^k \rightarrow [0, 2\pi)^k$ as

$$T_i(\Theta) = \tan^{-1} \left(\frac{\int_{\triangle\theta_i} \rho(\theta) \sin(\theta) d\theta}{\int_{\triangle\theta_i} \rho(\theta) \cos(\theta) d\theta} \right).\tag{4.20}$$

Without loss the generality, we assume that $T_i(\Theta) \in [0, \pi/2)$ and $1 < i < k$ since there needs only a rotation for other case. Let $\theta^- = \frac{\theta_{i-1} + \theta_i}{2}$ and $\theta^+ = \frac{\theta_i + \theta_{i+1}}{2}$; Then, we have

$$T_i(\Theta) = \tan^{-1} \left(\frac{\int_{\theta^-}^{\theta^+} \rho(\theta) \sin(\theta) d\theta}{\int_{\theta^-}^{\theta^+} \rho(\theta) \cos(\theta) d\theta} \right).\tag{4.21}$$

Let $\phi(\theta) = \rho(\theta) \cos(\theta)$ and $\psi(\theta) = \rho(\theta) \sin(\theta)$. Then, at the fixed point $\Theta = \mathbf{T}(\Theta)$, the

Jacobian matrix of the Lloyd map is the tridiagonal matrix

$$\left\{ \begin{array}{l} \frac{\partial T_i}{\partial \theta_{i-1}} = \frac{-\psi(\theta^-) \int_{\theta^-}^{\theta^+} \phi(\theta) d\theta + \phi(\theta^-) \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta}{2(1 + \tan^2 \theta_i) \left(\int_{\theta^-}^{\theta^+} \phi(\theta) d\theta \right)^2} \\ \frac{\partial T_i}{\partial \theta_{i+1}} = \frac{\psi(\theta^+) \int_{\theta^-}^{\theta^+} \phi(\theta) d\theta - \phi(\theta^+) \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta}{2(1 + \tan^2 \theta_i) \left(\int_{\theta^-}^{\theta^+} \phi(\theta) d\theta \right)^2} \\ \frac{\partial T_i}{\partial \theta_i} = \frac{\partial T_i}{\partial \theta_{i-1}} + \frac{\partial T_i}{\partial \theta_{i+1}} \end{array} \right. \quad (4.22)$$

From (4.18) and (4.19), we know that at a fixed point Θ ,

$$\int_{\theta^-}^{\theta^+} \phi(\theta) d\theta = M_i \cos(\theta_i) \quad \text{and} \quad \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta = M_i \sin(\theta_i)$$

where $M_i = |\mathbf{y}_i| \int_{\theta^-}^{\theta^+} \rho(\theta) d\theta$; then,

$$\left\{ \begin{array}{l} \frac{\partial T_i}{\partial \theta_{i-1}} = \frac{\rho(\theta^-) \sin(\theta_i - \theta^-)}{2M_i(1 + \tan^2 \theta_i) \cos^2(\theta_i)} = \frac{\rho(\theta^-) \sin(\theta_i - \theta^-)}{2M_i} \\ \frac{\partial T_i}{\partial \theta_{i+1}} = \frac{\rho(\theta^+) \sin(\theta^+ - \theta_i)}{2M_i(1 + \tan^2 \theta_i) \cos^2(\theta_i)} = \frac{\rho(\theta^+) \sin(\theta^+ - \theta_i)}{2M_i} \end{array} \right. \quad (4.23)$$

since $0 < \theta_i - \theta^- = \frac{\theta_i - \theta_{i-1}}{2} < \pi$ and $0 < \theta_+ - \theta_i = \frac{\theta_{i+1} - \theta_i}{2} < \pi$. From (4.23) one easily sees that $\partial \mathbf{T} / \partial \Theta$ at the fixed point is a non-negative matrix.

Using (4.22) and (4.23), we have

$$\begin{aligned} M_i \left(1 - \sum_{j=1}^k \frac{\partial T_i}{\partial \theta_j} \right) &= M_i \left(1 - 2 \frac{\partial T_i}{\partial \theta_{i-1}} - 2 \frac{\partial T_i}{\partial \theta_{i+1}} \right) \\ &= M_i - \rho(\theta^-) \sin(\theta_i - \theta^-) - \rho(\theta^+) \sin(\theta^+ - \theta_i) \\ &= \left(\int_{\theta^-}^{\theta^+} \phi(\theta) d\theta \right)^2 + \left(\int_{\theta^-}^{\theta^+} \psi(\theta) d\theta \right)^2 \\ &\quad - \left[-\psi(\theta^-) \int_{\theta^-}^{\theta^+} \phi(\theta) d\theta + \phi(\theta^-) \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta \right] \\ &\quad - \left[\psi(\theta^+) \int_{\theta^-}^{\theta^+} \phi(\theta) d\theta - \phi(\theta^+) \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta \right] \end{aligned}$$

$$\begin{aligned}
&= \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} \phi(\alpha)\phi(\beta) d\alpha d\beta + \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} \psi(\alpha)\psi(\beta) d\alpha d\beta \\
&\quad - (\psi(\theta^+) - \psi(\theta^-)) \int_{\theta^-}^{\theta^+} \phi(\theta) d\theta + (\phi(\theta^+) - \phi(\theta^-)) \int_{\theta^-}^{\theta^+} \psi(\theta) d\theta \\
&= \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} [(\phi(\alpha) - \psi'(\alpha))\phi(\beta) + (\psi(\alpha) + \phi'(\alpha))\psi(\beta)] d\alpha d\beta \\
&= \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} [-\rho'(\alpha)\rho(\beta)\sin(\alpha)\cos(\beta) + \rho'(\alpha)\rho(\beta)\sin(\beta)\cos(\alpha)] d\alpha d\beta \\
&= \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} \rho'(\alpha)\rho(\beta)\sin(\beta - \alpha) d\alpha d\beta \\
&= \frac{1}{2} \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} (\rho'(\alpha)\rho(\beta) - \rho'(\beta)\rho(\alpha)) \sin(\beta - \alpha) d\alpha d\beta \\
&= \frac{1}{2} \int_{\theta^-}^{\theta^+} \int_{\theta^-}^{\theta^+} \rho(\alpha)\rho(\beta) \left(\frac{\rho'(\beta)}{\rho(\beta)} - \frac{\rho'(\alpha)}{\rho(\alpha)} \right) \sin(\beta - \alpha) d\alpha d\beta, \\
&> 0,
\end{aligned}$$

where the last step follows from (4.16). Thus, by the Gerschgorin theorem, the spectral radius of the Jacobian matrix is less than 1 and we have the local convergence of the Lloyd iteration on the circle.

Theorem 3. *Let $\mathbf{S}$ be the unit circle and assume that $\rho(\cdot)$ is a smooth density function that satisfies*

$$\left(\frac{\rho'(\beta)}{\rho(\beta)} - \frac{\rho'(\alpha)}{\rho(\alpha)} \right) \sin(\alpha - \beta) > 0 \quad \forall \alpha \neq \beta.$$

Then, the Lloyd map $\mathbf{T}$ is a local contraction near its fixed points. Consequently, the Lloyd iteration is locally convergent.

4.4 Computational examples

Three different surfaces were used to illustrate CCVT point sets. In all examples, Algorithm 11 was used with $\alpha_1 = \beta_1 = 0$ and $\alpha_2 = \beta_2 = 1$. Monte Carlo sampling, i.e., random sampling according to a given density function, on a given surface plays a key role in that method. The main ingredient in our sampling procedure is to use the

rejection method (60). Additionally, we need to implement the projection process **proj** onto a surface.

4.4.1 CCVT's on the surface of the unit sphere

First, let $\mathbf{S}$ be the surface of the unit sphere, $\mathbf{S} = \{(x, y, z) \mid x^2 + y^2 + z^2 = 1\}$. The projection process **proj** is quite simple for this surface because of its simple geometry. It is easy to show that the mass centroid of any region of the surface of the unit sphere is always located inside the unit sphere, so the **proj** process can be easily implemented as follows. The projection of a given point (x, y, z) inside the unit sphere (except for the origin, i.e., the center) onto the surface of the sphere is given by

$$\mathbf{proj}(x, y, z) = \frac{1}{r}(x, y, z) \quad \text{where } r = \sqrt{x^2 + y^2 + z^2}. \quad (4.24)$$

We use the following procedure to sample points on $\mathbf{S}$, the surface of unit sphere, for a given density function $\rho(\cdot)$ defined on $\mathbf{S}$.

Procedure 1. Set $\hat{\rho} = \max_{(x,y,z) \in \mathbf{S}} \rho(x, y, z)$;

1. sample three random numbers X' , Y' , and Z' uniformly distributed on $[0, 1]$; set $X = 2X' - 1$, $Y = 2Y' - 1$, and $Z = 2Z' - 1$;
2. set $R = \sqrt{X^2 + Y^2 + Z^2}$; if $0 < R \leq 1$, then set $(x, y, z) = \mathbf{proj}(X, Y, Z)$; otherwise, go to step 1;
3. sample another random number U uniformly distributed on $[0, 1]$; if $U < \rho(x, y, z)/\hat{\rho}$, return (x, y, z) ; otherwise go to step 1.

The results of some computationally determined 256-point CCVT's on the unit sphere are given in Fig. 4.1. Three different density functions were chosen: a constant density function; $e^{-6.0z^2}$ which has a large variation, i.e., its values range from 1 to e^{-6} and reaches its maximum on the circle $\{(x, y, z) \mid x^2 + y^2 = 1, z = 0\}$; and $e^{-3.0(1-z)^2}$

which has a large peak at the north pole $(0, 0, 1)$. The Voronoi diagrams were produced using the STRIPACK package (59). Also, the energies for these tessellations are given in Table 4.1.

Table 4.1 Energies of 256-point constrained centroidal Voronoi tessellations of a sphere for three density functions.

Density function	$\rho(x, y, z) = 1$	$\rho(x, y, z) = e^{-6.0z^2}$	$\rho(x, y, z) = e^{-3.0(1-z)^2}$
Monte Carlo simulation	1.93818E-01	5.67047E-02	3.08113E-02
Constrained CVT	9.99540E-02	2.53291E-02	1.29333E-02

4.4.2 CCVT's on a developable surface

We next choose the developable surface $\mathbf{S} = \{(x, y, z) \mid z = -x^2, |x| \leq \frac{1}{2}, |y| \leq \frac{1}{2}\}$. It is easy to show that the mass centroid of any region of this surface always belongs to the set $Q = \{(x, y, z) \mid |x| < \frac{1}{2}, |y| < \frac{1}{2}, -\frac{1}{4} < z < -x^2\}$. For a given point $(x, y, z) \in Q$, its projection onto $\mathbf{S}$, i.e., $(\tilde{x}, \tilde{y}, \tilde{z}) = \mathbf{proj}(x, y, z)$, is found by solving

$$\begin{cases} \frac{2\tilde{x}(z - \tilde{z})}{x - \tilde{x}} = 1 \\ \tilde{z} = -\tilde{x}^2 \\ \tilde{y} = y. \end{cases}$$

These may be combined into the cubic equation

$$\tilde{x}^3 + \frac{2z+1}{2}\tilde{x} - x = 0. \quad (4.25)$$

Since x always belongs to $(-\frac{1}{2}, \frac{1}{2})$ and z is restricted to $(-\frac{1}{4}, -x^2)$, it is not difficult to verify that (4.25) has only one real solution; thus, $(\tilde{x}, \tilde{y}, \tilde{z})$ is uniquely determined.

A special procedure is given in the following to do sampling on the developable surface $\mathbf{S}$ for a given density function $\rho(\cdot)$ defined on $\mathbf{S}$. Two rejection steps are used.

Procedure 2. Set $\hat{\rho} = \max_{(x,y,z) \in \mathbf{S}} \rho(x, y, z)$;

1. *sample two random numbers X' and Y' uniformly distributed on $[0, 1]$; set $X = X' - \frac{1}{2}$ and $Y = Y' - \frac{1}{2}$;*
2. *set $V = \sqrt{(1 + 4X^2)/2}$ and sample a random number U_1 uniformly distributed in $[0, 1]$; if $U_1 < V$, set $x = X$, $y = Y$ and $z = -x^2$, and go to step 3; otherwise, go to step 1;*
3. *sample another random number U_2 uniformly distributed in $[0, 1]$; if $U_2 < \rho(x, y, z)/\hat{\rho}$, return (x, y, z) ; otherwise go to step 1.*

The results of some computationally determined 256-point CCVT's on the developable surface are given in Fig. 4.2. Two different density functions were chosen: a constant density function and $e^{-20.0x^2}$ which has a large variation, i.e., its values range from 1 to e^{-5} and reaches its maximum on the segment $\{(x, y, z) \mid x = z = 0, y \in [-\frac{1}{2}, \frac{1}{2}]\}$. Since there is currently no software available for determining Voronoi diagrams for a given set of points on such a developable surface, only the positions of the generators are given in Fig. 4.2.

4.4.3 CCVT's on the surface of a torus

For the last example, we choose the surface of the torus $\mathbf{S} = \{(x, y, z) \mid (x - \frac{x}{r})^2 + (y - \frac{y}{r})^2 + z^2 = 0.3^2, r = \sqrt{x^2 + y^2}\}$. The projection process **proj** for this surface is similar to that for the surface of a sphere but is a little more complicated. It is easy to show that the mass centroid of any region of the surface of this torus will always belong to the set $Q = \{(x, y, z) \mid x^2 + y^2 < 1.3^2, |z| < 0.3\}$. Thus, for a given point $(x, y, z) \in Q$ except for the origin $(0, 0, 0)$, we have

$$\mathbf{proj}(x, y, z) = (\bar{x} + \hat{x}, \bar{y} + \hat{y}, \hat{z}),$$

where

$$\begin{cases} (\bar{x}, \bar{y}) = \frac{1}{d}(x, y) & \text{and } d = \sqrt{x^2 + y^2} \\ (\hat{x}, \hat{y}, \hat{z}) = \frac{0.3}{d}(x - \bar{x}, y - \bar{y}, z) & \text{and } d = \sqrt{(x - \bar{x})^2 + (y - \bar{y})^2 + z^2}. \end{cases}$$

The following procedure is for sampling points on the torus $\mathbf{S}$ for a given density function $\rho(\cdot)$ defined on $\mathbf{S}$.

Procedure 3. Set $\hat{\rho} = \max_{(x,y,z) \in \mathbf{S}} \rho(x, y, z)$;

1. sample three random numbers X' , Y' , and Z' uniformly distributed in $[0, 1]$; set $X = 1.3(2X' - 1)$, $Y = 1.3(2Y' - 1)$, and $Z = 0.3(2Z' - 1)$;
2. set $R = \sqrt{X^2 + Y^2}$ and $V = \sqrt{(X - X/R)^2 + (Y - Y/R)^2 + Z^2}$; if $0 < V \leq 0.3$, then set $(x, y, z) = \mathbf{proj}(X, Y, Z)$ and go to step 3; otherwise, go to step 1;
3. sample a random number U uniformly distributed in $[0, 1]$; if $U < \rho(x, y, z)/\hat{\rho}$, return (x, y, z) ; otherwise go to step 1.

The results of some computationally determined 256-point CCVT's on the torus are given in Fig. 4.3. Two different density functions were chosen: a constant density function; and $e^{-5.0|y|}$ which has a large variation, i.e., its values range from 1 to $e^{-6.5}$ and reaches its maximum on the two circles $\{(x, y, z) \mid (x - 1)^2 + z^2 = 0.3^2, y = 0\}$ and $\{(x, y, z) \mid (x + 1)^2 + z^2 = 0.3^2, y = 0\}$. Again, since there is no existing software for determining the Voronoi tessellations on a torus, only the positions of the generators are given in Fig. 4.3.

4.5 CCVT's for interpolation and quadrature on the sphere

We now examine the use of the generators of centroidal Voronoi tessellations of the surface of the sphere for interpolation and quadrature. We will only consider the case of uniformly distributed points, i.e., of a constant density function $\rho(\mathbf{x})$.

The *mesh norm* h of a set of points $\{\mathbf{x}_i\}_{i=1}^k$ on the unit sphere S^2 is defined by

$$h = \max_{\mathbf{x} \in S^2} \min_{i=1, \dots, k} \cos^{-1}(\mathbf{x}^T \mathbf{x}_i).$$

Of course, it is topologically impossible to tessellate the surface of a sphere exactly uniformly. Note, however, that it is clear that a “uniform” tessellation of the surface of a sphere into hexagonal-like regions would result in $h\sqrt{k} \approx \sqrt{8\sqrt{3}\pi/9} \approx 2.2$ when k is large. Thus, we can use h as an indicator of the uniformity of point distributions on the sphere. In Fig. 4.4 we provide plots of h and $h(n+1)$ vs. n for $k = (n+1)^2$ -point CCVT’s of the surface of the unit sphere. (We set $k = (n+1)^2$ to be consistent with the plot given below for global interpolation on the sphere.) We determine the mesh norm h by sampling 40,000 points $\mathbf{x}$ on the sphere S^2 and then selecting the maximum value of $\min_{i=1, \dots, k_n} \cos^{-1}(\mathbf{x}^T \mathbf{x}_i)$ over the sampling points. We see from Fig. 4.4 that $h(n+1)$ is indeed nearly constant and nearly equal to 2.2 for CCVT point distributions. This implies that CCVT point distributions would be useful for piecewise polynomial interpolation on the sphere and for finite element discretizations of partial differential equations posed on a sphere.

We now consider global polynomial interpolation on the sphere S^2 . In (69; 73), good choices of points for this purpose are discussed in detail. Note that if the degree of interpolating polynomial is n , then $k_n = (n+1)^2$ interpolating points $\{\mathbf{x}_i\}_{i=1}^{k_n}$ are needed for global interpolation on the sphere. The “goodness” of a set of interpolation points can be characterized by the uniform norm of the interpolation operator. Following (69; 73), we determine this quantity as follows. Let $\tilde{G}_n(z) = \frac{n+1}{4\pi} \mathbf{P}_n^{(1,0)}(z)$, where $\mathbf{P}_n^{(a,b)}$ denote the Jacobi polynomials. Define

$$\mathbf{g}(\mathbf{x}) = \left(\tilde{G}_n(\mathbf{x}^T \mathbf{x}_1), \tilde{G}_n(\mathbf{x}^T \mathbf{x}_2), \dots, \tilde{G}_n(\mathbf{x}^T \mathbf{x}_{k_n}) \right)^T$$

and the $k_n \times k_n$ Gram matrix G of the reproducing kernel basis

$$G = \left(\mathbf{g}(\mathbf{x}_1), \mathbf{g}(\mathbf{x}_2), \dots, \mathbf{g}(\mathbf{x}_{k_n}) \right).$$

Then, the uniform norm of the interpolation operator Λ_n is given by

$$\|\Lambda_n\|_{L^\infty} = \max_{\mathbf{x} \in S^2} \|G^{-1}\mathbf{g}(\mathbf{x})\|_{\ell^1}.$$

In Fig. 4.5, we provide, for $k_n = (n+1)^2$ -point CCVT point distributions, a plot of $\|\Lambda_n\|_{L^\infty}$ vs. n , where n is the degree of the interpolating polynomial. We determine $\|\Lambda_n\|_{L^\infty}$ by sampling 40,000 points $\mathbf{x}$ on the sphere S^2 and then selecting the maximum value of $\|G^{-1}\mathbf{g}(\mathbf{x})\|_{\ell^1}$ over the sampling points.

We next turn to interpolatory quadrature on the surface of S^2 based on the interpolating polynomial of degree n . If $\{\mathbf{x}_i\}_{i=1}^{k_n}$ is a set of quadrature points, then the quadrature weights are determined by the requirement that

$$\int_{S^2} p(\mathbf{x}) dx = \sum_{i=1}^{k_n} w_i p(\mathbf{x}_i)$$

for all polynomials p of degree $\leq n$. Following (69; 73), we determine the quadrature weights $\mathbf{w} = (w_1, w_2, \dots, w_{k_n})$ by solving the linear algebraic system

$$G\mathbf{w} = \mathbf{e},$$

where $\mathbf{e}$ is the vector having all components equal to one. In Fig. 4.6, we provide a plot of the maximum and minimum values for the quadrature weights vs. n , the degree of the polynomial.

In (69; 73), four types of point sets for global interpolation and quadrature on a sphere are compared. One of the point sets is based on minimizing a Coulomb type potential energy between points; it is dismissed as yielding very large interpolation operator norms and very poor quadrature weights, including some that are negative. CCVT point sets are, in these respects, vastly superior to Coulomb potential point sets. Compared to the other three types of point sets discussed in (69; 73), CCVT point sets are as good as the best of those with respect to quadrature weights; note from Fig. 4.6 that the quadrature weights for quadrature points determined from CCVT's are all positive; the

spread between the maximum and minimum quadrature weights is comparable to that of the best of the point sets discussed in (69; 73). CCVT points sets are also every bit as good as the best point set of (69; 73) with respect to the mesh norm h . With respect to the size of the norm of the interpolation operator, CCVT point sets do not perform as well as do the three best point sets discussed in (69; 73). However, as shown in Fig. 4.5, the growth in this norm for CCVT is acceptable for most practical purposes. Moreover, it is important to note that CCVT point sets may be determined at very much less cost than what is needed for the three good point sets discussed in (69; 73), and the CCVT concept are also applicable when studying similar problems for more general surfaces.

One important observation is that all three algorithms discussed in Section 4.3 (and indeed all common algorithms for determining CVT's and CCVT's) merely locate *local* minimizers of the energy functional (4.9). This may account for the lack of monotonicity in the plots in Figures 4.4–4.6. Moreover, it is possible that if global minimizers were located that the performance of CCVT point sets for global interpolation on the sphere would be as good as the best point set discussed in (69; 73). Algorithms for the determination of CVT and CCVT point sets that are global minimizers of the energy functional (4.9) are currently under study.

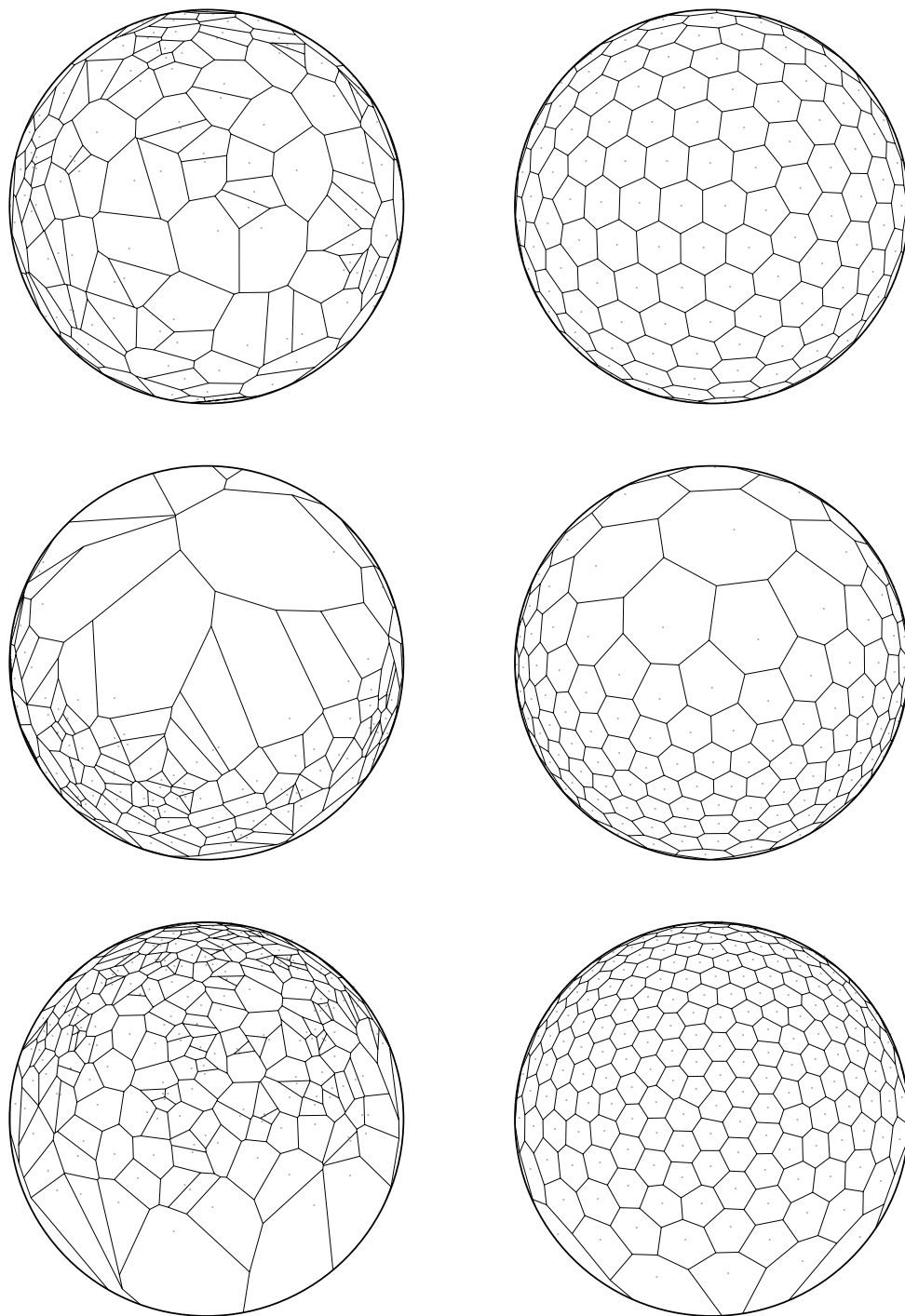


Figure 4.1 Voronoi diagrams of 256 generators on the surface of unit sphere. Left: Monte Carlo point set; right: constrained CVT point set; top: $\rho(x, y, z) = 1$; middle $\rho(x, y, z) = e^{-6.0z^2}$; bottom: $\rho(x, y, z) = e^{-3.0(1-z)^2}$.

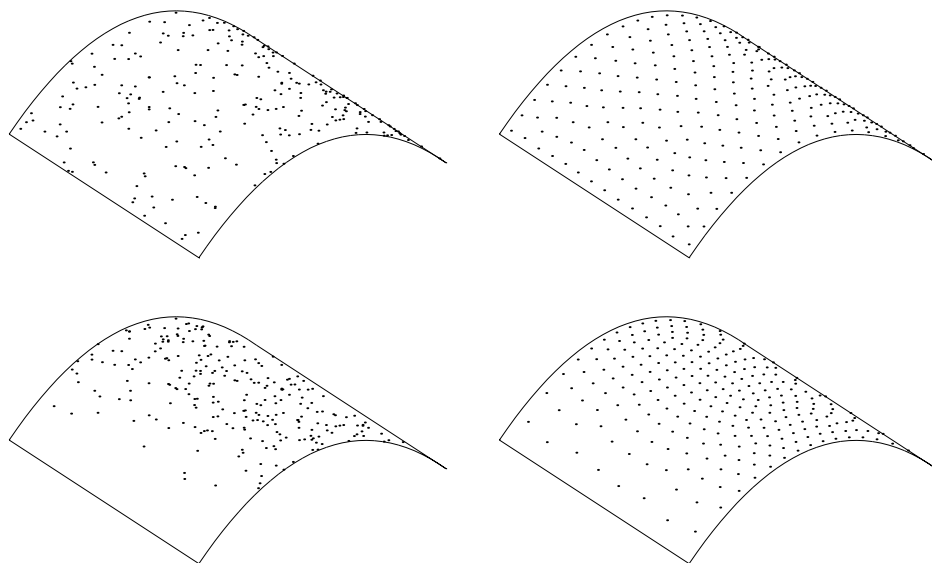


Figure 4.2 Voronoi diagrams of 256 generators on a developable surface. Left: Monte Carlo point set; right: Constrained CVT point set; top: $\rho(x, y, z) = 1$; bottom: $\rho(x, y, z) = e^{-20.0x^2}$.

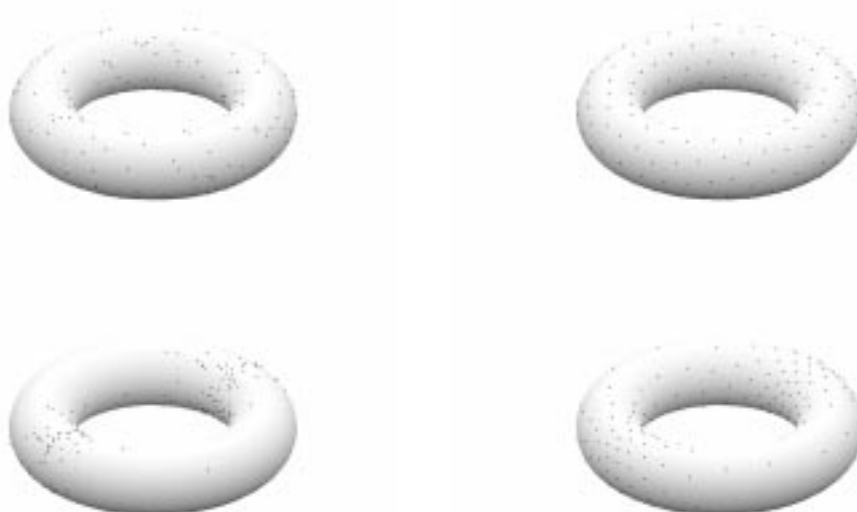


Figure 4.3 Voronoi diagrams of 256 generators on a torus. Left: Monte Carlo point set; right: Constrained CVT point set; top: $\rho(x, y, z) = 1$; bottom: $\rho(x, y, z) = e^{-5.0|y|}$.

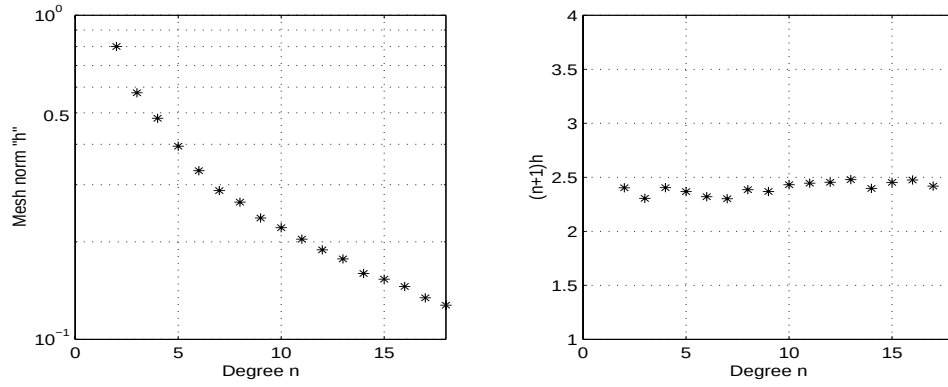


Figure 4.4 Mesh norm of constrained CVT generators vs. degree of the polynomial n ; the number of generators $k_n = (n + 1)^2$.

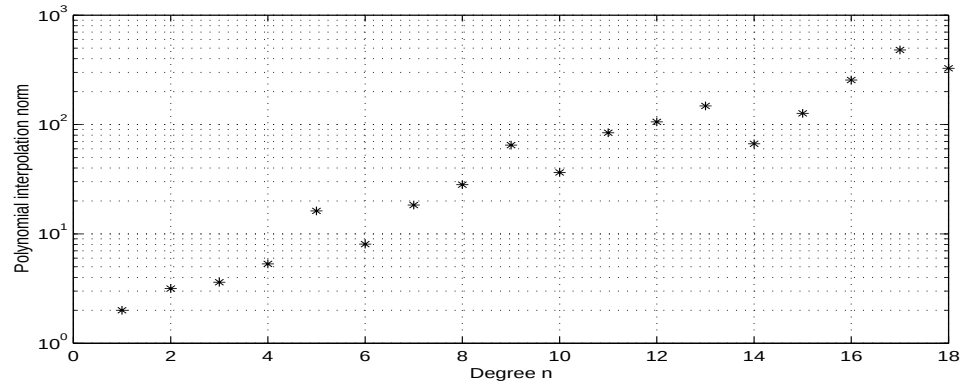


Figure 4.5 Global polynomial interpolation norms using constrained CVT generators vs. degree of the polynomial n ; the number of generators $k_n = (n + 1)^2$.

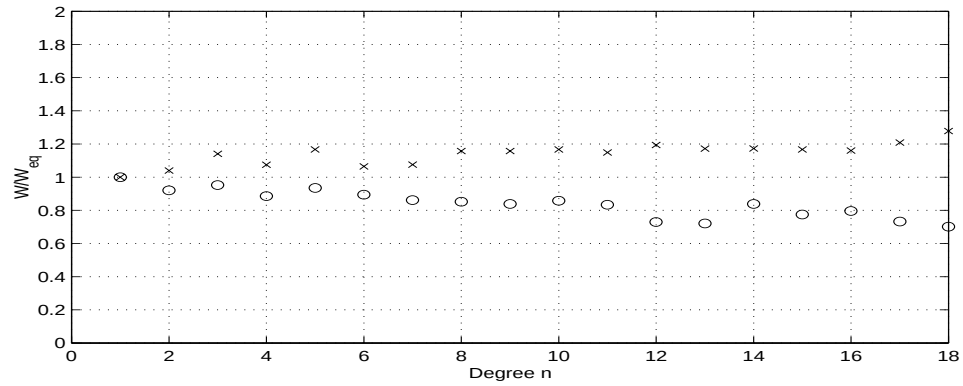


Figure 4.6 Maximum and minimum value of the quadrature weights (scaled by $4\pi/k_n$) when the quadrature points are chosen to be constrained CVT generators vs. degree of the polynomial n ; the number of generators is $k_n = (n + 1)^2$.

CHAPTER 5. CONCLUSIONS AND CURRENT WORK

In Chapter 2, we examined several probabilistic algorithms and their parallel implementations for determining centroidal Voronoi tessellations. From the results of some computational experiments, it seems that a new probabilistic algorithm we introduced performs much better than the random MacQueen's method and is more suitable for parallel computation. The parallel implementation of Algorithm 3 for determining point placements exhibits near perfect linear speedup respective to the number of processors.

There also are a number of issues that need further study. These include, for example, improving the convergence rate of MacQueen's method, finding better parallel implementations, optimal choices for s (or equivalently $q = ps$), and optimal choices of the parameters $\{\alpha_i, \beta_i\}_{i=1}^2$ for Algorithms 3 and 6. Of course, theoretical convergence analyses for Algorithms 3, 5, and 6 would also be desirable.

The new algorithms we have presented in Chapter 3 result in, for both uniform and nonuniform point distributions, high-quality point sets and high-quality support regions. Furthermore, being probabilistic in nature, the algorithms are totally meshfree, i.e., they do not require, at any stage, the use of coarse or fine meshes. They also can be easily parallelized. Algorithms 7 and 8 for support radii determination also possess obvious opportunities for parallelization.

In addition to testing our point placement and support radii determination algorithms in complex three dimensional domains, we are currently studying a number of other issues connected with the implementation and use of the algorithms. These include the following.

- Connecting the density function used for generating centroidal Voronoi point sets to a priori and a posteriori error estimates; then adaptive point generation algorithms could be easily defined.
- Using anisotropic (tensor valued) density functions for determining point distributions with special properties such as high aspect ratio spacings; these would be useful for, e.g., resolving boundary layers.
- Testing the use of the algorithms in function approximation methods, e.g., interpolation, and in Galerkin methods for the numerical solution of partial differential equations.
- Analyzing properties of the algorithms such as convergence behavior.

Bibliography

- [1] S. ARYA, D. MOUNT, N. NETANYAHU, R. SILVERMAN, AND A. WU; An optimal algorithm for approximate nearest neighbor searching, *J. ACM* **45** 1998, pp. 891-923.
- [2] E. ABAYA AND F. WISE; *Convergence of vector quantizers with applications to optimal quantization*, SIAM J. Appl. Math. **44**, 1984, pp. 183–189.
- [3] T. ASANO, B. BHATTACHARYA, M. KEIL, AND F. YAO; *Clustering algorithms based on minimum and maximum spanning trees*, Proceedings of the 4th Annual Symposium on Computational Geometry, Urbana, 1988, pp.252–257.
- [4] J. BENTLEY; Multidimensional binary search trees used for associative searching, *Comm. ACM* **18** 1975, pp. 509-517.
- [5] G. BARLOW; *Hexagonal territories*, Animal Behav. **22**, 1974, pp. 876–878.
- [6] T. BELYTSCHKO, Y. KRONGAUZ, D. ORGAN, M. FLEMING, AND P. KRYSL; Meshless methods: an overview and recent developments, *Comput. Meths. Appl. Mech. Engrg* **139** 1996, pp. 3–47.
- [7] T. BELYTSCHKO, Y. LU AND L. GU; Element-free Galerkin methods, *Int. J. Numer. Meth. Engrg.* **37** 1994, pp. 229–256.
- [8] I. BABUSKA AND J. MELENK; The partition of unity finite element method: basic theory and applications, *Comput. Meths. Appl. Mech. Engrg* **139** 1996, pp. 289–314.

- [9] Y. CHOI AND S. KIM; Node generation scheme for meshfree method by Voronoi diagram and weighted bubble packing, *Proc. Fifth U.S. National Congress on Computational Mechanics*, Boulder, 1999.
- [10] R. CLARK; *Optisim – An extended dissimilarity selection method for finding diverse representative subsets*, J. Chem. Inform. Comp. Sci. **37**, 1997, pp. 1181–1188.
- [11] CRAY INC. WEB SITE, <http://www.cray.com>
- [12] P. DAVIS AND P. RABINOWITZ; *Methods of Numerical Integration*, 2nd edition, Academic, 1984.
- [13] Q. DU, V. FABER, AND M. GUNZBURGER; Centroidal Voronoi tessellations: Applications and algorithms, *SIAM Review* **41** 1999, pp. 637-676.
- [14] Q. DU AND M. GUNZBURGER; Grid generation and optimization based on centroidal Voronoi tessellations, accepted by *Appl. Math. Comput.*, 2001.
- [15] C. DUARTE AND J. T. ODEN; Hp Clouds – a meshless method to solve boundary value problems, *Num. Meth. PDE* **12** 1996, pp. 673–705.
- [16] Q. DU AND T. WONG; On the Lloyd’s algorithm for computing CVTs, *Appl. Numer. Anal.*, accepted, 2000.
- [17] A. FAULDS AND B. KING; Sensor location in feedback control of partial differential equation systems, *Proc. the 200 IEEE CCA/CACSD*, IEEE, Washington, 2000, 536–541.
- [18] J. FRIEDMAN; *Exploratory projection pursuit*, J. Amer. Stat. Assoc. **82**, 1987, pp.249–266.
- [19] M. GAREY AND D. JOHNSON; *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, 1979.

- [20] A. GERSHO; *Asymptotically optimal block quantization*, IEEE Trans. Inform. Theory **25**, 1979, pp. 373–380.
- [21] A. GERSHO AND R. GRAY; *Vector Quantization and Signal Compression*, Kluwer, Boston, 1992.
- [22] R. GRAY AND D. NEUHOFF; *Quantization*, IEEE Trans. Inform. Theory **44**, 1998, pp. 2325–2383.
- [23] M. GRIEBEL AND M. SCHWEITZER; A particle-partition of unity method for the solution of elliptic, parabolic and hyperbolic PDEs, *SIAM J. Sci. Comp.*, Vol. **22**, No. 3, pp. 853–890, 2000
- [24] H. IMAI AND M. INABA; *Geometric k-clustering with applications*, Proceedings of the 3rd International Congress of Industrial and Applied Mathematics, to appear, 1995.
- [25] P. HALL; *On polynomial-based projection indices for exploratory projection pursuit*, Ann. Stat. **17**, 1989, pp. 589–605.
- [26] J. HARTIGAN; *Clustering Algorithms*, Wiley-Interscience, New York, 1975.
- [27] J. HARTIGAN; *Asymptotic distribution for clustering centers*, Ann. Stat. **6**, 1978, pp. 117–131.
- [28] P. HUBER; *Projection pursuit, with discussion*, Ann. Stat. **13**, 1985, pp.435–525.
- [29] P. HECKERT; *Color image quantization frame buffer display*, ACM Trans. Comp. Graph. **16**, 1982, pp. 297–304.
- [30] F. HICKERNELL AND H. HONG; *The asymptotic efficiency of randomized nets for quadrature*, Math. Comp. **68**, 1999, pp. 767–791.

- [31] H. HONDA; *Description of cellular patterns by Dirichlet domains: The two-dimensional case*, J. Theor. Biol. **72**, 1978, pp. 523–543.
- [32] H. HONDA; *Geometrical modelling for cells in tissues*, Inter. Rev. Cytol. **81**, 1983, pp. 191–248.
- [33] M. HASEGAWA AND M. TANEMURA; *On the pattern of space division by territories*, Ann. Inst. Stat. Math. **28**, 1976, pp. 509–519.
- [34] M. HASEGAWA AND M. TANEMURA; *Spatial patterns of territories*, Recent Developments in Statistical Inference and Data Analysis, ed. by K. Matusita, North-Holland, 1980, pp. 73–78.
- [35] S. HASEGAWA, H. IMAI, M. INABA, N. KATOH, AND J. NAKANO; *Efficient algorithms for variance-based k-clustering*, Proceedings of the First Pacific Conference on Computer Graphics and Applications, World Scientific, 1993, pp. 75–89.
- [36] H. IMAI, N. KATOH, AND M. INABA; *Applications of weighted Voronoi diagrams and randomization to variance-based k-clustering*, Proceedings of the 10th ACM Symposium on Computational Geometry, 1994, pp. 332–339.
- [37] M. INABA, H. IMAI, AND N. KATOH; *Experimental results of randomized clustering algorithms*, Proceedings of the 12th ACM Symposium on Computational Geometry: Communication Section, 1996, to appear.
- [38] M. IRI, K. MUROTA, AND T. OHYA; *A fast Voronoi-diagram algorithm with applications to geographical optimization problems*, Proceedings of the 11th IFIP Conference on System Modelling and Optimzation, Lecture Notes in Control and Information Sciences **59**, 1984, Springer, pp. 273–288.
- [39] M. JHUN; Bootstrapping k-means clustering, *J. Japanese Soc. Comput. Statist.* **3**, 1990, pp. 1-14.

- [40] A. JAIN AND R. DUBES; *Algorithms for Clustering Data*, Prentice-Hall, Englewood Cliffs, NJ, 1998.
- [41] R. KLEIN; *Concrete and Abstract Voronoi Diagrams*, Lecture Notes in Computer Science **400**, Springer, Berlin, 1989.
- [42] D. LEE, S. BAEK, AND K. SUNG; Modified K-means algorithm for vector quantizer design, *IEEE Signal Proc. Lett.* **4** 1997, pp. 2–4.
- [43] Y. LINDE, A. BUZO, AND R. GRAY; An algorithm for vector quantizer design, *IEEE Trans. Comm.* **28** 1980, pp. 84-95.
- [44] W. K. LIU, S. JUN AND Y. F. ZHANG; Reproducing kernel particle methods, *Int. J. Num. Methods Fluids*, Vol. 20, 1081-1106(1995).
- [45] S. LLOYD; Least square quantization in PCM, *IEEE Trans. Infor. Theory* **28** 1982, pp. 129-137.
- [46] X.-Y. LI, S.-H. TENG, AND A. UNGOR; Biting: advancing front meets sphere packing, *Int. J. Numer. Meth. Engrg.*, **49**, pp. 61-81, 2000.
- [47] J. MACQUEEN; Some methods for classification and analysis of multivariate observations, *Proc. Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol I, Ed. by L. Le Cam and J. Neyman, University of California, 1967, pp. 281-297.
- [48] MPI STANDARD WEB SITE, <http://www-unix.mcs.anl.gov/mpi/index.html>
- [49] D. NEWMAN; *The hexagon theorem*, *IEEE Trans. Inform. Theory* **28**, 1982, pp. 137–139.
- [50] R. NICOLAIDES; *Direct discretization of planar div-curl problems*, *SIAM J. Numer. Anal.* **29**, 1992, pp. 32–56.

- [51] H. NIEDERREITER; *Random Number Generation and Quasi-Monte Carlo Methods*, SIAM, Philadelphia, 1992.
- [52] A. OKABE, B. BOOTS, AND K. SUGIHARA; *Spatial Tessellations: Concepts and Applications of Voronoi Diagrams*, Wiley, Chichester, 1992.
- [53] K. PARNA; *Strong consistency of k-means clustering criterion in separable metric spaces*, Tartu Riikl. Ul. Toimetised **733**, 1986, pp. 86–96.
- [54] C. PAPADIMITRIOU AND K. STEIGLITZ; *Combinatorial Optimization: Algorithms and Complexity*, Prentice-Hall, Englewood Cliffs, 1982.
- [55] D. POLLARD; Strong consistency of k-means clustering, *Ann. Stat.* **10** 1982, pp. 135-140.
- [56] D. POLLARD; *A central limit theorem for k-means clustering*, *Ann. Stat.* **10**, 1982, pp. 919–926.
- [57] F. PREPARATA AND M. SHAMOS; *Computational Geometry – An Introduction*, Springer, New York, 1985.
- [58] A. PAPAGEORGIOU AND J. TRAUB; *Fast evaluation of multidimensional integral*, *Comput. in Phys.* **11**, 1997, pp. 574–579.
- [59] R. RENKA; ALGORITHM 772. STRIPACK: Delaunay triangulation and Voronoi diagrams on the surface of a sphere, *ACM Trans. Math. Soft.* **23** 1997, pp. 416-434.
- [60] S. ROSS; *A First Course in Probability*, 5th Edition, Prentice Hall, Englewood Cliffs, 1998.
- [61] H. SAMET; *The Design and Analysis of Spatial Data Structures*, Addison Wesley, Reading, 1990.

- [62] W. SHEPPARD; *On the calculation of the most probable values of frequency constants for data arranged according to equidistant divisions of a scale*, Proc. London Math. Soc. **24**, 1898, pt.2, pp. 353–390.
- [63] J. SWEGLE, S. ATTAWAY, F. MELLO, AND D. HICKS; An analysis of the smoothed particle hydrodynamics, Tech. Rep. SAND93-2513 UC-705, Sandia National Laboratories, Albuquerque, 1994.
- [64] J. SPANIER AND E. GELBARD; *Monte Carlo Principles and Neutron Transport Problems*, Addison-Wesley, New York, 1969.
- [65] A. SUZUKI AND M. IRI; *Approximation of a tessellation of the plane by a Voronoi diagram*, J. Oper. Res. Soc. Japan **29**, 1986, pp. 69–96.
- [66] M. SNIR, S. OTTO, S. HUSS-LEDERMAN, D. WALKER, AND J. DONGARRA; *MPI: The Complete Reference*, MIT, 1996.
- [67] W. STUTE AND L. ZHU; *Asymptotics of k-means clustering based on projection pursuit*, Sankhya Ser. A **57**, 1995, pp. 462–471.
- [68] I. SLOAN AND H. WOŹNIAKOWSKI; *When are quasi-Monte Carlo algorithms efficient for high-dimensional integrals?* J. Complexity **14**, 1998, pp. 1–33.
- [69] I. SLOAN AND R. WOMERSLEY; Constructive polynomial approximation on the sphere, *J. Approx. Theory* **103** 2000, pp. 91–118.
- [70] M. TANEMURA AND M. HASEGAWA; *Geometrical models of territory. I. Models for synchronous and asynchronous settlement of territories*, J. Theor. Biol. **82**, 1980, pp. 477–496.
- [71] S. WAN, S. WONG, AND P. PRUSINKIEWICZ; *An algorithm for multidimensional data clustering* ACM Trans. Math. Soft. **14**, 1988, pp. 153–162.

- [72] R. WOMERSLEY AND I. SLOAN; How good can polynomial interpolation on the sphere be?, *Advance in Computational Mathematics* (to appear).
- [73] *Interpolation and Cubature on the Sphere*, <http://www.maths.unsw.edu.au/~rsw/Sphere/>.